



NOVUS

Smart Editorial & Publishing Platform

AY 2026-2027

AUTHORS:

Rei Khalil S. Galido
Roman Rico B. Albania
Joshmar L. Clavero
Joshua Concepcion

CO-AUTHOR:

Carl Dominique Bueno

CONTRIBUTES TO:



INDIRECTLY AIDS:

Table of Contents

Executive Summary	i
List of Figures	ii
List of Tables	iii
Introduction	1
Project Context.....	1
Statement of the Problem	1
Objectives	2
Significance of the Project	3
Scope and Limitations	4
Review of Related Literature / Systems	5
Technical Background.....	8
Current System	8
Proposed System	10
Project Feasibility	12
Requirements Analysis	13
Product Vision	13
Product Backlog / User Stories	14
Use Case Diagram.....	27
Use Case Full Description/Fully Dressed Use Case.....	27
Test Cases based on Fully Dressed Use Case	97
Gap Analysis	97
System Analysis and Design	98
System Context Diagram	98
Container Diagram	99
Component Diagram	100
Entity Relationship Diagram	101
Results and Discussion	102
Design Thinking Stages	102
Stage 1: Empathize	102
PERSONA 1: MARIA	105
PERSONA 2: DANIEL.....	105
PERSONA 3: ROBERT.....	106
PERSONA 4: ALEX	106

Stage 2: Define.....	114
The second phase of the design thinking process is by putting together all the information gathered from the previous phase. This stage is to define the problem statements clearly which should be based on human-centered needs, rather than being focused on the business goal.	114
Release Plan	115
Prototype	116
Use Classes and Characteristics	129
Conclusion	131
References	132
Appendices	133
Appendix A: Project Vision	133
Appendix B: Schedule	134
Appendix C: Product Roadmap	135
Appendix D: Users' Manual	136
Appendix E: Team Meetings.....	137
Appendix F: Source Code.....	138
Appendix G: Finalized Design Thinking Deliverables.....	139

Executive Summary

(

Refer to the following when writing this part of your paper:

[*How to Write an Executive Summary for Your Proposal*](#)

[*How to Write a Compelling Executive Summary*](#)

)

List of Figures

(

Add this to your paper if there you have inserted figures; otherwise, delete this section, including the header

To add a list of figures,

- 1. Ensure that each figure has been properly captioned (Right click → Insert caption)*
- 2. Put the cursor below the **List of Figures** heading, then go to the **References** tab and click on **Insert Tables of Figures** located in the **Captions** group. The caption label should be set to **Figure**.*

)

List of Tables

(

Add this to your paper if there you have inserted figures; otherwise, delete this section, including the header

To add a list of figures,

- 1. Ensure that each figure has been properly captioned (Right click → Insert caption)*
- 2. Put the cursor below the **List of Tables** heading, then go to the **References** tab and click on **Insert Tables of Figures** located in the **Captions** group. The caption label should be set to **Table**.*

)

Introduction

The media industry in the Philippines is undergoing a critical shift. As digital consumption grows and audiences increasingly move away from traditional print and social media-dependent distribution, independent publications face mounting pressure to modernize their operations or risk being left behind [1]. BOSS Media Philippines Inc., the client organization of this project, is one such publication — a Filipino media company producing content that spans local and international events, featured personalities, and cultural issues, yet operating today without any dedicated digital infrastructure to support its editorial, publishing, or revenue functions. All article assignments, submissions, reviews, revisions, and approvals are handled manually across personal messaging applications, with no system, no tracking, and no automation of any kind.

Project Context

BOSS Media Philippines Inc., publisher of BOSS Magazine, aims to scale into a globally recognized Filipino media brand with a sustainable digital revenue model. To achieve this, the organization is developing Novus, a dual-purpose platform designed to solve current operational bottlenecks and drive future growth.

Currently, the core production workflow between the Publisher, Editor, Writers, and Graphic Designer is highly fragmented. The team faces several critical challenges:

- ❑ Informal Channels: Assignments, submissions, and design files are routed through disjointed platforms like Messenger, Viber, and Gmail.
- ❑ Lack of Standardization: There are no formal design or editorial briefs, deadline tracking systems, or centralized review hubs.
- ❑ Zero Pipeline Visibility: The Publisher relies entirely on manual updates from the Editor and lacks real-time oversight of the production process.

Novus will directly address these issues by providing a centralized internal system to standardize the editorial pipeline, alongside a public-facing platform to expand audience reach and monetize their digital content.

Statement of the Problem

BOSS Magazine PH is a comprehensive Filipino lifestyle publication — covering a broad spectrum of topics including fashion, culture, modern trends, business, and society — that currently operates without any dedicated digital infrastructure to support its editorial, publishing, or revenue functions.

I. Fragmented Editorial Workflow and Inability to Scale

The first and most critical problem is the absence of a centralized editorial workflow system, which directly limits the magazine's ability to scale. The sole editor of BOSS Magazine PH manually manages 10 or more article submissions per publishing cycle, receiving drafts through 4 or more different communication platforms including Messenger, Viber, and email simultaneously. This results in 21 to 49 work-related messages per day, forces the editor to spend 31 to 40 hours per week on editorial tasks, and leaves no traceable record of submission history, revision rounds, or approval decisions. Furthermore, as the writer roster grows, this editorial workload grows proportionally with no additional support — resulting in cascade publishing delays where one late article pushes back the entire issue, and a publishing cycle that regularly exceeds 2 weeks to complete.

II. Time-Consuming Manual Content Reviews

The second problem is the lack of an automated content review mechanism. Every article submitted by a writer goes directly to the editor for a full manual review — covering grammar, spelling, tone, brand voice, and structure — before any feedback is given. Because there is no first-pass automated review tool, the editor spends more than 2 hours per day writing and explaining revision notes, and an additional 1 to 2 hours chasing writers for follow-ups and resubmissions. Only 3 to 4 out of every 10 submitted articles require no revision, meaning the majority of submissions add significant manual labor to an already overloaded workflow.

III. No Dedicated Digital Platform or Organic Reach

The third problem is the lack of a functional and reliable public digital presence. While BOSS Magazine PH previously attempted to establish a website using a third-party builder (Webtulz), the platform proved unviable, difficult to maintain, and is currently non-functional. As a result, the magazine has been forced to revert to publishing exclusively through Facebook. This limits the magazine's reach to existing social media algorithms, provides no mechanism to attract new readership organically through search engines, and prevents the publication from being taken seriously as a professional media entity by brand partners or international audiences.

IV. Zero Direct Revenue Generation

The fourth problem is the complete absence of a revenue model. Despite producing consistent editorial content, BOSS Magazine PH generates zero direct revenue from its published output. There is no subscription system, no digital issue sales mechanism, and no paywall — meaning the magazine bears the full operational cost of production with no financial return from its readership.

Objectives

Main Objective

To develop NOVUS, a centralized, AI-assisted digital publishing platform that streamlines editorial operations and introduces a subscription model, enabling BOSS Magazine PH to become a self-sustaining media enterprise.

Specific Objectives

To ensure the system's operational success and feasibility, the development of Novus is guided by the following measurable targets:

1. **Workflow & Capacity:** To streamline the editorial process of BOSS Magazine PH by implementing a centralized pipeline system that can handle a minimum of 50 article submissions per month, ensuring all updates and progress are tracked and visible to the team in one place.
2. **Workload Reduction:** To reduce the time editors spend reviewing each article from an average of 45 minutes to approximately 20 minutes through the use of an AI-assisted pre-screening feature.
3. **Portal Performance:** To develop a reader-facing website for BOSS Magazine PH, meeting standard web performance benchmarks upon launch achieving a good rating having an average of 85 across its core evaluation metrics
4. **Monetization Readiness:** Establish the technical foundation for future digital sales by integrating a payment gateway and successfully validating it through simulated test transactions in a sandbox environment.

Significance of the Project

Primary Beneficiaries

The implementation of Novus directly benefits the core production team and its audience. For the editor, AI-assisted reviews and a centralized dashboard eliminate hours of manual administration, allowing a shift toward high-value strategic editing. The publisher gains real-time pipeline visibility and enforces deadlines while unlocking a critical digital revenue model through subscriptions and sales. Writers transition from fragmented chat handoffs to a dedicated portal, where instant AI feedback on grammar and tone significantly accelerates the revision cycle. Finally, readers benefit from a professional standalone website that offers premium subscriptions, digital purchases, and an interactive AI agent for personalized content discovery.

Improvement to the Current Situation

Novus drastically improves the magazine's current operational, editorial, commercial, and reputational standing. Operationally, it replaces a delayed, manual publishing cycle with a structured, trackable workflow. Editorially, automated AI pre-checks raise the baseline quality of submissions, drastically reducing manual revision rounds. Commercially, the platform transforms a zero-revenue operation into a monetized business with active subscriptions and digital storefronts. Ultimately, Novus elevates BOSS Magazine PH from a social media presence to an authoritative, professional digital publication.

Sustainable Development Goal (SDG) Alignment

SDG 8 — Decent Work and Economic Growth. Novus directly contributes to SDG 8 by enabling BOSS Magazine PH to build a sustainable digital business model that supports the livelihoods of its editorial team, writers, and publisher. By introducing a subscription-based revenue stream, the platform creates a foundation for the magazine to compensate its contributors fairly and grow as a viable media enterprise. Additionally, by reducing the editor's unsustainable 31–40-hour editorial workload through automation, Novus promotes healthier and more equitable working conditions within the organization.

SDG 9 — Industry, Innovation, and Infrastructure. Novus also aligns with SDG 9 by building the digital infrastructure that BOSS Magazine PH currently lacks — transforming an informal, chat-based operation into a technology-enabled publishing system. The integration of AI-assisted editorial review and a reader-facing AI engagement agent represents a direct application of innovation to a real industry challenge, demonstrating how small independent publishers in the Philippines can leverage modern technology to operate at a level previously only accessible to large media organizations.

Scope and Limitations

Scope of Novus

The scope of Novus encompasses a comprehensive, role-based platform designed to manage the entire editorial and publication lifecycle. It features a dedicated writer portal with an integrated AI-assisted review system that automatically evaluates submissions for readability and grammar. This feeds into a centralized editorial dashboard equipped with structured revision management and approval workflows, effectively eliminating the need for external chat applications. For public engagement and monetization, the platform includes a streamlined web interface alongside a flexible global payment architecture to handle digital subscriptions and individual issue sales. Additionally, an admin dashboard provides high-level overview on production workflows and revenue metrics.

Limitations of the Project

Despite its broad functionality, the current iteration of Novus has clear boundaries. It is exclusively a web-based application without native mobile apps, offline functionality, or integration with external content management systems. Within the editorial process, it lacks real-time collaborative editing and advanced detection for AI-generated submissions. Similarly, while the final submission and approval of visual assets occur securely within the platform, graphic designers must still rely on third-party software for the actual creation of these graphics. Furthermore, the system does not handle physical print management, or advertising inventory tracking. These functional constraints are acknowledged as deliberate boundaries for the initial release, serving as opportunities for future expansion as the publication scales.

Review of Related Literature / Systems

This chapter reviews the foundational research, industrial benchmarks, and system architectures relevant to the deployment of Novus: Smart Editorial and Publishing Platform. To satisfy the rigorous technical requirements of systems evaluation and explicitly address baseline defensibility, this review synthesizes overlapping academic and industrial sources. This comprehensive evaluation establishes the empirical basis for the core engineering milestones, validates the machine learning integrations, and contrasts the unified software architecture against disjointed consumer-grade alternatives

A. Editorial Workflow and Content Management Systems

The baseline metrics defining the Novus content pipelines—specifically the target to process a scalable minimum of 50 articles per month and reduce manual review times from an average of 45 minutes down to 20 minutes—are systematically derived from documented newsroom efficiency trends. Analysis by CFlow demonstrates that unstructured content operations experience a 30% drop in absolute productivity when coordination relies on fragmented communication mediums, confirming that 68% of media teams experience severe operational bottlenecks due to a lack of centralized pipeline visibility [2]. However, when automated pipeline management environments are formally implemented, content turnaround times are systematically compressed by up to 50% [2]. Partida further demonstrates that smaller, independent publishing houses frequently suffer from systemic delays because they adopt general-purpose, single-user drafting environments—such as personal messaging accounts and loose word processors—that inherently fail to manage multi-role editorial states [3]. The PubLink framework resolves these tracking deficiencies by processing digital manuscripts as structured objects characterized by explicit metadata states and immutable workflow transitions, ensuring that the migration of content from draft submission to final layout approval is secure, fully auditable, and traceably isolated from standard consumer chat noise [4]. Furthermore, Brechner and Cervone evaluate the cognitive overhead induced when content teams partition their focus across distinct software suites, indicating that visualizing operational streams through an integrated Kanban-style interface independently reduces operational cycle times by 24% [5], [6]. This significant reduction is achieved entirely by eliminating mental friction associated with tracking manuscript statuses across disparate environments.

B. Digital Media Sustainability and Owned Platform Strategy

Transitioning small independent publishers away from volatile third-party social networking distribution networks is a critical requirement for long-term viability. Audience analysis metrics indicate that user reliance on mainstream social network news algorithms has degraded significantly, experiencing drops of up to 11 points in key demographics [7]. This paradigm shift forces independent digital media entities to establish standalone web applications to maintain organic search discovery and directly engage their readership. Industrial economic reporting from Deployteq establishes that 80% of publishers globally are prioritizing reader-supported subscription models as their primary revenue framework, heavily exemplified by prominent modern publications that derive over 60% of total operational profits purely

from gated subscription structures [8]. In emerging digital economies such as the Philippines, transaction execution drop-off rates increase dramatically when localized digital wallet architectures are missing from user checkouts. Oliveira et al. demonstrate that ensuring direct native access to prevailing regional payment methods—validating the integration of platforms like PayMongo—is vital for maximizing financial conversion behaviors [9]. Furthermore, Radu verifies that content platforms integrating automated billing engines retain consumer subscriptions at rates 34% higher than operations utilizing manual invoicing or external transaction links [10]. This retention performance occurs because automated billing systems significantly lower user execution effort, a factor that Hamari, Hanner, and Koivisto identify as the single most critical driver governing freemium subscription renewals [11].

Algorithmic Calibration: Psycholinguistics, Readability, and Threshold Defense

To prove that the automated first-pass screening framework is scientifically grounded rather than arbitrary; its evaluation metrics are explicitly derived from established readability formulas and psycholinguistic research. The engine evaluates content using a strict 60/40 weighted matrix with an unyielding 75% passing threshold. The dominant 60% weight allocated to grammatical accuracy acts as a defensive brand measure. Pitler and Nenkova demonstrate that human readers process typos and mechanical syntax breakdowns through fast, subconscious cognitive reflexes that instantly trigger negative author assessment and destroy a publication's perceived credibility [12]. Because grammar compliance is inherently objective, it can be mathematically quantified with high confidence, ensuring that poorly proofread drafts are rejected immediately. Conversely, the 40% readability allocation acts as a critical optimization defense against gradual reader abandonment. As established by Dale and Chall, modern digital audiences primarily scan text patterns, and content that routinely forces readers to exceed standard cognitive memory limits results in page abandonment [13]. Novus incorporates the Flesch Reading Ease calculation to guarantee text tracks tightly within a target 60 to 70 range, optimizing content for digital reader consumption without altering semantic intent. The mathematical decision to set the passing threshold at exactly 75% reflects established data within Automated Essay Evaluation structures. Koppel, Schler, and Argamon verify that professional human editors evaluating complex article narrative standards exhibit an inter-rater reliability agreement rate varying between 75% and 80% [14]. Forcing the platform baseline above 80% introduces severe algorithmic rigidity, causing the system to falsely reject creative formatting or advanced narrative journalism. Conversely, setting the threshold lower than 75% allows text to pass even if it completely fails one of the core quality pillars. To account for high-quality writing that relies on deliberate colloquialisms or stylistic non-compliance, the platform integrates a Human-in-the-Loop (HITL) bypass mechanism, allowing authorized editors to manually override a low AI score, thereby protecting human editorial control and continuously refining the machine learning prompt criteria.

Modular Monolith Architectures

Recent literature advocates for the modular monolith as a pragmatic default for enterprise applications, particularly in domains like digital publishing where bounded contexts (e.g., editorial, subscription, analytics) benefit from clear internal boundaries without incurring the operational overhead of distributed systems. A modular monolith preserves the deployability and transactional simplicity of a single codebase

while enforcing separation of concerns through well-defined module interfaces, enabling independent evolvability and team autonomy within a unified deployment unit [15]. This architecture is especially advantageous for platforms with fewer than 50 engineers and no extreme scaling disparities between modules, as it avoids the distributed-system tax of network latency, eventual consistency, and complex orchestration manifests. For NOVUS, a modular monolith allows the editorial, subscriber, and public domains to evolve independently while maintaining ACID-compliant transactions across user roles and content workflows, deferring microservices extraction until specific modules (e.g., AI inference) require independent scaling or technology stacks.

Decoupled Web Frameworks

The integration of reactive frontends such as Vue.js with robust backend APIs like Django REST Framework (DRF) exemplifies the decoupled architecture pattern prevalent in modern full-stack development. Vue.js, leveraging its Composition API and component-based model, provides a reactive user interface that communicates with DRF endpoints via Axios or similar HTTP clients, enabling seamless data flow between frontend state and backend resources. DRF facilitates this decoupling by exposing RESTful APIs with built-in authentication, serialization, and pagination, while CORS middleware ensures cross-origin requests from the Vue development server are securely handled, [16]. This separation allows frontend and backend teams to iterate independently, with Vue managing UI/UX concerns (e.g., role-based views, client-side routing) and Django handling domain logic, persistence, and security. For NOVUS, this architecture supports distinct public, subscriber, and editorial interfaces consuming a unified API layer, ensuring consistent data access while enabling modular UI development,.

Technical Background

The current system of BOSS Media Philippines Inc. operates entirely without dedicated software or digital infrastructure. All editorial and publishing processes are carried out through a combination of consumer-grade communication applications and general-purpose productivity tools. The primary platforms in use include Facebook Messenger, Viber, WhatsApp, Gmail, and Google Docs, which serve simultaneously as briefing channels, submission inboxes, revision communication tools, and publication coordinators. No editorial management software, content management system, project tracking tool, or publishing platform has been formally implemented.

In terms of hardware, the team operates using personal computers and mobile devices owned by individual staff members and writers. There is no shared server, internal network, or cloud workspace dedicated to editorial operations. File storage and version control are handled informally through Google Drive folders and shared Google Docs links, with no naming convention, folder structure, or access control policy in place.

The graphic designer works using Canva and other design applications on personal devices, with no shared design system or file handoff protocol. The publisher manages the physical printing and social media distribution manually, coordinating through the same consumer messaging platforms used throughout the rest of the workflow.

Current System

Table 1 lists all current processes in the editorial and publishing workflow of BOSS Media Philippines Inc. Each process is identified by a unique process ID, a process name, and a description of how the process is currently performed. These processes were documented through direct data gathering with the Editor and Publisher and validated through the writer survey conducted in March 2026.

TABLE I.
List of Processes in the Current System of BOSS Media Philippines Inc.

Process ID	Process Name	Process Details
P001	Topic Assignment	The Editor assigns a topic or story angle to a Writer through Facebook Messenger, Viber, Gmail, or a verbal conversation. No standardized brief template or deadline system is used.
P002	Article Writing	The Writer drafts the article using Google Docs or a notes application. The Writer determines formatting independently as no writing guide is provided.
P003	Article Submission	The Writer submits the completed article through the same fragmented communication channels used for the assignment. No centralized submission portal exists.

P004	Editorial Review	The Editor manually reviews the submitted article for grammar, tone, structure, and brand alignment. This takes one to two days per article, with 61 to 80 percent of submissions requiring at least one revision.
P005	Feedback & Revision	The Editor sends revision notes through private messaging. The Writer revises and resubmits within three to five days on average. This loop repeats for two to three rounds per article.
P006	Article Approval	The Editor approves the final version and informs the Writer through chat. Only three to four out of every ten articles are approved on the first submission.
P007	Layout Production	Approved articles are forwarded to the Graphic Designer through chat or email. The Designer arranges the magazine layout with no formal design brief or file-sharing system.
P008	Layout Review	The completed layout is sent back to the Editor for a final check. Revision notes are relayed again through informal messaging.
P009	Publishing	Once the layout is confirmed, the Publisher receives the final files and manually publishes the issue to social media platforms and coordinates physical printing. No publishing schedule system is used.

SWOT Analysis

The SWOT analysis below evaluates the internal strengths and weaknesses of BOSS Media Philippines Inc. as an organization, as well as the external opportunities and threats it faces in the current Philippine media landscape. Each point is derived from the data gathering conducted with the editorial team and from the broader industry context established in the literature review.

TABLE II.
SWOT Analysis of BOSS Media Philippines Inc.

Strengths	Weaknesses
• Established publication identity with a Filipino-focused editorial voice and brand • Dedicated and committed core team willing to improve the current workflow • Existing readership base on Facebook and Instagram • Content covers a broad range of topics increasing discoverability potential	• No centralized editorial workflow or project tracking system in place • Single editor handling 10+ articles per cycle with no automation support • 61–80% of articles require revision; only 3–4 out of 10 are approved on first submission • Zero direct revenue generation from published content • No formal publishing schedule; delays regularly exceed 2 weeks
Opportunities	Threats
• Filipino Facebook news reliance dropped 11 points in 2024, creating demand for owned platforms (Newman et al., 2024) • 80% of publishers globally are investing in subscription models as a primary revenue stream (Deployteq, 2024)	• Continued reliance on Facebook risks further reach reduction as algorithm changes deprioritize media content • Growing competition from digital-native Filipino media brands with existing platforms and revenue models

- | | |
|--|---|
| <ul style="list-style-type: none"> • Commercial LLM APIs now make AI-assisted editorial review accessible to small publications • Growing Filipino internet penetration and mobile-first consumption opens wider digital audiences | <ul style="list-style-type: none"> • AI-generated content flooding the media space raises audience trust concerns across all publications • Operational burnout risk — single editor handling a growing workload with no system support |
|--|---|

Proposed System

A. Technical Background & Architecture

Novus is a cloud-native editorial management system deployed on a unified PaaS environment. It refactors traditional multi-platform configurations into a single relational database that handles transactional user data, active document states, and append-heavy audit logs using native text-field features. The platform consists of a modern web frontend and incorporates four core feature pillars:

- **Visual Pipeline Tracking:** A dashboard replaces fragmented communication channels, providing centralized status visibility across all article submission stages.
- **Unified AI-Powered Editorial Assistant:** Uses a single, high-tier generative language model API restricted to structured JSON output to evaluate grammar and extract editorial briefs simultaneously, optimizing token usage and reducing latency.
- **Targeted Notification Protocol:** Employs a server-side, API-driven email service exclusively for transactional messages, deadlines, and state transitions to prevent notification fatigue.
- **Streamlined Payment Infrastructure:** Integrates a single localized payment gateway dedicated to domestic e-wallets and local cards.

B. First Pass Review: Scoring Framework (Writer Side)

Articles undergo an automated pre-screening to achieve a minimum composite score of 75% before advancing to editorial review. Submissions falling below this threshold are automatically returned for revision.

- **Readability (40% weight):** Evaluates audience consumption ease via the Flesch Reading Ease formula, targeting an 8th to 9th-grade reading level (scores between 60 and 70).
- **Grammar (60% weight):** Assesses grammatical accuracy, applying a flat 5-point deduction for each identified error.
- **Human-in-the-Loop (HITL) Override:** Authorized editors can manually bypass failing grades to accommodate intentional stylistic choices without algorithmic restriction, providing feedback data to refine future grading criteria.

C. Contextual Narrative Analysis (Editor's Side)

Submissions passing the initial threshold trigger a secondary background AI evaluation that generates a private Editor's Brief attached to the article record, accessible only to editors and publishers across three pillars:

- **Hook and Reader Engagement:** Analyzes introductory paragraphs for clear main points and stakeholder hooks.
- **Story Flow and Logic:** Traces structural coherence to flag timeline jumps, missing background data, or weak transitions.
- **Targeted Content Extraction:** Pulls out the **Marketing Highlight** (standalone promotional sentences for social media/design teams) and the **Editorial Bottleneck** (the most confusing paragraph requiring human rewrite skills).

D. Change Management and User Training

To ensure platform adoption, the implementation includes:

- **Onboarding Session:** Live walkthrough demonstrations for all platform users.
- **Digital Handbook:** Reference documentation detailing submission formats and revision protocols.
- **Pilot Phase:** Running Novus alongside existing workflows for at least one issue prior to full migration.

E. Evidence Collection & Validation Framework

The platform evaluates its operational, financial, and technical objectives using a multi-tiered data-collection framework:

- **Time-in-Status Tracking:** Logs chronological duration per pipeline stage to quantify reductions in review times and measure publication throughput.
- **Training Effectiveness:** Tracks post-training user error rates to validate digital onboarding materials.
- **Automated Override Rate:** Monitors HITL override frequencies against legacy baseline approvals to calibrate pre-screening algorithms.
- **System Usability:** Uses standardized surveys to measure cognitive load reduction compared to disjointed legacy messaging apps.
- **Revenue vs. Cost Analysis:** Tracks subscription revenue against unified cloud hosting and API expenses to prove economic sustainability.
- **Technical Audits:** Executes automated web performance tests to verify SEO, accessibility, and speed benchmarks.

To develop and deploy Novus, we have selected technologies including:

- **Vercel:** A platform that supports seamless deployment and global hosting services for our Vue.js frontend, providing automatic edge-caching, continuous integration, and secure SSL.
- **Render:** A modern cloud platform utilized to host our Django backend API and managed PostgreSQL database, offering automated builds and scalable infrastructure without heavy operational overhead.
- **Cloudflare R2:** An S3-compatible object storage solution chosen for the cost-effective and highly available storage of dynamic website assets, article media, and user uploads.

- **Vue.js:** A progressive JavaScript framework selected for the frontend to manage dynamic client-side rendering, user interfaces, and role-based routing as a Single Page Application (SPA).
- **Django & Django REST Framework:** A high-level Python web framework chosen for the backend API to enforce centralized business logic, securely manage token-based authentication (SimpleJWT), and expose robust RESTful endpoints.
- **PostgreSQL (SQL):** An advanced relational database ideal for ensuring data integrity and storing core platform state, such as user profiles, editorial articles, and subscription ledgers.
- **PayMongo:** A unified payment gateway integrated to securely process localized e-wallet transactions and credit card subscriptions through a streamlined API.
- **Resend:** A developer-focused email service utilized to reliably construct and dispatch transactional communications, including system notifications and digital receipts.
- **Anthropic Claude API:** An advanced artificial intelligence engine integrated to evaluate editorial submissions and generate deterministic narrative briefs utilizing structured outputs.

Our tech stack choices prioritize solutions that enforce a secure, decoupled architecture, optimize the editorial workflow, and provide the essential scalability, performance, and maintainability required for a modern digital publication.

Project Feasibility

Operational Feasibility

The operational feasibility of the Novus platform is highly promising, as it directly resolves the critical pain points identified in recent stakeholder surveys. To address disjointed workflows and unclear timelines, the system introduces centralized editorial pipelines with automated draft status transitions, ensuring complete transparency for writers, editors, and managers. Furthermore, communication gaps and fragmented ideation are mitigated by embedding contextual editorial feedback and deterministic AI-generated scoring directly into the review process before human editors intervene. Finally, the shift to a cloud-native, browser-based workspace eliminates physical and operational friction, allowing the editorial team to collaborate seamlessly regardless of their location, operating system, or hardware limitations.

Economic Feasibility

From a financial perspective, the project demonstrates exceptional viability through its strategic reliance on cost-effective, modern infrastructure. The initial development and staging phases require zero upfront capital expenditure, leveraging the generous free tiers provided by Vercel, Cloudflare R2, Render, and Resend. As the platform transitions into a live production environment, the baseline operational expenditure remains highly predictable and remarkably low, starting at approximately PHP800 per month for managed compute and database hosting. Additionally, external integrations like PayMongo and the Anthropic Claude API utilize strict pay-as-you-go models, ensuring that infrastructure costs scale strictly in proportion to user adoption and subscription revenue generation, thereby securing a strong return on investment.

Technical Feasibility

The technical foundation of Novus is built upon stable, highly supported frameworks, ensuring a high degree of technical feasibility. By utilizing a decoupled architecture consisting of a Vue.js frontend and a Django REST Framework backend, the system guarantees a strict separation of concerns, which enhances both long-term maintainability and independent scaling. Data integrity is secured through PostgreSQL, which handles complex relational data—such as subscription ledgers and user profiles—while utilizing native JSON capabilities to store dynamic AI-generated briefs with ACID-compliant reliability. Furthermore, delegating specialized functions to established enterprise APIs drastically reduces custom development overhead and mitigates the risks associated with infrastructure maintenance.

Schedule

Feasibility

The development of Novus is strategically phased across three academic terms under the Project-Based Learning curriculum, ensuring a modular and low-risk delivery:

- **Phase 1 (MNTSDEV):** Focuses on system design, requirement elicitation, and the development of the initial high-fidelity prototypes for the editorial dashboard.
- **Phase 2 (MSYSADD):** Dedicated to technical modeling, including UML diagrams and the architecture of the RAG pipeline and unified database schema.
- **Phase 3 (MCSPROJ):** Encompasses the final coding phase, unified API integration, rigorous system testing, and deployment to the production environment.

Requirements Analysis

Product Vision

The vision for Novus is to democratize professional-grade editorial infrastructure for independent publishers worldwide, transforming fragmented content production into a streamlined, high-output operation. By centralizing the editorial pipeline — from initial assignment to final approval — Novus aims to eliminate the administrative friction that typically stifles small-to-medium media houses. Through the integration of AI-assisted review systems, the platform empowers editorial teams to maintain rigorous standards without the need for massive human headcount, ensuring that even the smallest publications can produce polished, content at the speed of modern digital discourse.

Beyond operational efficiency, Novus envisions a future where creators reclaim control over their financial destiny through owned platforms and direct reader engagement. The mission is to move publishing away from total reliance on social media algorithms and toward a sustainable 'freemium' ecosystem that converts passive scrollers into invested subscribers. By providing built-in, secure hybrid payment architectures and interactive AI discovery tools, Novus equips publishers with the professional digital presence and monetization strategies required to turn a labor of love into a profitable international brand, ensuring the long-term survival and independence of diverse voices in the global media landscape.

Project Lean Canvas

The Lean Canvas summarizes the key strategic elements of Novus, including the identified problem, proposed solution, unique value proposition, unfair advantage, customer segments, key metrics, channels, cost structure, and revenue streams.

Problem - Editor manually reviews 10+ articles per cycle with no automation – single point of failure - No centralized system for submission, revision tracking, or deadline management - BOSS Magazine PH has no website and no revenue stream from its published content	Solution - AI-assisted first-pass review for grammar and readability - Centralized editorial dashboard for submissions, revisions, approvals, and deadline tracking - Public magazine website with subscription and digital issue sales	Unique Value Proposition - The only all-in-one editorial and publishing platform built for independent Filipino magazines - Manage writers, enforce deadlines, publish professionally, and earn from contents without juggling chat threads	Unfair Advantage - Built on real validated client data – not hypothetical - First AI editorial system deployed in local independent PH magazine publishing - Direct client access enables faster iteration	Customer Segments - BOSS Magazine PH editor and publisher – primary users - Writers submitting content to the magazine - Filipino fashion and culture readers – subscribers and digital issue buyers
Existing Alternatives - Messenger, Viber, email for article submission and revision coordination - Google Docs for drafting with no submission or review system - The only platforms that highlight BOSS magazine's presence are Facebook, Instagram, and a non-operational WebToolz website - No subscription or payment system – all content free	Key Metrics - Publishing cycle time reduced from 2+ weeks - Editor hours saved per week from 31-40 hours - Messages reduced from 21-40 - Number of active reader subscriptions - AI review accuracy - Revenue generated per publishing cycle	High Level Concept - Novus is Notion + Substack + AI Editor for independent Filipino magazine publishers - One platform from brief to publish to revenue	Channels - Direct deployment at BOSS Magazine PH as first live client - BOSS Magazine PH social media driving readers to the platform - Word of mouth in the local independent publishing community	Early Adopters - BOSS Magazine PH editor and publisher - Current BOSS Magazine PH writer roster - Existing BOSS Magazine PH social media followers who become first subscribers
Cost Structure - AI API Fees: Variable usage costs for LLM-powered editorial reviews - Cloud & Storage: Monthly costs for app hosting, databases, and digital media storage - Payment Processing: Transaction-based fees for subscriptions and digital sales - Domain & Security: Fixed annual costs for the domain, SSL, and professional email		Revenue Streams - Monthly reader subscriptions – recurring access to premium magazine content - Annual reader subscription plan – yearly rate - Single digital issue purchases – one-time buy per magazine issue - Future: platform licensing to other small publishers who adopt		

Product Backlog / User Stories

TABLE VI.

PRODUCT BACKLOG

Persona	Story Title	User Story	Acceptance Criteria
Writer	Login and Role-Based Redirect	As a Writer, I want to log in and be automatically directed to my Writer Dashboard, so that I can access my workspace without manual navigation.	- Writer opens Novus and lands on the Login / Role Landing screen. - Writer enters registered email and password, then taps "Log In." - If invalid, system shows a pop-up "Incorrect email or password. Please try again." Writer remains on the Login screen with the password field cleared. - If valid, system shows "Login Successful" and navigates to the Dashboard Redirect screen, confirming the role "Writer."

			5. System automatically navigates the Writer to the Writer Dashboard within 2 seconds — no further action is needed.
Writer	Viewing the Writer Dashboard	As a Writer, I want to see my active, in-revision, and approved articles as soon as I log in, so that I know what needs my attention.	1. On arrival, system displays three counters: Active, In Revision, Approved, and a task list of assigned/drafted articles with Title, Status Badge, and Deadline. 2. Writer taps “+ New Submission” → system navigates to a blank Article Submission Form. 3. Writer taps an existing “Drafting” article → system navigates to the Article Submission Form pre-filled with the saved draft. 4. Writer taps an article marked “Needs Revision” → system navigates to the Revision Notes Inbox. 5. No confirmation pop-up is required here since these are navigation-only actions.
Writer	Viewing the Pinned Assignment Brief	As a Writer, I want my assignment brief pinned to my writing screen, so that I always know the topic, word count, tone, and deadline without leaving the editor.	1. When the Article Submission Form opens for an assigned topic, system displays a collapsible “Assignment Brief” panel. 2. Panel shows Topic, Word Count Target, Tone, and Deadline as read-only fields. 3. Writer taps the collapse icon to show/hide the panel; it remains visible while scrolling the article body. 4. Writer cannot edit any field in the panel — tapping into it produces no cursor and saves no changes. 5. This panel persists through drafting, auto-save, and submission.
Writer	Drafting an Article with Cloud Auto-Save	As a Writer, I want my draft to save automatically in the cloud as I type, so that I never lose progress.	1. Writer enters Article Title, selects Category, enters Word Count, and types the body. 2. System auto-saves every 30 seconds of activity, or immediately after 3 seconds of typing pause. 3. On successful save, system shows “Saved to Cloud” near the top of the

			editor, fading after 2 seconds; Writer remains on the same screen. 4. If the network drops, system shows a persistent banner “Offline — changes will sync once reconnected” and keeps saving locally. 5. Once reconnected, system auto-syncs, briefly shows “Synced to Cloud” for 2 seconds; Writer remains on the same screen. 6. Writer may tap “Save Draft” manually at any time — same confirmation applies, no navigation change.
Writer	Submitting an Article for AI Pre-Screening	As a Writer, I want to submit my finished draft for automated review, so that I get an initial quality score before the Editor sees it.	1. Once drafting is complete, Writer taps “Submit for Review.” 2. System shows a confirmation pop-up “Submit this article for AI review? This will lock further edits until scoring is complete.” with Cancel / Confirm. 3. Writer taps “Cancel” → pop-up closes, Writer remains on the Article Submission Form, no status change. 4. Writer taps “Confirm” → system changes status to “In Review,” shows “Article submitted successfully,” and navigates to the AI Pre-Screen Result screen.
Writer	Viewing the AI Pre-Screen Result	As a Writer, I want to see my grammar and readability score right after submitting, so that I know whether my article advances or needs revision.	1. System displays the composite score out of 100 (Grammar + Readability breakdown) within 60 seconds of submission. 2. If score is 75% or higher, system shows “Your article passed pre-screening and has been sent to the Editor.” Writer taps “Continue” → returns to the Writer Dashboard. 3. If score is below 75%, system shows “Your article did not meet the minimum score and has been returned for revision.” Writer taps “Revise Now” → navigates to the Article Submission Form with flagged sentences highlighted, or taps “Later” → returns to the Writer Dashboard.

Writer	Tracking Submission Status	As a Writer, I want to track my article's progress through the pipeline, so that I always know its stage without asking the Editor directly.	1. Writer taps "View Status" on a submitted article → opens the Submission Status View. 2. Screen shows a vertical tracker: Submitted → AI Review → Editor Review → Revision Requested → Approved → Published, with the current stage highlighted. 3. Writer taps "Back" → returns to the Writer Dashboard; no confirmation pop-up needed. 4. When the stage changes, system adds a notification badge; tapping it navigates directly to the relevant screen.
Writer	Responding to Revision Notes and Resubmitting	As a Writer, I want the Editor's revision notes attached directly to my draft, so that feedback is clear and I can resubmit once I've made changes.	1. Writer opens the Revision Notes Inbox; system displays each comment beside its corresponding paragraph, with Editor name and timestamp. 2. Writer taps "View Full Article" → opens the Article Submission Form with the comments shown inline. 3. Writer edits the flagged sections; auto-save applies throughout. 4. Writer taps "Mark Resolved" on an addressed comment → system shows "Marked as Resolved" and collapses that comment; Writer remains on the same screen. 5. Once all comments are resolved, Writer taps "Submit Revised Draft." 6. System shows a confirmation pop-up "Resubmit this article to the Editor?" with Cancel / Confirm. 7. Writer taps "Confirm" → system shows "Revised article resubmitted successfully," changes status to "In Review," and navigates the Writer back to the Writer Dashboard.
Writer	Receiving Deadline Reminders	As a Writer, I want visual and automated reminders for my due dates, so that I don't miss the deadline.	1. On the Writer Dashboard, each active article shows a color-coded deadline badge: Yellow (< 48 hrs), Red (< 24 hrs). 2. System automatically sends an email reminder 24 hours before the deadline. 3. System sends an in-app notification 1 hour before the deadline; tapping it

			navigates directly to the Article Submission Form. 4. If the deadline passes without submission, system flags the article “Overdue” and moves it to the top of the task list.
Editor	AI Pre-Screening Engine	As an editor, I want the system to check articles before I read them, so I only spend time on good submissions.	1. Writer sends in an article. 2. The system checks it for errors and copied content. 3. The system gives a result — pass or fail. 4. Articles that copied too much content are automatically turned down. 5. The editor gets a simple report showing what was flagged. 6. The editor can still accept a turned-down article if needed, with a short explanation.
Editor	Sprint Board	As an editor, I want to see all articles and where they are in the process, so I always know what is going on.	1. Editor opens the board. 2. The board shows all articles sorted by their current stage. 3. Each article shows the title, the writer, and the due date. 4. Editor can move articles from one stage to another. 5. Editor can search or filter articles by writer or due date.
Editor	Approvals and Rejections	As an editor, I want to approve or send back an article with one click, so the process is quick and easy to track.	1. Editor finishes reading an article. 2. Editor clicks Approve to mark it as done and lock it from further changes. 3. Or, editor clicks Send Back to return it to the writer for changes. 4. Editor writes a short note explaining what needs to be fixed. 5. The system saves a record of what was decided and when.
Editor	Reassignment and Priority	As an editor, I want to move an article to a different writer or mark it as urgent, so nothing gets stuck or delayed.	1. Editor opens an article. 2. Editor picks a different writer from the list to take over.

			3. The new writer gets notified right away. 4. Editor can mark the article as urgent so it stands out. 5. The change is saved so everyone can see what happened.
Graphics Designer	File Submission	As a graphic designer, I want a place to upload my work, so it is always connected to the right article.	1. Designer opens the upload page. 2. Designer chooses which article the files belong to. 3. Designer uploads the image or layout files. 4. The system confirms the upload was successful. 5. The editor gets notified that new files are ready to review.
Graphics Designer	File Versions	As a graphic designer, I want to upload updated versions of my work without losing the old ones, so the editor can see the changes clearly.	1. Designer uploads a new version of a file. 2. The system keeps the old version and saves the new one separately. 3. Both the designer and editor can view all versions. 4. The editor can leave feedback on any version. 5. The designer marks one version as the final one when ready.
Publisher	Publishing and Deadlines	As a publisher, I want to set issue deadlines and batch-publish approved articles, so that I can automatically release complete issues without manual publishing errors.	1. Publisher opens 'Issue Management', selects a target release date via datetime picker, and clicks 'Save', which updates the global issue release date and syncs the countdown timer across all user dashboards. 2. Clicking 'Publish Issue' triggers an automated system check on all linked articles; if any article is in 'Draft' or 'Review' status, the system aborts execution and displays an error modal listing the unapproved articles. 3. When all articles are in 'Approved' status, confirming the prompt executes a database batch update setting all article statuses to 'Published' and populating published timestamps simultaneously.

			4. Upon completion, the system updates the issue status to 'Live', displays a success modal with the total count of published articles, and provides a direct public preview link.
Publisher	Financial and Subscription Dashboard	As a publisher, I want a real-time revenue and subscription dashboard, so that I can track Monthly Recurrent Revenue (MRR) and platform monetization trends.	1. Upon loading the financial dashboard, the system queries backend APIs to display core summary metrics: Total Active Subscribers, Current MRR, and Recent Transactions. 2. Selecting a date filter ('Weekly', 'Monthly', or 'Yearly') issues an API query that dynamically re-renders the 12-month subscriber growth chart and transaction table without a full page reload. 3. When payment channel triggers a 'charge.paid' or 'subscription.created' webhook, the server validates the payload signature, records the transaction, and pushes dynamic updates to the dashboard view within 3 seconds. 4. System automatically aggregates daily subscription totals every 24 hours at 00:00 UTC to maintain rapid query speeds for historical data.
Publisher	Editorial Workflow and Assignment Tracking	As a publisher, I want to assign articles to specific editors and set section deadlines, so that I can manage staff workload and ensure balanced issue coverage.	1. Publisher opens 'Editorial Workspace', views unassigned draft submissions and selects an assigned Editor from a dropdown menu. 2. System updates the article assignment record, changes status to 'In Review', and sends an automated notification to the designated Editor's dashboard. 3. Publisher sets individual section deadlines (e.g., Feature, News, Opinion), which automatically calculates and displays workload progress bars across all active team members. 4. Publisher receives a real-time status alert if any assigned article remains untouched within 24 hours of its section deadline.
Admin	Role-Based Access Control (RBAC)	As an Admin, I will first log in using my administrator account. After successfully logging in, I will click User Management from the	1. Admin can log in and access the User Management page.

		dashboard menu to open the user management screen. I will select a user from the list, then click Edit Role to change the user's role (Writer, Editor, or Publisher etc.) or click Suspend Account if necessary. After clicking Save or Confirm, I want the system to immediately update the user's permissions so they can only access the menus and features assigned to their role.	2. Admin can create, edit, suspend, and reactivate user accounts. 3. Admin can assign or change a user's role (Writer, Editor, Publisher). 4. Users immediately receive their updated permissions without logging out. 5. A confirmation dialog appears before suspending an account. 6. After saving changes, a success notification (e.g., User role updated successfully.) appears. 7. The system remains on the User Management page and automatically refreshes the user list. 8. Every account creation, role update, suspension, or reactivation is recorded in the audit log.
Admin	API Token Monitoring	As an Admin, I will log in and open the Admin Dashboard. I will click the API Monitoring menu to view the API usage screen. I want to see the total API requests, monthly usage, estimated cost, and remaining quota for each AI service. If usage approaches the monthly limit, I want the system to notify me so I can take action before exceeding the budget.	1. Admin can open the API Monitoring page from the dashboard. 2. Dashboard displays total API requests, remaining quota, monthly usage percentage, and estimated cost. 3. API usage is grouped by service (e.g., OpenAI, Gemini, etc.). 4. When usage reaches 90% of the monthly limit, a warning notification is displayed. 5. When usage exceeds the limit, an alert notification appears on the dashboard. 6. Dashboard data refreshes automatically or through a Refresh button. 7. If the page refreshes successfully, a notification confirms that the latest data has been loaded.
Admin	Global Settings and Maintenance	As an Admin, I will log in and open the Settings menu from the dashboard. I will select Global Settings to access the maintenance settings screen. I will enable or disable Maintenance Mode using a toggle switch or schedule a start and end time for automatic activation.	1. Admin can open the Global Settings page. 2. Admin can enable or disable Maintenance Mode using a toggle switch.

		After saving the changes, I want the website to immediately display the maintenance page to public visitors while allowing internal users to continue accessing the system.	3. Admin can schedule automatic start and end times for maintenance. 4. A confirmation dialog appears before enabling Maintenance Mode. 5. After clicking Save, a success notification (e.g., Maintenance settings saved successfully.) is displayed. 6. Public users immediately see the "Coming Soon" page when maintenance is active. 7. Internal users (Writers, Editors, Publishers) can still log in and access the backend. 8. After saving, the system stays on the Global Settings page and displays the updated configuration.
Reader	Landing on the Public Homepage	As a Reader, I want a clear homepage when I visit the site, so that I can immediately see featured and recent content.	1. Reader navigates to the BOSS Magazine PH website URL. 2. System loads the Homepage: hero article banner, category navigation bar, and a grid of latest articles. 3. No login is required to view this screen. 4. Reader taps a category → proceeds to the Article Listing Page, or taps an article → proceeds to the Single Article View, or taps the language toggle.
Reader	Browsing Articles by Category	As a Reader, I want to filter articles by category, so that I can find content relevant to my interests.	1. Reader taps a category tag from the Homepage or nav bar. 2. System navigates to the Article Listing Page, showing only articles tagged with that category. 3. Reader may type a keyword into the search bar; system filters results in real time and shows "No results found" if nothing matches. 4. Reader taps an article card → proceeds to the Single Article View, or taps "Back" → returns to the Homepage with scroll position preserved.
Reader	Reading an Article and Encountering the Paywall	As a Reader, I want to read an article's content and be clearly told if it requires a subscription, so that I understand what's free versus paid.	1. Reader taps an article card from the Homepage or Article Listing Page.

			2. System navigates to the Single Article View and renders the headline, byline, and body. 3. If the article is free-tier, the full body renders with no interruption. 4. If the article is premium, system renders the headline and first two paragraphs, blurs the rest, and shows a “Subscribe to read the full article” button. 5. Reader taps “Subscribe” → proceeds to the Subscription Page, or taps “Back” → returns to the previous screen with no state change.
Reader	Registering for an Account	As a Reader, I want to create an account, so that I can proceed to subscribe for premium access.	1. Reader taps “Register” from the nav bar or the paywall prompt. 2. System navigates to the Registration screen with fields for Full Name, Email, and Password. 3. Reader fills in the fields and taps “Create Account.” 4. If any field is invalid, system shows inline error text and keeps the Reader on the Registration screen. 5. If valid, system shows “Account created successfully! Welcome to BOSS Magazine PH” and automatically logs the Reader in. 6. System navigates the Reader to the Subscription Page, since registration alone does not unlock paywalled content.
Reader	Selecting a Subscription Plan	As a Reader, I want to choose between a monthly or annual plan, so that I can pick what fits my budget.	1. Subscription Page shows two plan cards: “Monthly” and “Annual” (Annual marked “Best Value”). 2. Reader taps a plan card → system highlights it and updates the “Continue” button to reflect the chosen plan and price. 3. Reader taps “Continue” → system navigates to the Payment Checkout screen, carrying over the selected plan.

			4. Reader may instead tap “Back” → returns to the previous screen with no charge and no confirmation pop-up.
Reader	Completing Payment and Becoming a Subscriber	As a Reader, I want to check out securely with my preferred payment method, so that I can unlock premium content immediately.	1. Reader selects a payment method (GCash, Maya, Credit Card, or Debit Card) and enters the required details. 2. Reader taps “Pay and Subscribe”; system processes the transaction through the payment gateway. 3. If payment fails, system shows “Payment unsuccessful. Please try again or use a different method.” Reader remains on the Checkout screen; sensitive card fields are cleared. 4. If payment succeeds, system shows “Payment successful! Your subscription is now active” and upgrades the account from Reader to Subscriber. 5. System sends a payment confirmation email with receipt and subscription details. 6. System automatically navigates the newly upgraded Subscriber back to the article originally attempted, now fully unlocked.
Reader	Subscribing to the Email Newsletter	As a Reader, I want to opt into the email newsletter, so that I receive updates without checking the site manually.	1. Reader scrolls to the newsletter signup section on the Homepage or footer. 2. Reader enters an email address and taps “Subscribe to Newsletter.” 3. If the email format is invalid, system shows inline error text and keeps the Reader on the same screen. 4. If valid, system shows “You’re subscribed! Check your inbox for a welcome email.” and the field updates to show “Subscribed.”
Subscriber	Paywall Bypass	As a subscriber, I want my active account to instantly unlock premium articles, so that I can read paid content seamlessly across all my devices.	1. When a user visits a premium article in URL, backend middleware verifies their session token and checks their account subscription status. 2. If subscription status is 'Active', the server renders the complete article body with no blur overlays or truncation.

			3. Authenticating with active subscriber credentials on any secondary device or browser immediately grants full reading permissions via session validation. 4. If status is 'Expired' or 'Unauthenticated', the system truncates text at 200 words, applies a blur overlay to the remaining text, and presents a 'Renew/Subscribe to Continue Reading' CTA.
Subscriber	Bookmark and Save	As a subscriber, I want to save specific articles into custom folders, so that I can organize and re-read them later from my profile.	1. Clicking the 'Save' icon on an article opens a selection popover displaying existing user folders and a '+ New Folder' input field. 2. Selecting a folder or creating a new one sends a POST request to save the record to the database, changing the save icon to an active filled state and showing a success toast. 3. Saved items populate under their corresponding folder categories in the 'My Library' tab on the profile page. 4. Clicking 'Remove' on a saved article issues a DELETE request, animating the item out of the DOM and immediately removing it from the user's library list.
Subscriber	Subscription Management	As a subscriber, I want to view my billing details and manage auto-renewal, so that I retain full control over my recurring payments.	1. Navigating to 'Account Settings > Billing' displays the current subscription tier, status ('Active', 'Past Due', or 'Canceled'), and next renewal date. 2. Clicking 'Cancel Subscription' opens a retention prompt offering alternative choices ('Pause for 1 Month', 'Downgrade Tier', or 'Proceed to Cancel'). 3. Selecting 'Proceed to Cancel' and confirming sends an API request to payment channel to disable recurring auto-renewal for the account. 4. Account status updates to 'Pending Cancellation', premium access remains active until the end of the current billing cycle, and an automated confirmation email is sent with the exact access expiration date.

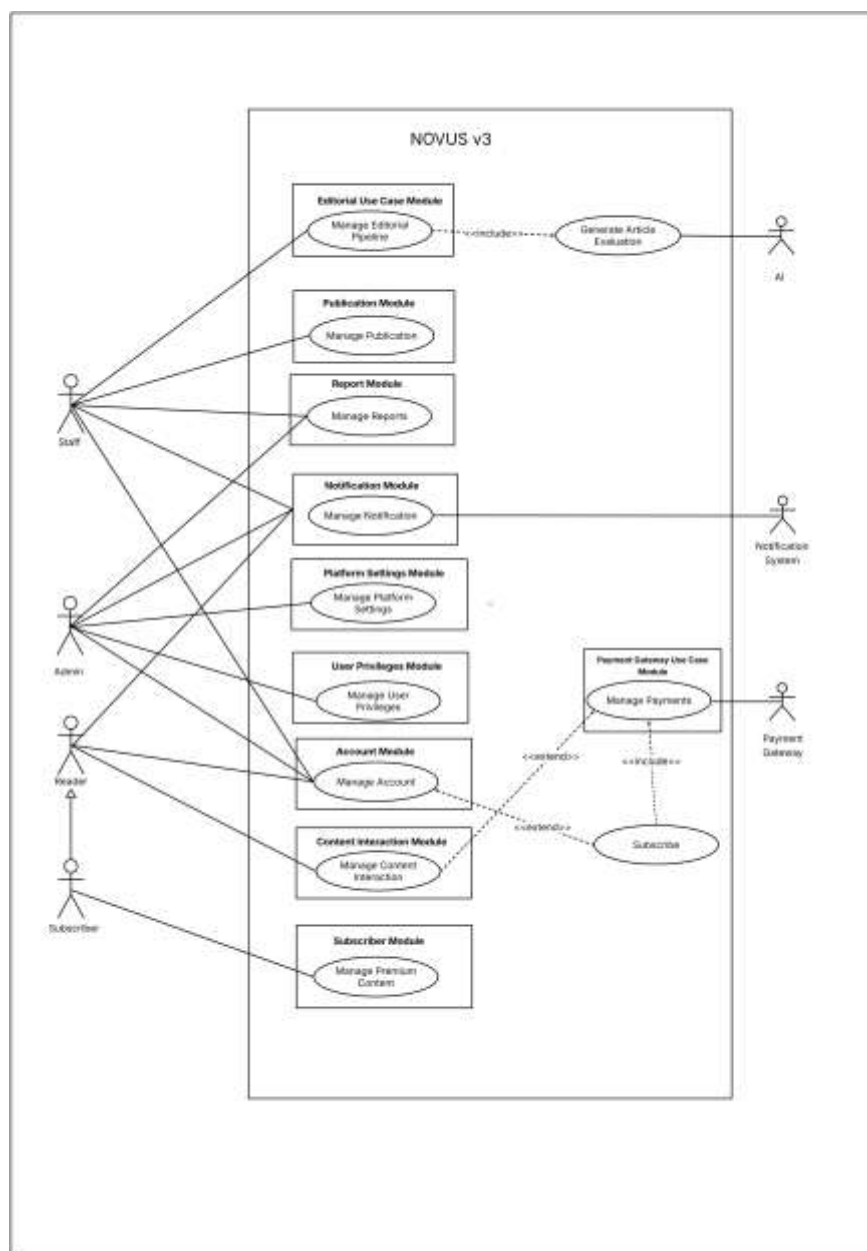
All Roles (System-Wide)	Forgot Password Recovery	As a registered user (any internal role or Subscriber), I want to reset my password if I forget it, so that I can regain access without contacting support.	1. On the Login screen, the user taps "Forgot Password?" 2. System navigates to the Password Reset Request screen with an Email field. 3. User enters their registered email and taps "Send Reset Link." 4. If the email is not found, system shows a pop-up "No account found with this email." User remains on the same screen. 5. If found, system shows a confirmation pop-up "Reset link sent. Please check your email." and navigates the user back to the Login screen. 6. User opens the emailed link, which opens the New Password screen. 7. User enters and confirms a new password, then taps "Reset Password." 8. If the password is too weak, system shows inline error text and keeps the user on the same screen. 9. If valid, system shows a success pop-up "Password reset successfully. Please log in with your new password." and navigates the user to the Login screen.
All Roles (System-Wide)	Logging Out	As a logged-in user, I want to log out of my account, so that my session ends securely on shared or public devices.	1. From any authenticated screen, the user taps their profile icon, then taps "Log Out." 2. System shows a confirmation pop-up "Are you sure you want to log out?" with Cancel / Confirm. 3. User taps "Cancel" → pop-up closes, user remains on the current screen. 4. User taps "Confirm" → system ends the session, shows "Logged out successfully," and navigates the user to the Login / Role Landing screen.
All Roles (System-Wide)	Session Timeout Handling	As a logged-in user, I want to be notified if my session expires from inactivity, so that I understand why I was signed out and can log back in easily.	1. System monitors inactivity; after 30 minutes with no interaction, the session is flagged expired. 2. On the user's next action, system intercepts it and shows a pop-up "Your session has expired. Please log in again."

			3. User taps “OK” → system navigates the user to the Login screen. 4. Once logged back in, system returns the user to the last screen they were viewing before the timeout, where possible.
All Roles (System-Wide)	Viewing In-App Notifications	As a logged-in user, I want a central place to see all my notifications, so that I don’t miss important status changes.	1. User taps the notification bell icon from any authenticated screen. 2. System opens a Notification Panel listing recent events, newest first, with unread items visually marked. 3. User taps a notification → system marks it read and navigates directly to the relevant screen. 4. User taps “Mark all as read” → system clears unread markers with a confirmation label; user remains on the panel. 5. User taps outside the panel or the close icon → panel closes; user remains on their current screen.

Use Case Diagram

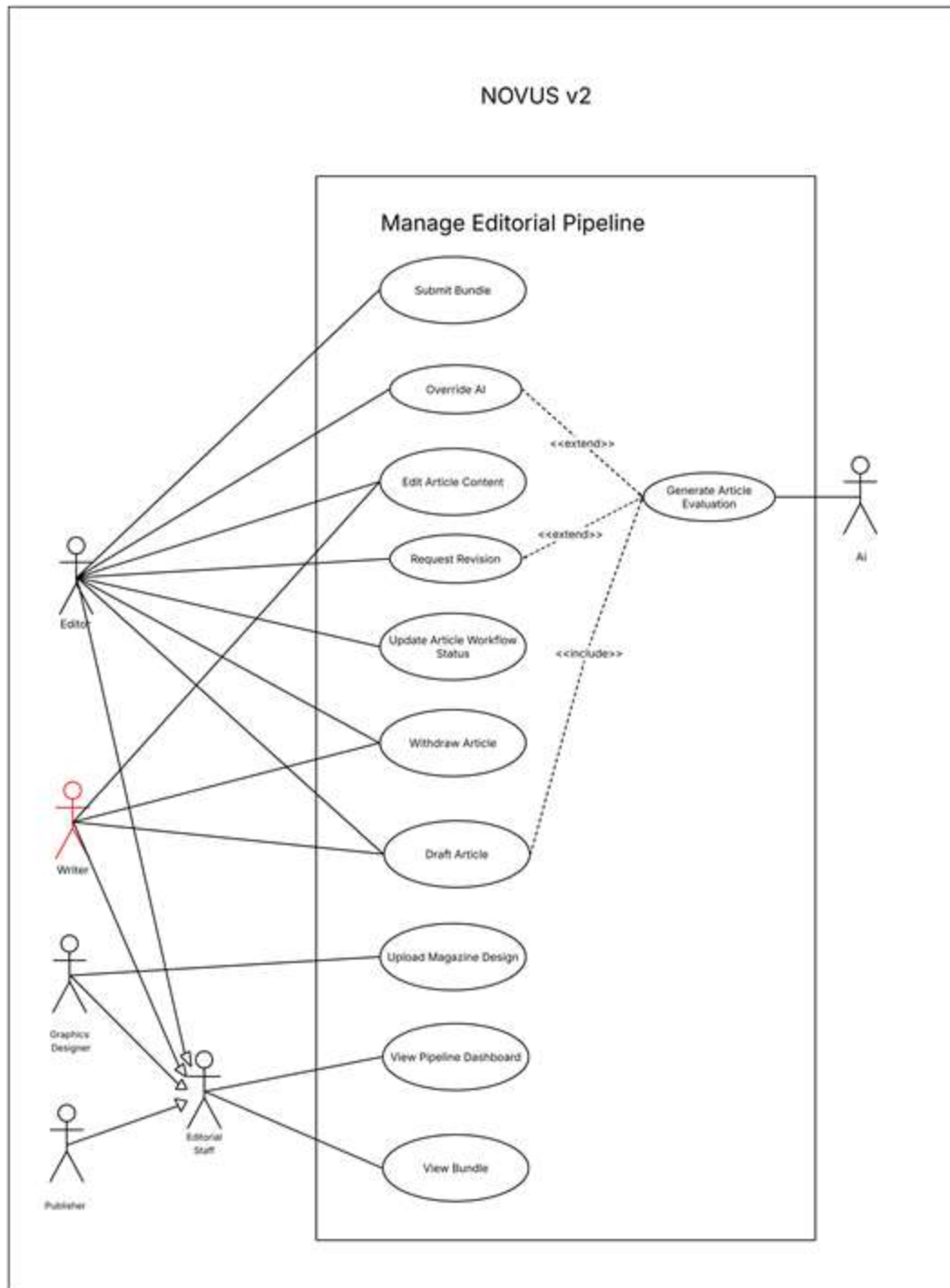
Use Case Full Description/Fully Dressed Use Case

Figure 1. NOVUS Use Case Diagram



a. Fully Dressed Use Cases

Figure 2. Manage Editorial Pipeline Use Case Diagram



Use Case Name: Submit Bundle

Use Case ID:	UC-1.1
Author	Rei Khalil Galido
Purpose	Allow an Editor to formally submit a completed content bundle (article draft, images, metadata) into the editorial pipeline for tracking and review.

Related Use Case	None
Requirement Traceability	PR-EDT-01
Priority	High
Preconditions	• System is online • Editor is authenticated and logged in • A draft bundle exists and is marked ready by its Writer & Graphics Designer
Postconditions	• Bundle status changes to "Submitted" • Bundle appears on the Pipeline Dashboard
Actors	• Editor
Flow of Actions	BASIC FLOW 1. Editor navigates to the Editorial Pipeline module. 2. Editor selects "Submit Bundle." 3. System displays draft bundles ready for submission. 4. Editor selects the target bundle and reviews its contents. 5. Editor clicks "Submit." 6. System validates that all required components (text, assets, metadata) are present. 7. System sets the bundle's workflow status to "Submitted" and timestamps the action. 8. System displays a confirmation notice to the Editor. ALTERNATE FLOW A1 (Incomplete Bundle) 1. The user initiates the operation for the target record. 2. The system validates the required information and components. 3. The system detects that one or more required items are missing or incomplete. 4. The system identifies the missing items and displays corrective guidance. 5. The system prevents the incomplete operation from changing the workflow state. 6. The user supplies or corrects the missing information and retries. 7. The system revalidates the corrected record before continuing. EXCEPTION FLOW E1 (Save Failure) 1. The user confirms the requested change. 2. The system validates the requested transition. 3. The system attempts to persist in the change to the authoritative data store. 4. The persistence operation fails or times out. 5. The system retains the last confirmed state and does not report success. 6. The system logs the failure and displays a controlled retry message. 7. The user retries after service recovery. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied.

	7. After authorization is restored, the user may retry.
--	---

Use Case Name: Draft Article

Use Case ID:	UC-1.2
Author	Rei Khalil Galido
Purpose	Allow a Writer & Editor to create and save a new article draft within the pipeline.
Related Use Case	Includes: Generate Article Evaluation (UC-1.11)
Requirement Traceability	PR-EDT-02
Priority	High
Preconditions	• System is online • Writer or Editor is authenticated and logged in • Writer has an assigned topic or pitch
Postconditions	• A new article draft exists with status "Drafting" or "Awaiting Evaluation" • An AI evaluation has been generated for the draft
Actors	• Writer • Editor • AI (secondary, via include)
Flow of Actions	BASIC FLOW 1. Writer or Editor selects "Draft Article" from the Editorial Pipeline module. 2. System opens a blank article editor. 3. Writer or Editor enters the headline, body content, and relevant metadata (tags, category). 4. Writer or Editor saves the draft. 5. System stores the draft and assigns it a unique bundle reference. 6. System includes Generate Article Evaluation (UC-1.11) to automatically analyze the draft. 7. System displays the evaluation summary alongside the draft. ALTERNATE FLOW A1 (Save as Incomplete) 1. The user initiates the operation for the target record. 2. The system validates the required information and components. 3. The system detects that one or more required items are missing or incomplete. 4. The system identifies the missing items and displays corrective guidance. 5. The system prevents the incomplete operation from changing the workflow state. 6. The user supplies or corrects the missing information and retries. 7. The system revalidates the corrected record before continuing. EXCEPTION FLOW E1 (Evaluation Service Unavailable) 1. The workflow reaches the point where an AI result is expected. 2. The system checks for a current result for the exact content version. 3. The system determines that the result is unavailable, incomplete, or delayed. 4. The system preserves the content and marks evaluation as pending/unavailable.

	5. The system queues or retries the evaluation. 6. The system informs the Editor that manual review may be required. 7. When a valid result arrives, the system attaches it to the correct version. UC-1.3: Edit Article Content Use Case Name: Manage Editorial Pipeline_Edit Article Content
--	--

Use Case Name: Edit Article Content

Use Case ID:	UC-1.3
Author	Rei Khalil Galido
Purpose	Allow an Editor or Writer to modify the text content of an article already in the pipeline.
Related Use Case	None
Requirement Traceability	PR-EDT-03
Priority	High
Preconditions	• System is online • User is authenticated as Editor or Writer • Target article exists in the pipeline and is not locked/published
Postconditions	• Article content is updated • Edit history/version log is updated
Actors	• Editor • Writer
Flow of Actions	BASIC FLOW 1. User opens the target article bundle from the pipeline. 2. User selects "Edit Content." 3. System loads the article into an editable text editor. 4. User modifies headline, body text, or metadata. 5. User clicks "Save Changes." 6. System validates the content and stores the new version. 7. System logs the edit (author, timestamp) in the article's version history. ALTERNATE FLOW A1 (Discard Changes) 1. The user chooses to cancel, discard, or leave the operation. 2. The system checks for unsaved changes or a pending transaction. 3. The system requests confirmation when the action could lose work or affect billing. 4. The user confirms cancellation. 5. The system discards only uncommitted changes or safely terminates the pending operation. 6. The system restores the last confirmed state and returns the user to the appropriate screen. 7. The system does not create an unintended version, state change, or entitlement. EXCEPTION FLOW E1 (Concurrent Edit Conflict) 1. The user opens the target and begins the operation.

	2. Another authorized session changes the same record. 3. The system compares the held version with the authoritative version. 4. The system detects stale data. 5. The system prevents silent overwrite. 6. The system displays the latest state or a comparison/merge option. 7. The user reviews the current version and retries. UC-1.4: Request Revision Use Case Name: Manage Editorial Pipeline_Request Revision
--	--

Use Case Name: Request Revision

Use Case ID:	UC-1.4
Author	Rei Khalil Galido
Purpose	Allow an Editor to request revisions for either a Writer-submitted article or a magazine design uploaded by a Graphics Designer, based on editorial or design review.
Related Use Case	Extends: Generate Article Evaluation (UC-1.11) for articles; may also be initiated directly from the magazine design review workflow.
Requirement Traceability	PR-EDT-04
Priority	High
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision
Postconditions	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actors	• Editor • Writer (for article revision requests) • Graphics Designer (for magazine-design revision requests)
Flow of Actions	BASIC FLOW 1. Editor opens the submitted article or uploaded magazine design requiring review. 2. System identifies the target type as either a Writer-submitted article or a Graphics Designer-uploaded magazine design. 3. System displays the latest version, responsible contributor, current workflow status, and revision history. 4. Editor selects "Request Revision." 5. System prompts the Editor for specific revision notes appropriate to the selected target type. 6. Editor enters actionable feedback. For articles, feedback may address content, structure, grammar, factual support, or editorial requirements. For magazine designs, feedback may address page layout, typography, imagery, branding, dimensions, or other design requirements. 7. Editor reviews and confirms the revision request.

8. System revalidates that the target remains reviewable and that the Editor still has permission.
9. System preserves the submitted version and records the revision request, Editor, timestamp, target type, and feedback.
10. System changes the target status to "Revision Requested."
11. System notifies the responsible Writer for an article or Graphics Designer for a magazine design.
12. System updates the pipeline/dashboard and makes the revision request visible to the responsible contributor.

ALTERNATE FLOW

A1 (No AI Feedback Available)

1. Editor opens an article for which the AI evaluation has not completed.
2. System confirms that the article is still in a reviewable state.
3. Editor performs manual editorial review without relying on an AI result.
4. Editor selects "Request Revision" and enters specific revision notes.
5. System records the request without requiring an AI evaluation reference.
6. System changes the article's status to "Revision Requested" and notifies the Writer.

ALTERNATE FLOW

A2 (Magazine Design Revision)

1. Editor opens a magazine design uploaded by a Graphics Designer.
2. System displays the latest design version, design status, Designer, and prior revision history.
3. Editor reviews the design against issue layout, branding, typography, imagery, dimensions, and other configured design requirements.
4. Editor selects "Request Revision" and enters design-specific corrections.
5. System validates that the design is still the current reviewable version.
6. System records the revision request and changes the design status to "Revision Requested."
7. System notifies the responsible Graphics Designer and exposes the revision notes in the design workspace.

EXCEPTION FLOW

E1 (Notification Failure)

1. The system successfully records the revision request and changes the target status.
2. The system creates a notification for the responsible Writer or Graphics Designer.
3. The notification service fails to deliver the message.
4. The system keeps the revision state and stored notes unchanged.
5. The system records the failed delivery and marks the notification for retry or follow-up.
6. The Editor can see the follow-up indicator on the dashboard.
7. The system retries delivery when the notification service becomes available.

EXCEPTION FLOW

E2 (Target State Changed Concurrently)

1. Editor opens an article or magazine design and begins writing revision notes.
2. Another authorized user submits a newer version or changes the target status.
3. Editor attempts to confirm the revision request.
4. System detects that the target version or workflow state has changed.
5. System prevents the Editor from attaching the revision request to a stale version.
6. System displays the latest version and current status.

	7. Editor reviews the current version and submits a new revision request if it is still necessary. UC-1.5: Override AI Use Case Name: Manage Editorial Pipeline_Override AI
--	--

Use Case Name: Override AI

Use Case ID:	UC-1.5
Author	Rei Khalil Galido
Purpose	Allow an Editor to manually override the AI's evaluation verdict for an article, using editorial judgment.
Related Use Case	Extends: Generate Article Evaluation (UC-1.11) at extension point "OverrideRequested"
Requirement Traceability	PR-EDT-05
Priority	Medium
Preconditions	• System is online • Editor is authenticated and logged in • Generate Article Evaluation has produced a verdict the Editor disagrees with
Postconditions	• Article's evaluation verdict is flagged as "Editor Override" • Override justification is recorded
Actors	• Editor
Flow of Actions	BASIC FLOW 1. Editor reviews the AI evaluation verdict for an article. 2. Editor selects "Override AI." 3. System requires the Editor to enter a justification for the override. 4. Editor submits the justification. 5. System updates the article's evaluation record to reflect the manual override. 6. System allows the article to proceed in the pipeline despite the original AI verdict. ALTERNATE FLOW A1 (Partial Override) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and requires the next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Missing Justification) 1. The user reaches the condition described in this flow. 2. The system detects the condition during the relevant basic flow step. 3. The system validates the current record, permission, and business rules.

	4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and requires the next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-1.6: Update Article Workflow Status Use Case Name: Manage Editorial Pipeline_Update Article Workflow Status
--	--

Use Case Name: Update Article Workflow Status

Use Case ID:	UC-1.6
Author	Rei Khalil Galido
Purpose	Allow an Editor to move an article through its editorial workflow stages (e.g., In Review, Approved, Ready for Publication).
Related Use Case	None
Requirement Traceability	PR-EDT-06
Priority	Medium
Preconditions	• System is online • Editor is authenticated and logged in • Article exists in the pipeline
Postconditions	• Article's workflow status is updated • Status change is logged in the article's history
Actors	• Editor
Flow of Actions	BASIC FLOW 1. Editor opens the target article's pipeline entry. 2. Editor selects "Update Workflow Status." 3. System displays available next-stage options based on the current status. 4. Editor selects the new status. 5. Editor confirms the change. 6. System updates the article's status and records the transition with a timestamp. ALTERNATE FLOW A1 (Revert Status) 1. The AI or workflow produces a result qualifying for the alternate approval path. 2. The system verifies the result belongs to the current version. 3. The system checks the configured threshold and mandatory-review rules. 4. If all conditions pass, the system advances the workflow. 5. The system records the rule-based advancement. 6. The system notifies the next responsible role. 7. If human review remains mandatory, the system leaves the item in review. EXCEPTION FLOW E1 (Invalid Transition) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules.

	4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-1.7: Withdraw Article Use Case Name: Manage Editorial Pipeline_Withdraw Article
--	---

Use Case Name: Withdraw Article

Use Case ID:	UC-1.7
Author	Rei Khalil Galido
Purpose	Allow an Editor or Writer to remove an article from the active pipeline before it is published.
Related Use Case	None
Requirement Traceability	PR-EDT-07
Priority	Medium
Preconditions	• System is online • User is authenticated as Editor or Writer • Article is not yet published
Postconditions	• Article status changes to "Withdrawn" • Article is removed from active pipeline views
Actors	• Editor • Writer
Flow of Actions	BASIC FLOW 1. User opens the target article's pipeline entry. 2. User selects "Withdraw Article." 3. System prompts for confirmation and a withdrawal reason. 4. User confirms. 5. System sets the article status to "Withdrawn" and logs the reason. 6. System removes the article from active pipeline dashboards. ALTERNATE FLOW A1 (Restore Withdrawn Article) 1. The user selects the withdrawal operation. 2. The system retrieves the latest authoritative state.

	3. The system checks whether withdrawal is still permitted. 4. If permitted, the system requests confirmation and any required reason. 5. The user confirms. 6. The system records the withdrawal and updates the state. 7. If publication is already irreversible, the system blocks withdrawal and directs the user to the appropriate post-publication process. EXCEPTION FLOW E1 (Already Published) 1. The user selects the withdrawal operation. 2. The system retrieves the latest authoritative state. 3. The system checks whether withdrawal is still permitted. 4. If permitted, the system requests confirmation and any required reason. 5. The user confirms. 6. The system records the withdrawal and updates the state. 7. If publication is already irreversible, the system blocks withdrawal and directs the user to the appropriate post-publication process. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-1.8: Upload Magazine Design Use Case Name: Manage Editorial Pipeline_ Upload Magazine Design
--	---

Use Case Name: Upload Magazine Design

Use Case ID:	UC-1.8
Author	Rei Khalil Galido
Purpose	Allow a Graphics Designer to upload a magazine design for an issue, create a versioned submission, and provide an Editor with a design that can be reviewed and returned for revision.
Related Use Case	None
Requirement Traceability	PR-EDT-08
Priority	High
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements
Postconditions	• Design file is securely stored as a versioned asset • Design is linked to the target issue

	• New version is submitted for Editor review • Prior versions remain traceable • Editor is notified
Actors	• Graphics Designer • Editor (secondary reviewer)
Flow of Actions	BASIC FLOW User navigates to the specific issue or bundle record in the system. User selects "Upload Magazine Designs." System opens a secure file upload interface. User selects the compiled digital magazine file (e.g., PDF) from their local device. System validates the file format and ensures it is within the maximum file size limits. System securely uploads the file, attaches it to the issue record, and timestamps the action. System displays a success confirmation notice to the user. ALTERNATE FLOW A1 (Version Replacement) 1. Graphics Designer opens an issue that already has a previously uploaded design. 2. System displays the active design version and prior version history. 3. Graphics Designer selects a corrected design file. 4. System validates Designer permission, target issue, file type, file size, and integrity. 5. System stores the corrected file as a new version rather than silently destroying the previous version. 6. System marks the new version as the active design submission. 7. System notifies the Editor that a revised design is ready for review. ALTERNATE FLOW A2 (Upload After Revision Request) 1. Graphics Designer opens a design with status "Revision Requested." 2. System displays the Editor's latest revision notes and the version being revised. 3. Graphics Designer prepares a corrected design. 4. Graphics Designer uploads the corrected file. 5. System creates a new version linked to the same revision cycle. 6. System preserves the prior submitted version and its revision notes. 7. System changes the new version to "Submitted for Review" and notifies the Editor. EXCEPTION FLOW E1 (Unsupported Format or Size Limit Exceeded) 1. Graphics Designer selects a design file. 2. System validates the file extension, detected content type, and file size before storage. 3. System detects an unsupported format or a file exceeding the configured size limit. 4. System rejects the file and does not create a new design version. 5. System displays the accepted formats and maximum file size. 6. Graphics Designer exports or selects a compliant file. 7. Graphics Designer retries the upload. EXCEPTION FLOW E2 (Upload Interruption)

	1. Graphics Designer begins transferring a valid design file. 2. The network connection is interrupted during the transfer. 3. System detects that the file was not completely transferred. 4. System marks the upload as failed or resumable according to storage capability. 5. System prevents the incomplete file from becoming the active design version. 6. System informs the Graphics Designer that the upload must be retried or resumed. 7. Graphics Designer retries after connectivity is restored. EXCEPTION FLOW E3 (Unauthorized Target Issue) 1. Graphics Designer selects an issue for upload. 2. System verifies that the Designer is assigned or authorized for the selected issue. 3. System determines that the Designer does not have permission for the target issue. 4. System blocks the upload before storing the file. 5. System displays an access-denied message without exposing restricted issue details. 6. Graphics Designer selects an authorized issue or requests assignment from the appropriate staff member. EXCEPTION FLOW E4 (Concurrent Design Submission) 1. Graphics Designer opens the issue while another authorized Designer is preparing a newer design version. 2. The other Designer submits a new version before the first upload is finalized. 3. System detects the changed active version when the first Designer confirms the upload. 4. System does not silently overwrite the newer version. 5. System creates the first Designer's file as a separate version only if versioning rules permit it; otherwise it asks the Designer to refresh. 6. System displays the current active version and latest submission history. 7. Graphics Designer reviews the current state and retries using the latest version information. UC-1.9: View Pipeline Dashboard Use Case Name: Manage Editorial Pipeline_View Pipeline Dashboard
--	---

Use Case Name: View Pipeline Dashboard

Use Case ID:	UC-1.9
Author	Rei Khalil Galido
Purpose	Allow Editorial Staff to view an overview of all bundles currently active in the editorial pipeline, including status and assignees.
Related Use Case	None
Requirement Traceability	PR-EDT-09
Priority	Medium
Preconditions	• System is online • User is authenticated as Editor, Writer, Publisher or Graphics Designer

Postconditions	• Dashboard is displayed reflecting current pipeline state
Actors	• Editorial Staff (Editor, Writer, Graphics Designer, Publisher)
Flow of Actions	BASIC FLOW 1. Editorial Staff member navigates to the Editorial Pipeline module. 2. Editorial Staff selects "View Pipeline Dashboard." 3. System retrieves all active bundles and their current statuses. 4. System displays the dashboard with filters (by status, assignee, deadline). 5. Editorial Staff member may apply filters to narrow the view. ALTERNATE FLOW A1 (No Active Bundles) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Data Load Failure) 1. The user opens the dashboard. 2. The system verifies session and permissions. 3. The system queries current records. 4. If no records match, the system displays the configured empty state. 5. If retrieval fails, the system displays a controlled error and retry option. 6. The system logs the failure when appropriate. 7. The user retries or changes the filter. UC-1.10: View Bundle Use Case Name: Manage Editorial Pipeline_View Bundle

Use Case Name: View Bundle

Use Case ID:	UC-1.10
Author	Rei Khalil Galido
Purpose	Allow Editorial Staff to view the full detail of a single content bundle, including its content, assets, status, and history.
Related Use Case	None
Requirement Traceability	PR-EDT-10
Priority	Medium
Preconditions	• System is online • User is authenticated as Editor, Writer, Graphics Designer, or Publisher • Target bundle exists
Postconditions	• Bundle detail view is displayed
Actors	• Editorial Staff (Editor, Writer, Graphics Designer, Publisher)

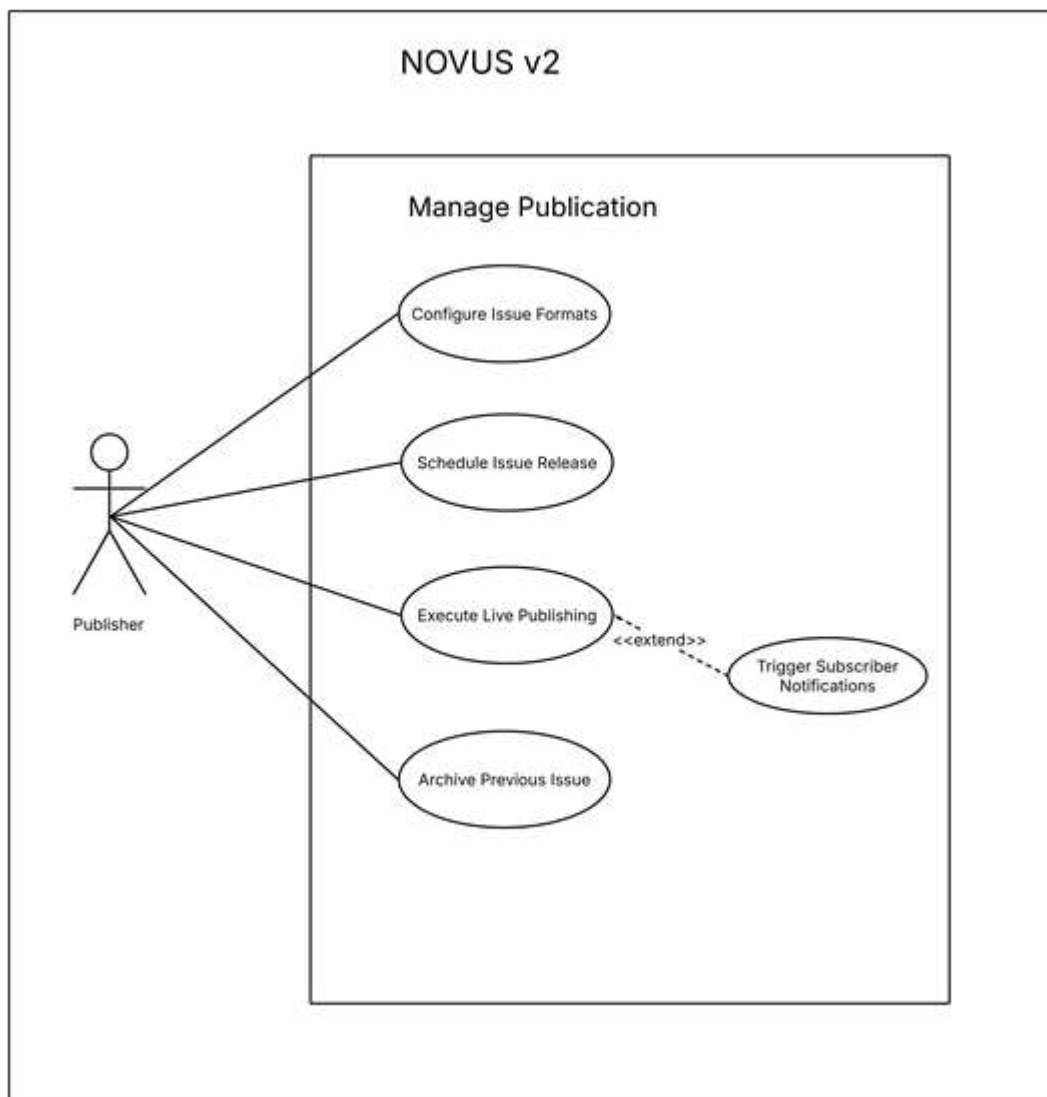
Flow of Actions	BASIC FLOW 1. Editorial Staff member selects a bundle from the Pipeline Dashboard or search. 2. System retrieves the bundle's content, assets, status, and revision history. 3. System displays the full bundle detail view. 4. Editorial Staff member reviews the information. ALTERNATE FLOW A1 (Navigate to Related Action) 1. The user selects a file and begins upload. 2. The system validates role, target assignment, file type, file size, and upload rules. 3. The system rejects invalid files before finalization. 4. For a valid file, the system transfers it to controlled storage. 5. The system verifies transfer completion and file integrity. 6. The system creates a new version and preserves prior versions. 7. If interrupted, the system prevents the incomplete file from becoming active and marks it failed or resumable. 8. The user retries or resumes after correcting the problem. EXCEPTION FLOW E1 (Bundle Not Found) 1. The user opens the dashboard. 2. The system verifies session and permissions. 3. The system queries current records. 4. If no records match, the system displays the configured empty state. 5. If retrieval fails, the system displays a controlled error and retry option. 6. The system logs the failure when appropriate. 7. The user retries or changes the filter. UC-1.11: Generate Article Evaluation Use Case Name: Manage Editorial Pipeline_Generate Article Evaluation
-----------------	--

Use Case Name: Generate Article Evaluation

Use Case ID:	UC-1.11
Author	Rei Khalil Galido
Purpose	Allow an Editor to request revisions for either a Writer-submitted article or a magazine design uploaded by a Graphics Designer, based on editorial or design review.
Related Use Case	Extends: Generate Article Evaluation (UC-1.11) for articles; may also be initiated directly from the magazine-design review workflow.
Requirement Traceability	PR-EDT-11
Priority	High
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision
Postconditions	• Target status changes to "Revision Requested" • Current submitted version is preserved

	• Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actors	• Editor • Writer (for article revision requests) • Graphics Designer (for magazine-design revision requests)
Flow of Actions	BASIC FLOW 1. System sends the article's content to the AI evaluation service (triggered via include from Draft Article). 2. AI analyzes the article for grammar, and readability. 3. AI returns a quality score and a recommendation (Approve / Reject). 4. System attaches the evaluation result to the article record. 5. System exposes extension point "RevisionRequested" to the Editor if the recommendation is "Reject." 6. System exposes extension point "OverrideRequested" to the Editor for any evaluation result. 7. System notifies the Editor that evaluation is complete. ALTERNATE FLOW A1 (High-Confidence Approval) 1. The AI or workflow produces a result qualifying for the alternate approval path. 2. The system verifies the result belongs to the current version. 3. The system checks the configured threshold and mandatory-review rules. 4. If all conditions pass, the system advances the workflow. 5. The system records the rule-based advancement. 6. The system notifies the next responsible role. 7. If human review remains mandatory, the system leaves the item in review. EXCEPTION FLOW E1 (AI Service Timeout) 1. The workflow reaches the point where an AI result is expected. 2. The system checks for a current result for the exact content version. 3. The system determines that the result is unavailable, incomplete, or delayed. 4. The system preserves the content and marks evaluation as pending/unavailable. 5. The system queues or retries the evaluation. 6. The system informs the Editor that manual review may be required. 7. When a valid result arrives, the system attaches it to the correct version. MODULE 2: Manage Publication Actors: Publisher UC-2.1: Configure Issue Formats Use Case Name: Manage Publication_Configure Issue Formats

Figure 3. Manage Publication Use Case Diagram

**Use Case Name: Configure Issue Formats**

Use Case ID:	UC-2.1
Author	Joshmar Clavero

Purpose	Allow the Publisher to define and configure the output formats (digital layout templates, PDF/e-reader formats) available for a publication issue.
Related Use Case	• None
Requirement Traceability	PR-PUB-01
Priority	Medium
Preconditions	• System is online • Publisher is authenticated and logged in
Postconditions	• Issue format configuration is saved and available for scheduling
Actors	• Publisher
Flow of Actions	BASIC FLOW 1. Publisher navigates to the Manage Publication module. 2. Publisher selects "Configure Issue Formats." 3. System displays existing format templates (e.g., Web, PDF, ePub). 4. Publisher creates a new format or edits an existing one (dimensions, cover template, file type). 5. Publisher saves the configuration. 6. System validates and stores the format settings. ALTERNATE FLOW A1 (Duplicate Format) 1. The user selects a file and begins upload. 2. The system validates role, target assignment, file type, file size, and upload rules. 3. The system rejects invalid files before finalization. 4. For a valid file, the system transfers it to controlled storage. 5. The system verifies transfer completion and file integrity. 6. The system creates a new version and preserves prior versions. 7. If interrupted, the system prevents the incomplete file from becoming active and marks it failed or resumable. 8. The user retries or resumes after correcting the problem. EXCEPTION FLOW E1 (Invalid Parameters) 1. The user selects a file and begins upload. 2. The system validates role, target assignment, file type, file size, and upload rules. 3. The system rejects invalid files before finalization. 4. For a valid file, the system transfers it to controlled storage. 5. The system verifies transfer completion and file integrity. 6. The system creates a new version and preserves prior versions. 7. If interrupted, the system prevents the incomplete file from becoming active and marks it failed or resumable. 8. The user retries or resumes after correcting the problem. UC-2.2: Schedule Issue Release Use Case Name: Manage Publication_Schedule Issue Release

Use Case Name: Schedule Issue Release

Use Case ID:	UC-2.2
Author	Joshmar Clavero
Purpose	Allow the Publisher to set a future release date and time for an upcoming issue.
Related Use Case	• None
Requirement Traceability	PR-PUB-02
Priority	High
Preconditions	• System is online • Issue content (approved bundles) is ready • Issue format is configured
Postconditions	• Issue is queued with a scheduled release date/time
Actors	• Publisher
Flow of Actions	BASIC FLOW 1. Publisher selects an issue that is ready for release. 2. Publisher selects "Schedule Issue Release." 3. System displays a calendar/time picker. 4. Publisher selects the release date and time. 5. Publisher confirms. 6. System sets the issue status to "Scheduled" and stores the release timestamp. ALTERNATE FLOW A1 (Reschedule) 1. The user enters a date/time. 2. The system validates format and scheduling rules. 3. The system detects that the value violates a constraint. 4. The system rejects the value and explains the rule. 5. The user enters a compliant future value. 6. The system revalidates and continues. EXCEPTION FLOW E1 (Invalid Date) 1. The user enters a date/time. 2. The system validates format and scheduling rules. 3. The system detects that the value violates a constraint. 4. The system rejects the value and explains the rule. 5. The user enters a compliant future value. 6. The system revalidates and continues. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-2.3: Execute Live Publishing

	Use Case Name: Manage Publication_Execute Live Publishing
--	---

Use Case Name: Execute Live Publishing

Use Case ID:	UC-2.3
Author	Joshmar Clavero
Purpose	Allow the Publisher to manually trigger immediate publishing of an issue, bypassing or overriding a schedule.
Related Use Case	• Extended by: Trigger Subscriber Notifications (UC-2.3.1) — optional
Requirement Traceability	PR-PUB-03
Priority	High
Preconditions	• System is online • Issue is fully approved and ready
Postconditions	• Issue is published live and accessible per its access tier
Actors	• Publisher
Flow of Actions	BASIC FLOW 1. Publisher opens a ready issue. 2. Publisher selects “Execute Live Publishing.” 3. System prompts for confirmation. 4. Publisher confirms. 5. System publishes the issue immediately, updates status to “Published,” and makes it live on the platform. At this point, the optional Trigger Subscriber Notifications extension (UC-2.3.1) may fire. 6. System logs the publishing event. ALTERNATE FLOW A (Flow) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Publishing Failure) 1. The user initiates the operation for the target record. 2. The system validates the required information and components. 3. The system detects that one or more required items are missing or incomplete. 4. The system identifies the missing items and displays corrective guidance. 5. The system prevents the incomplete operation from changing the workflow state. 6. The user supplies or corrects the missing information and retries. 7. The system revalidates the corrected record before continuing.

	EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-2.3.1: Trigger Subscriber Notifications Use Case Name: Manage Publication_Trigger Subscriber Notifications
--	---

Use Case Name: Trigger Subscriber Notifications

Use Case ID:	UC-2.3.1
Author	Joshmar Clavero
Purpose	Allow the Publisher to send notifications to subscribers and readers about a new issue or article, as an optional extension of the live publishing process.
Related Use Case	• Extends: Execute Live Publishing (UC-2.3)
Requirement Traceability	PR-PUB-04
Priority	Medium
Preconditions	• System is online • Execute Live Publishing (UC-2.3) has reached the point where the issue is published or is about to be published
Postconditions	• Notification is dispatched to the relevant subscriber segment
Actors	• Publisher
Flow of Actions	BASIC FLOW 1. Immediately after Execute Live Publishing (UC-2.3) publishes the issue, System offers the Publisher the option to trigger subscriber notifications. 2. Publisher elects to proceed with the extension. 3. System displays notification channel options (email, push, in-app). 4. Publisher customizes the notification message and target segment. 5. Publisher confirms send. 6. System dispatches the notifications, logs delivery statistics, and returns control to Execute Live Publishing (UC-2.3). ALTERNATE FLOW A1 (Schedule Notification) 1. The user enters a date/time. 2. The system validates format and scheduling rules. 3. The system detects that the value violates a constraint. 4. The system rejects the value and explains the rule. 5. The user enters a compliant future value. 6. The system revalidates and continues.

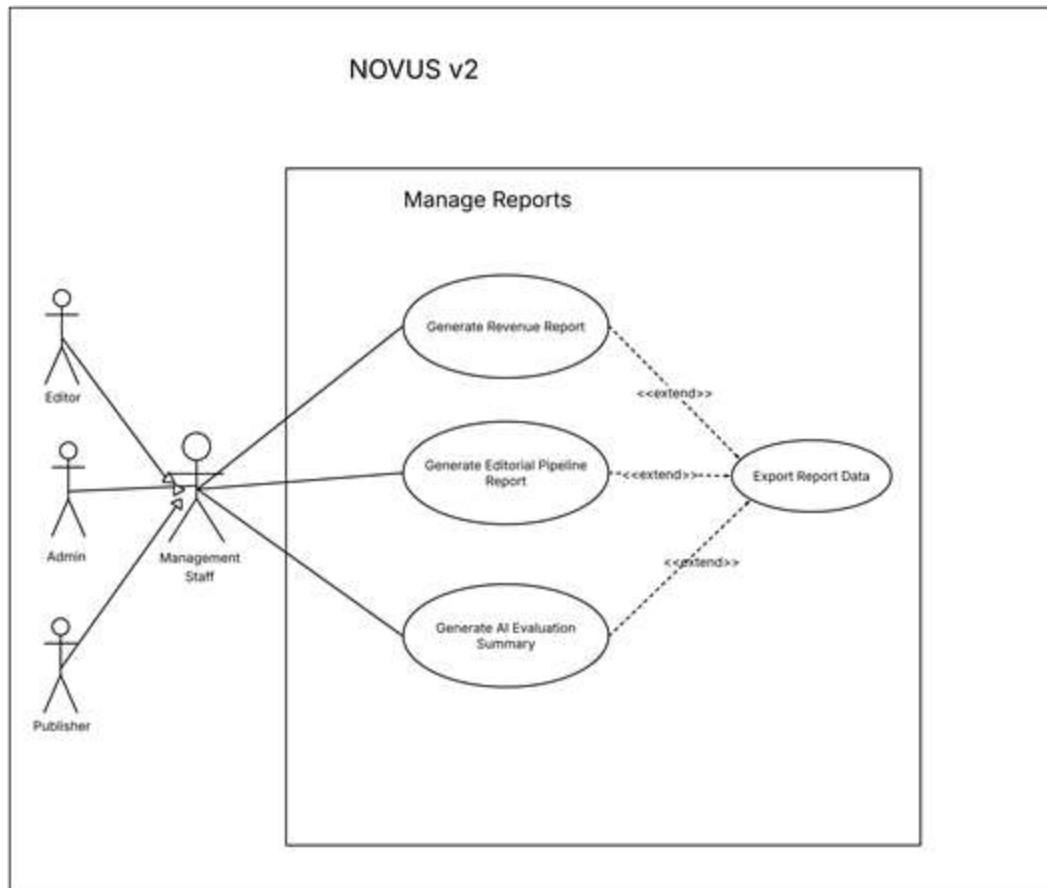
	EXCEPTION FLOW E1 (Delivery Failure) 1. The user initiates the operation for the target record. 2. The system validates the required information and components. 3. The system detects that one or more required items are missing or incomplete. 4. The system identifies the missing items and displays corrective guidance. 5. The system prevents the incomplete operation from changing the workflow state. 6. The user supplies or corrects the missing information and retries. 7. The system revalidates the corrected record before continuing. EXCEPTION FLOW E2 (Extension Declined) 1. The user confirms publication. 2. The system validates publish readiness. 3. The system starts the publication transaction and records the pre-publication state. 4. The publishing service returns an error, timeout, or incomplete result. 5. The system verifies what, if anything, reached the live environment. 6. The system does not report full publication when verification is incomplete. 7. The system records the failure and exposes controlled recovery/retry. UC-2.5: Archive Previous Issue Use Case Name: Manage Publication_Archive Previous Issue
--	--

Use Case Name: Archive Previous Issue

Use Case ID:	UC-2.5
Author	Joshmar Clavero
Purpose	Allow the Publisher to move an older, already-published issue into the archive for historical access.
Related Use Case	• None
Requirement Traceability	PR-PUB-05
Priority	Low
Preconditions	• System is online • Issue has been published and is no longer the current issue
Postconditions	• Issue is flagged as archived and relocated to the archive listing
Actors	• Publisher
Flow of Actions	BASIC FLOW 1. Publisher navigates to the issue list. 2. Publisher selects an old issue. 3. Publisher selects "Archive Previous Issue." 4. System prompts for confirmation. 5. Publisher confirms. 6. System updates the issue status to "Archived" and relocates it to the archive section. ALTERNATE FLOW A1 (Bulk Archive)

	1. The user confirms publication. 2. The system validates publish readiness. 3. The system starts the publication transaction and records the pre-publication state. 4. The publishing service returns an error, timeout, or incomplete result. 5. The system verifies what, if anything, reached the live environment. 6. The system does not report full publication when verification is incomplete. 7. The system records the failure and exposes controlled recovery/retry. EXCEPTION FLOW E1 (Active Issue Protection) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. MODULE 3: Manage Reports Actors: Editor; Admin; Publisher; Management Staff (generalized parent of Editor, Admin, and Publisher) UC-3.1: Generate Revenue Report Use Case Name: Manage Reports_Generate Subscription Revenue Report
--	---

Figure 4. Manage Reports Use Case Diagram



Use Case Name: Generate Revenue Report

Use Case ID:	UC-3.1
Author	Rei Khalil Galido
Purpose	Allow Management Staff to generate a report summarizing subscription and payment revenue over a selected period.
Related Use Case	• Extends into: Export Report Data (UC-3.1.1)
Requirement Traceability	PR-RPT-01
Priority	High
Preconditions	• System is online • User is authenticated as Editor, Admin, or Publisher • Payment data exists for the requested period
Postconditions	• Report is generated and available for viewing/export
Actors	• Management Staff (Editor, Admin, Publisher)
Flow of Actions	BASIC FLOW 1. Management Staff navigates to the Manage Reports module. 2. Management Staff selects "Generate Subscription Revenue Report."

	3. System prompts for a date range and filters (plan type, region). 4. User submits the parameters. 5. System compiles revenue data from payment records. 6. System displays the report on screen; Management Staff may optionally trigger the Export Report Data extension (UC-3.1.1) to download the report as PDF or CSV. ALTERNATE FLOW A1 (Recurring Report) 1. The user enters a date/time. 2. The system validates format and scheduling rules. 3. The system detects that the value violates a constraint. 4. The system rejects the value and explains the rule. 5. The user enters a compliant future value. 6. The system revalidates and continues. EXCEPTION FLOW E1 (No Data Found) 1. The user requests the selected list/report. 2. The system queries the current data set. 3. The system finds no matching records. 4. The system displays the empty state or configured fallback. 5. The system does not present fabricated or stale data. 6. The user adjusts filters or returns to another section. UC-3.1.1: Export Report Data Use Case Name: Manage Reports_Export Report Data
--	--

Use Case Name: Export Report Data

Use Case ID:	UC-3.1.1
Author	Rei Khalil Galido
Purpose	Serve as a shared extension that converts the currently displayed report's on-screen data into a downloadable PDF or CSV file.
Related Use Case	• Extends: Generate Subscription Revenue Report (UC-3.1), Generate Editorial Pipeline Report (UC-3.2), Generate AI Evaluation Summary (UC-3.3)
Requirement Traceability	PR-RPT-04
Priority	Medium
Preconditions	• System is online • A report has been generated and is currently displayed by UC-3.1, UC-3.2, or UC-3.3
Postconditions	• Report data is exported and downloaded in the selected format (PDF or CSV)
Actors	• Management Staff (Editor, Admin, Publisher)
Flow of Actions	BASIC FLOW 1. While viewing a generated report, Management Staff selects "Export." 2. System displays the available export formats (PDF, CSV). 3. Management Staff selects a format. 4. System converts the on-screen report data into the selected file format.

	5. System generates the downloadable file and returns control to the originating report use case (UC-3.1, UC-3.2, or UC-3.3). ALTERNATE FLOW A1 (Default Format Download) 1. The user selects a file and begins upload. 2. The system validates role, target assignment, file type, file size, and upload rules. 3. The system rejects invalid files before finalization. 4. For a valid file, the system transfers it to controlled storage. 5. The system verifies transfer completion and file integrity. 6. The system creates a new version and preserves prior versions. 7. If interrupted, the system prevents the incomplete file from becoming active and marks it failed or resumable. 8. The user retries or resumes after correcting the problem. EXCEPTION FLOW E1 (Export Failure) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-3.2: Generate Editorial Pipeline Report Use Case Name: Manage Reports_Generate Editorial Pipeline Report
--	--

Use Case Name: Generate Editorial Pipeline Report

Use Case ID:	UC-3.2
Author	Rei Khalil Galido
Purpose	Allow Management Staff to generate a report on editorial pipeline throughput, such as articles drafted, submitted, published, and average turnaround time.
Related Use Case	• Extends into: Export Report Data (UC-3.1.1)
Requirement Traceability	PR-RPT-02
Priority	Medium
Preconditions	• System is online • Pipeline activity data exists
Postconditions	• Report is generated and available for viewing/export
Actors	• Management Staff (Editor, Admin, Publisher)
Flow of Actions	BASIC FLOW 1. Management Staff selects “Generate Editorial Pipeline Report.” 2. System prompts for a date range and scope (team or author). 3. User submits the parameters. 4. System compiles pipeline metrics.

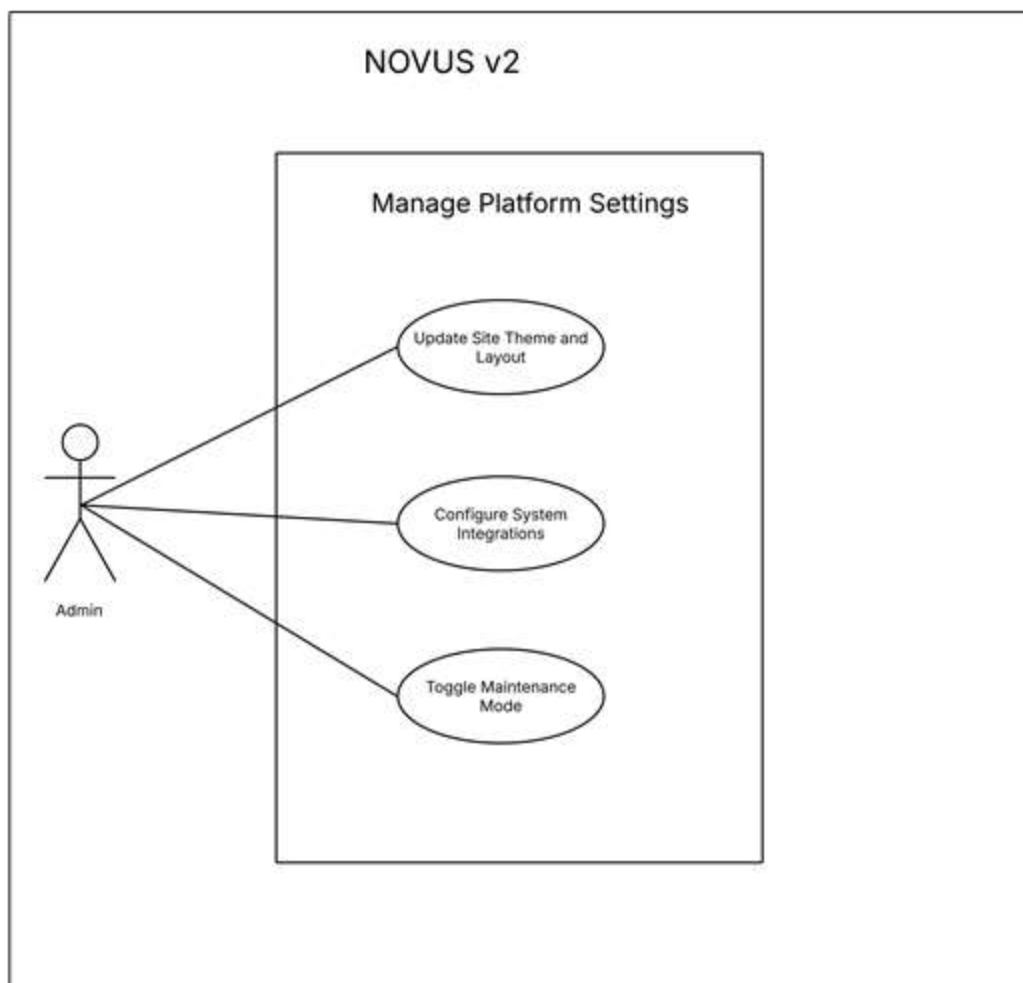
	5. System displays the report on screen; Management Staff may optionally trigger the Export Report Data extension (UC-3.1.1) to download the report. ALTERNATE FLOW A1 (Drill-Down) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Generation Timeout) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-3.3: Generate AI Evaluation Summary Use Case Name: Manage Reports_Generate AI Evaluation Summary
--	---

Use Case Name: Generate AI Evaluation Summary

Use Case ID:	UC-3.3
Author	Rei Khalil Galido
Purpose	Allow Management Staff to generate a summary report of AI article evaluations, including scores, override rate, and revision-request rate.
Related Use Case	• Extends into: Export Report Data (UC-3.1.1)
Requirement Traceability	PR-RPT-03
Priority	Medium
Preconditions	• System is online • AI evaluations exist for the requested period
Postconditions	• Report is generated and available for viewing/export
Actors	• Management Staff (Editor, Admin, Publisher)
Flow of Actions	BASIC FLOW 1. Management Staff selects “Generate AI Evaluation Summary.” 2. System prompts for a date range. 3. User submits the parameters. 4. System aggregates evaluation scores, override counts, and revision-request counts.

	5. System displays the summary report on screen; Management Staff may optionally trigger the Export Report Data extension (UC-3.1.1) to download the report. ALTERNATE FLOW A1 (Filter by Staff) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Insufficient Data) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. MODULE 4: Manage Platform Settings Actors: Admin UC-4.1: Update Site Theme and Layout Use Case Name: Manage Platform Settings_Update Site Theme and Layout
--	--

Figure 5. Manage Platform Settings Use Case Diagram



Use Case Name: Update Site Theme and Layout

Use Case ID:	UC-4.1
Author	Roman Rico Albania
Purpose	Allow the Admin to customize the platform's visual theme and homepage layout.
Related Use Case	None
Requirement Traceability	PR-SET-01
Priority	Low
Preconditions	• System is online • Admin is authenticated and logged in
Postconditions	• Theme and layout changes are applied site-wide
Actors	• Admin
Flow of Actions	BASIC FLOW 1. Admin navigates to the Manage Platform Settings module. 2. Admin selects "Update Site Theme and Layout." 3. System displays current theme and layout options.

	- Admin adjusts colors, and section ordering. - Admin previews the changes. - Admin saves. - System applies the changes site-wide. ALTERNATE FLOW A1 (Revert to Default) - The user reaches the condition described by this flow. - The system detects the condition during the relevant basic-flow step. - The system validates the current record, permission, and business rules. - The system performs the permitted alternate handling or blocks the invalid operation. - The system explains the resulting state and required next action. - The system records the event when audit/security rules require it. - The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Invalid Configuration) - The user reaches the condition described by this flow. - The system detects the condition during the relevant basic-flow step. - The system validates the current record, permission, and business rules. - The system performs the permitted alternate handling or blocks the invalid operation. - The system explains the resulting state and required next action. - The system records the event when audit/security rules require it. - The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) - The user initiates the protected operation. - The system validates the current session and required role or ownership permission immediately before the state-changing action. - The system detects an expired session or missing permission. - The system blocks the state change and does not partially commit it. - The system records the denied attempt when required by security policy. - The system redirects to authentication or displays access denied. - After authorization is restored, the user may retry. UC-4.2: Configure System Integrations Use Case Name: Manage Platform Settings_Configure System Integrations
--	--

Use Case Name: Configure System Integrations

Use Case ID:	UC-4.2
Author	Roman Rico Albania
Purpose	Allow the Admin to manage third-party integrations such as the payment gateway, and the email service.
Related Use Case	None
Requirement Traceability	PR-SET-02
Priority	High

Preconditions	• System is online • Admin is authenticated and logged in
Postconditions	• Integration configuration is updated and connection verified
Actors	• Admin
Flow of Actions	BASIC FLOW 1. Admin selects "Configure System Integrations." 2. System lists available integrations and their connection status. 3. Admin selects an integration to configure. 4. Admin enters or updates the API credentials or settings. 5. Admin tests the connection. 6. Admin saves. ALTERNATE FLOW A1 (Disable Integration) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Connection Test Failure) 1. The user submits credentials. 2. The system validates them against the account. 3. The system determines the credentials are incorrect. 4. The system rejects the operation without revealing sensitive details. 5. The system applies rate limiting or lockout rules when thresholds are reached. 6. The system displays the permitted recovery option. 7. The user retries or starts recovery. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-4.3: Toggle Maintenance Mode Use Case Name: Manage Platform Settings_Toggle Maintenance Mode

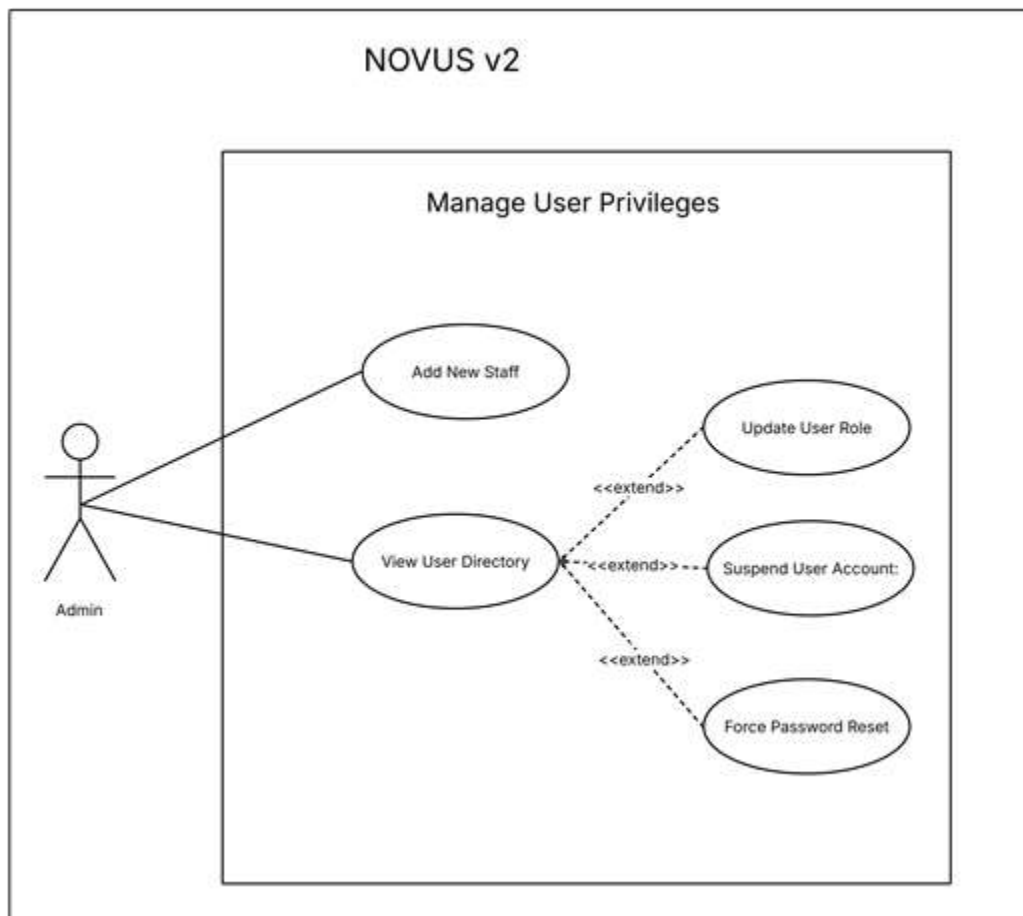
Use Case Name: Toggle Maintenance Mode

Use Case ID:	UC-4.3
Author	Roman Rico Albania

Purpose	Allow the Admin to place the platform into maintenance mode, restricting reader access during updates.
Related Use Case	None
Requirement Traceability	PR-SET-03
Priority	Medium
Preconditions	• System is online • Admin is authenticated and logged in
Postconditions	• Platform maintenance state is toggled and reflected to visitors
Actors	• Admin
Flow of Actions	BASIC FLOW 1. Admin selects "Toggle Maintenance Mode." 2. System prompts for confirmation and an optional custom maintenance message. 3. Admin confirms. 4. System enables maintenance mode, displaying the message to all non-Admin visitors. 5. Admin later disables maintenance mode to restore normal access. ALTERNATE FLOW A1 (Scheduled Maintenance) 1. The user enters a date/time. 2. The system validates format and scheduling rules. 3. The system detects that the value violates a constraint. 4. The system rejects the value and explains the rule. 5. The user enters a compliant future value. 6. The system revalidates and continues. EXCEPTION FLOW E1 (Publishing Conflict) 1. The user confirms publication. 2. The system validates publish readiness. 3. The system starts the publication transaction and records the pre-publication state. 4. The publishing service returns an error, timeout, or incomplete result. 5. The system verifies what, if anything, reached the live environment. 6. The system does not report full publication when verification is incomplete. 7. The system records the failure and exposes controlled recovery/retry. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. MODULE 5: Manage User Privileges Actors: Admin

	UC-5.1: Add New User Use Case Name: Manage User Privileges_Add New User
--	--

Figure 6. Manage User Privileges Use Case Diagram



Use Case Name: Add New Staff

Use Case ID:	UC-5.1
Author	Roman Rico Albania
Purpose	Allow the Admin to manually create a new staff or reader account with an assigned role.
Related Use Case	• None
Requirement Traceability	PR-USR-01
Priority	High
Preconditions	• System is online • Admin is authenticated and logged in
Postconditions	• New user account is created with the assigned role

Actors	• Admin
Flow of Actions	BASIC FLOW 1. Admin selects “Add New Staff.” 2. System displays a new-user form. 3. Admin enters the name, email, and assigns a role (Editor, Writer, Publisher, Admin, etc.). 4. Admin submits the form. 5. System creates the account and sends setup credentials to the user’s email. ALTERNATE FLOW A1 (Bulk Import) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Duplicate Email) 1. The user submits the operation. 2. The system checks unique identifiers against existing records. 3. The system finds an existing matching record. 4. The system prevents duplicate creation. 5. The system displays a controlled message without exposing unrelated private information. 6. The user chooses the permitted recovery action. 7. The system continues only with a valid unique target. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-5.2: View User Directory Use Case Name: Manage User Privileges_View User Directory

Use Case Name: View User Directory

Use Case ID:	UC-5.2
Author	Roman Rico Albania
Purpose	Allow the Admin to view a searchable list of all system users along with their roles and status.

Related Use Case	Extended by: Update User Role (UC-5.2.1), Suspend User Account (UC-5.2.2), Force Password Reset (UC-5.2.3)
Requirement Traceability	PR-USR-02
Priority	Medium
Preconditions	- System is online - Admin is authenticated and logged in
Postconditions	User directory is displayed
Actors	Admin
Flow of Actions	BASIC FLOW 1. Admin selects "View User Directory." 2. System retrieves and displays all user accounts. 3. Admin searches or filters by name, role, or status. From this listing, Admin may select a user record and optionally trigger Update User Role (UC-5.2.1), Suspend User Account (UC-5.2.2), or Force Password Reset (UC-5.2.3). ALTERNATE FLOW A1 (Export Directory) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Load Failure) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-5.2.1: Update User Role Use Case Name: Manage User Privileges_Update User Role

Use Case Name: Update User Role

Use Case ID:	UC-5.2.1
Author	Roman Rico Albania
Purpose	Allow the Admin to change a user's assigned role and permission level.
Related Use Case	Extends: View User Directory (UC-5.2)

Requirement Traceability	PR-USR-03
Priority	High
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing
Postconditions	• User's role and permissions are updated
Actors	• Admin
Flow of Actions	BASIC FLOW 1. While viewing the User Directory (UC-5.2), Admin selects a target user record and triggers "Update User Role." 2. System displays available roles. 3. Admin selects the new role. 4. Admin confirms. 5. System updates the user's role, applies the permission changes immediately, and returns control to the User Directory (UC-5.2). ALTERNATE FLOW A1 (Multiple Roles) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Last Admin Protection) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-5.2.2: Suspend User Account Use Case Name: Manage User Privileges_Suspend User Account

Use Case Name: Suspend User Account

Use Case ID:	UC-5.2.2
Author	Roman Rico Albania
Purpose	Allow the Admin to temporarily disable a user's access to the platform.
Related Use Case	• Extends: View User Directory (UC-5.2)

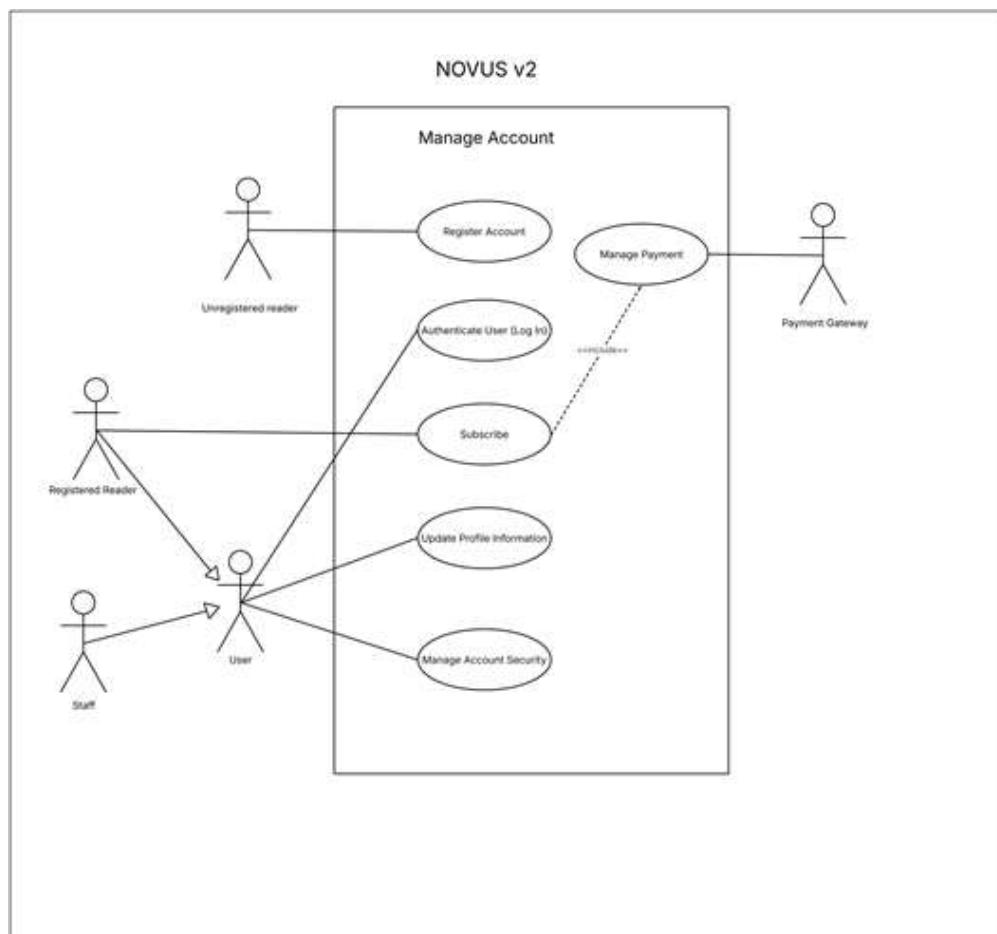
Requirement Traceability	PR-USR-04
Priority	High
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing
Postconditions	• User account is suspended and active sessions are revoked
Actors	• Admin
Flow of Actions	BASIC FLOW 1. While viewing the User Directory (UC-5.2), Admin selects a target user record and triggers "Suspend User Account." 2. System prompts for a reason and, optionally, a duration. 3. Admin confirms. 4. System suspends the account, revokes active sessions, and returns control to the User Directory (UC-5.2). ALTERNATE FLOW A1 (Reinstate Account) 1. The user requests an operation requiring active entitlement/account status. 2. The system retrieves the authoritative status. 3. The system determines the account or entitlement is inactive, expired, or suspended. 4. The system blocks the protected operation. 5. The system displays the permitted recovery action. 6. The system records the denied attempt when required. EXCEPTION FLOW E1 (Self-Suspension Protection) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-5.2.3: Force Password Reset Use Case Name: Manage User Privileges_Force Password Reset

Use Case Name: Force Password Reset

Use Case ID:	UC-5.2.3
Author	Roman Rico Albania
Purpose	Allow the Admin to require a user to reset their password on next login, for example after a security concern.
Related Use Case	• Extends: View User Directory (UC-5.2)

Requirement Traceability	PR-USR-05
Priority	Medium
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing
Postconditions	• Account is flagged for mandatory password reset
Actors	• Admin
Flow of Actions	BASIC FLOW 1. While viewing the User Directory (UC-5.2), Admin selects a target user record and triggers "Force Password Reset." 2. System asks the Admin to confirm the action. 3. System invalidates the current password and flags the account for mandatory reset. 4. System emails the user a reset link and returns control to the User Directory (UC-5.2). ALTERNATE FLOW A1 (Bulk Reset) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Undeliverable Email) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. MODULE 6: Manage Account Actors: Unregistered Reader; Registered Reader; Staff; User (generalized parent of Registered Reader and Staff); Payment Gateway (external) UC-6.1: Register Account Use Case Name: Manage Account_Register Account

Figure 7. Manage Account Use Case Diagram



Use Case Name: Register Account

Use Case ID:	UC-6.1
Author	Joshua Concepcion
Purpose	Allow an Unregistered Reader to create a new reader account.
Related Use Case	None
Requirement Traceability	PR-ACC-01
Priority	High
Preconditions	• System is online • Unregistered Reader has navigated to the application's URL
Postconditions	• A new Registered Reader account is created and able to log in
Actors	• Unregistered Reader
Flow of Actions	BASIC FLOW 1. Unregistered Reader clicks "Log In or Sign Up." 2. System displays the registration form. 3. Reader enters their name, email, and password. 4. Reader submits the form.

	5. System validates uniqueness of the email and creates the account. 6. System sends a verification email. 7. Reader verifies the email to activate the account. ALTERNATE FLOW A1 (SSO Registration) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Duplicate Email) 1. The user submits the operation. 2. The system checks unique identifiers against existing records. 3. The system finds an existing matching record. 4. The system prevents duplicate creation. 5. The system displays a controlled message without exposing unrelated private information. 6. The user chooses the permitted recovery action. 7. The system continues only with a valid unique target. UC-6.2: Authenticate User (Log In) Use Case Name: Manage Account_Authenticate User (Log In)
--	--

Use Case Name: Authenticate User (Log In)

Use Case ID:	UC-6.2
Author	Joshua Concepcion
Purpose	Allow a registered User, whether a Reader or Staff member, to log in to their account.
Related Use Case	None
Requirement Traceability	PR-ACC-02
Priority	High
Preconditions	• System is online • User has a registered, active account
Postconditions	• An authenticated session is created for the User
Actors	• User (Registered Reader, Staff)
Flow of Actions	BASIC FLOW 1. User navigates to the login page. 2. User enters their email/username and password. 3. User submits the form. 4. System validates the credentials.

	5. System creates an authenticated session and redirects the user to their dashboard. ALTERNATE FLOW A1 (SSO Login) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Invalid Credentials) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E2 (Suspended Account) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-6.3: Subscribe Use Case Name: Manage Account_Subscribe
--	---

Use Case Name: Subscribe

Use Case ID:	UC-6.3
Author	Joshua Concepcion
Purpose	Allow a Registered Reader to purchase a premium subscription plan, upgrading their access to Subscriber status.
Related Use Case	Includes: Manage Payment module — Subscribe to Premium Plan (UC-9.1)
Requirement Traceability	PR-ACC-03
Priority	High
Preconditions	• System is online • Registered Reader is authenticated and logged in

Postconditions	• On successful payment, the Reader's account is upgraded to Subscriber status
Actors	• Registered Reader • Payment Gateway (secondary, via include)
Flow of Actions	BASIC FLOW 1. Registered Reader selects "Subscribe." 2. System displays the available subscription plans. 3. Reader selects a plan. 4. System includes the Manage Payment module to process the transaction (UC-9.1). 5. Upon successful payment, system upgrades the Reader's account to Subscriber status. 6. System confirms the upgrade to the user. ALTERNATE FLOW A1 (Switch Plan Selection) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. ALTERNATE FLOW A2 (Existing Active Subscription) 1. The user opens the dashboard. 2. The system verifies session and permissions. 3. The system queries current records. 4. If no records match, the system displays the configured empty state. 5. If retrieval fails, the system displays a controlled error and retry option. 6. The system logs the failure when appropriate. 7. The user retries or changes the filter. ALTERNATE FLOW A3 (Checkout Cancellation) 1. The user chooses to cancel, discard, or leave the operation. 2. The system checks for unsaved changes or a pending transaction. 3. The system requests confirmation when the action could lose work or affect billing. 4. The user confirms cancellation. 5. The system discards only uncommitted changes or safely terminates the pending operation. 6. The system restores the last confirmed state and returns the user to the appropriate screen. 7. The system does not create an unintended version, state change, or entitlement. EXCEPTION FLOW E1 (Payment Failure) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules.

	4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-6.4: Update Profile Information Use Case Name: Manage Account_Update Profile Information
--	--

Use Case Name: Update Profile Information

Use Case ID:	UC-6.4
Author	Joshua Concepcion
Purpose	Allow a User to update their personal profile details, such as name, avatar, bio, and preferences.
Related Use Case	None
Requirement Traceability	PR-ACC-04
Priority	Low
Preconditions	• System is online • User is authenticated and logged in
Postconditions	• Profile information is updated
Actors	• User (Registered Reader, Staff)
Flow of Actions	BASIC FLOW 1. User navigates to Account Settings. 2. User selects "Update Profile Information." 3. System displays the current profile form. 4. User edits the desired fields. 5. User saves. 6. System validates and stores the updated profile. ALTERNATE FLOW A1 (Photo Upload) 1. The user selects a file and begins upload. 2. The system validates role, target assignment, file type, file size, and upload rules. 3. The system rejects invalid files before finalization. 4. For a valid file, the system transfers it to controlled storage.

	5. The system verifies transfer completion and file integrity. 6. The system creates a new version and preserves prior versions. 7. If interrupted, the system prevents the incomplete file from becoming active and marks it failed or resumable. 8. The user retries or resumes after correcting the problem. EXCEPTION FLOW E1 (Invalid Input) 1. The user enters or confirms information. 2. The system validates format, range, required fields, and business rules. 3. The system identifies invalid values. 4. The system highlights the affected fields and required corrections. 5. The system prevents persistence of invalid data. 6. The user corrects the values. 7. The system validates again and resumes the flow. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-6.5: Manage Account Security Use Case Name: Manage Account_Manage Account Security
--	--

Use Case Name: Manage Account Security

Use Case ID:	UC-6.5
Author	Joshua Concepcion
Purpose	Allow a User to change their password and manage security settings such as two-factor authentication.
Related Use Case	None
Requirement Traceability	PR-ACC-05
Priority	High
Preconditions	• System is online • User is authenticated and logged in
Postconditions	• Security settings are updated; other active sessions are invalidated if the password changed
Actors	• User (Registered Reader, Staff)
Flow of Actions	BASIC FLOW 1. User navigates to Account Settings. 2. User selects "Manage Account Security."

3. System displays security options: change password, enable two-factor authentication, view active sessions.
4. User updates the desired settings.
5. User confirms the changes.
6. System applies the changes and, if the password changed, invalidates other active sessions.

ALTERNATE FLOW

A1 (Enable 2FA)

1. The user reaches the condition described by this flow.
2. The system detects the condition during the relevant basic-flow step.
3. The system validates the current record, permission, and business rules.
4. The system performs the permitted alternate handling or blocks the invalid operation.
5. The system explains the resulting state and required next action.
6. The system records the event when audit/security rules require it.
7. The user continues from the appropriate point after the condition is resolved.

EXCEPTION FLOW

E1 (Incorrect Current Password)

1. The user reaches the condition described by this flow.
2. The system detects the condition during the relevant basic-flow step.
3. The system validates the current record, permission, and business rules.
4. The system performs the permitted alternate handling or blocks the invalid operation.
5. The system explains the resulting state and required next action.
6. The system records the event when audit/security rules require it.
7. The user continues from the appropriate point after the condition is resolved.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

1. The user initiates the protected operation.
2. The system validates the current session and required role or ownership permission immediately before the state-changing action.
3. The system detects an expired session or missing permission.
4. The system blocks the state change and does not partially commit it.
5. The system records the denied attempt when required by security policy.
6. The system redirects to authentication or displays access denied.
7. After authorization is restored, the user may retry.

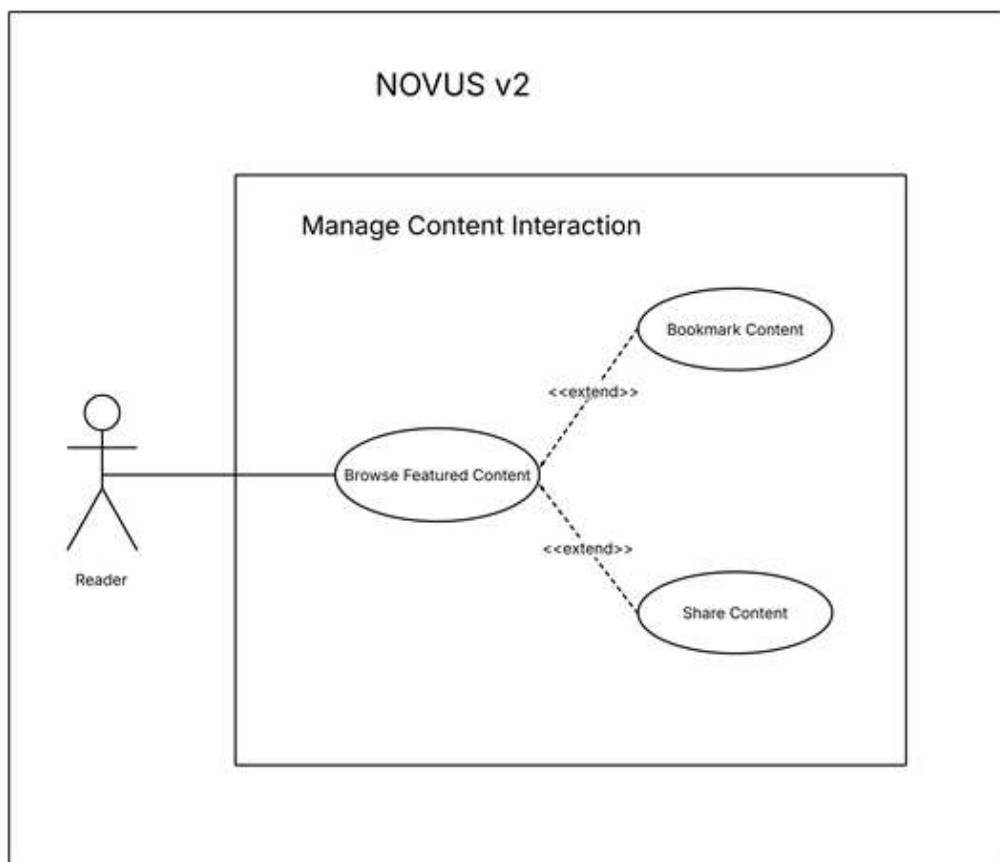
MODULE 7: Manage Content Interaction

Actors: Reader (Registered Reader)

UC-7.1: Browse Featured Content

Use Case Name: Manage Content Interaction_Browse Featured Content

Figure 8. Manage Content Interaction Use Case Diagram



Use Case Name: Browse Featured Content

Use Case ID:	UC-7.1
Author	Joshua Concepcion

Purpose	Allow a Reader to browse curated and featured articles and issues on the platform.
Related Use Case	• Extended by: Bookmark Content (UC-7.1.1), Share Content (UC-7.1.2)
Requirement Traceability	PR-INT-01
Priority	Medium
Preconditions	• System is online • Reader has access to the homepage or featured section
Postconditions	• Featured content is displayed to the Reader
Actors	• Reader
Flow of Actions	BASIC FLOW 1. Reader navigates to the homepage or featured section. 2. System retrieves and displays featured content. 3. Reader scrolls or browses through the featured items. 4. Reader selects an article to read; while viewing it, the Reader may optionally trigger Bookmark Content (UC-7.1.1) or Share Content (UC-7.1.2). ALTERNATE FLOW A1 (Category Filter) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (No Featured Content) 1. The user requests the selected list/report. 2. The system queries the current data set. 3. The system finds no matching records. 4. The system displays the empty state or configured fallback. 5. The system does not present fabricated or stale data. 6. The user adjusts filters or returns to another section. UC-7.1.1: Bookmark Content Use Case Name: Manage Content Interaction_Bookmark Content

Use Case Name: Bookmark Content

Use Case ID:	UC-7.1.1
Author	Joshua Concepcion
Purpose	Allow a Reader to save an article for later reading.
Related Use Case	• Extends: Browse Featured Content (UC-7.1)
Requirement Traceability	PR-INT-02

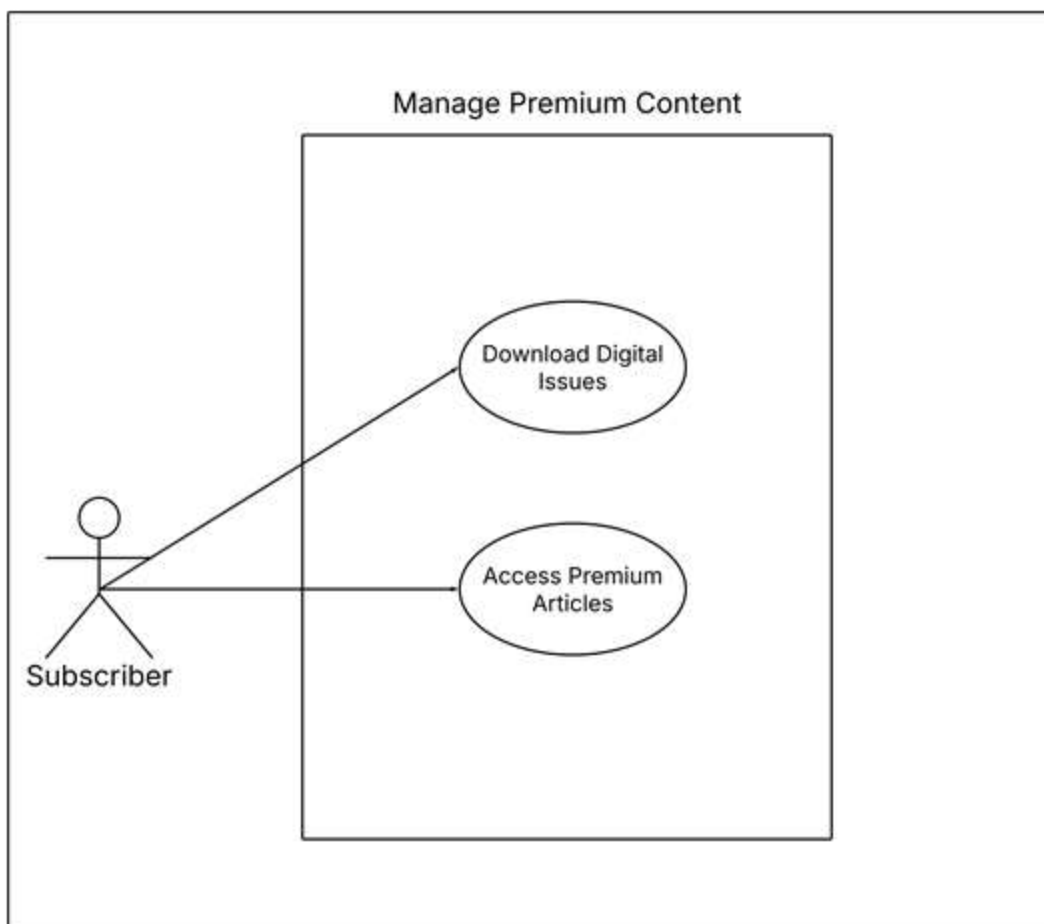
Priority	Low
Preconditions	• System is online • Reader is authenticated and logged in • Reader is viewing an article reached through Browse Featured Content (UC-7.1)
Postconditions	• Article is added to the Reader's bookmark list
Actors	• Reader
Flow of Actions	BASIC FLOW 1. While viewing an article reached through Browse Featured Content (UC-7.1), Reader selects "Bookmark." 2. System adds the article to the Reader's bookmark list. 3. System confirms with a visual indicator and returns control to Browse Featured Content (UC-7.1). ALTERNATE FLOW A1 (Remove Bookmark) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Not Logged In) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-7.1.2: Share Content Use Case Name: Manage Content Interaction_Share Content

Use Case Name: Share Content

Use Case ID:	UC-7.1.2
--------------	----------

Author	Joshua Concepcion
Purpose	Allow a Reader to share an article link via social media or a direct link copy.
Related Use Case	• Extends: Browse Featured Content (UC-7.1)
Requirement Traceability	PR-INT-03
Priority	Low
Preconditions	• System is online • Reader is viewing an article reached through Browse Featured Content (UC-7.1)
Postconditions	• A shareable link is generated and the share action is completed
Actors	• Reader
Flow of Actions	BASIC FLOW 1. While viewing an article reached through Browse Featured Content (UC-7.1), Reader selects “Share.” 2. System displays sharing options (social platforms, copy link). 3. Reader selects a channel. 4. System generates the shareable link, completes the share action, and returns control to Browse Featured Content (UC-7.1). ALTERNATE FLOW A1 (Copy Link) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Service Unreachable) 1. The user requests the external-service operation. 2. The system validates the endpoint and configured credentials. 3. The system attempts the connection or transaction. 4. The external service cannot be reached. 5. The system does not save unverified settings or treat the transaction as successful. 6. The system displays a controlled error and records diagnostics without exposing secrets. 7. The user retries after correction or service recovery. MODULE 8: Manage Premium Content Actors: Subscriber UC-8.1: Download Digital Issues Use Case Name: Manage Premium Content_Download Digital Issues

Figure 9. Manage Premium Content Use Case Diagram



Use Case Name: Download Digital Issues

Use Case ID:	UC-8.1
Author	Roman Rico Albania
Purpose	Allow a Subscriber to download a full digital issue for offline reading.
Related Use Case	None
Requirement Traceability	PR-PRM-01
Priority	Medium
Preconditions	• System is online • Subscriber is authenticated and logged in • Subscriber has an active premium subscription
Postconditions	• The digital issue file is downloaded to the Subscriber's device
Actors	• Subscriber
Flow of Actions	BASIC FLOW

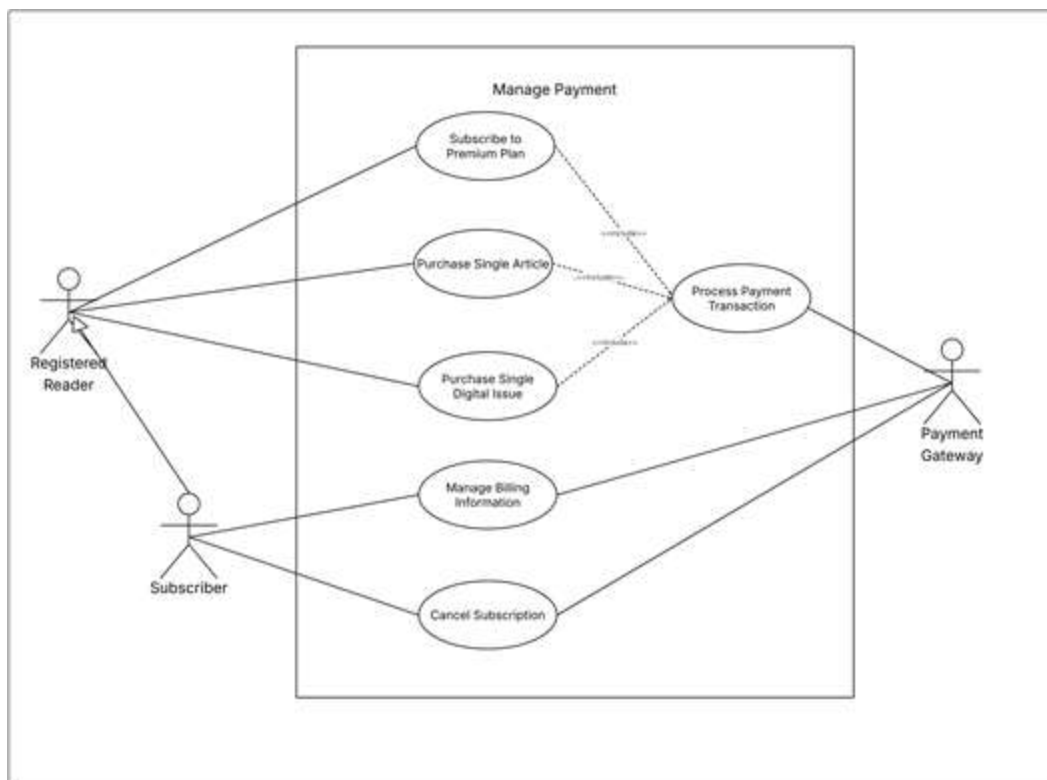
	1. Subscriber navigates to an available issue. 2. Subscriber selects "Download Digital Issue." 3. System verifies the Subscriber's active premium status. 4. System generates or serves the downloadable file. 5. Subscriber's device saves the file. ALTERNATE FLOW A1 (Format Choice) 1. The user selects a file and begins upload. 2. The system validates role, target assignment, file type, file size, and upload rules. 3. The system rejects invalid files before finalization. 4. For a valid file, the system transfers it to controlled storage. 5. The system verifies transfer completion and file integrity. 6. The system creates a new version and preserves prior versions. 7. If interrupted, the system prevents the incomplete file from becoming active and marks it failed or resumable. 8. The user retries or resumes after correcting the problem. EXCEPTION FLOW E1 (Expired Subscription) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. UC-8.2: Access Premium Articles Use Case Name: Manage Premium Content_Access Premium Articles
--	---

Use Case Name: Access Premium Articles

Use Case ID:	UC-8.2
Author	Roman Rico Albania
Purpose	Allow a Subscriber to view the full content of paywalled premium articles.
Related Use Case	None
Requirement Traceability	PR-PRM-02
Priority	High
Preconditions	• System is online • Subscriber is authenticated and logged in
Postconditions	• Full premium article content is rendered for the Subscriber
Actors	• Subscriber
Flow of Actions	BASIC FLOW 1. Subscriber selects a premium-marked article. 2. System verifies the Subscriber's active subscription status.

	3. System renders the full article content, bypassing the paywall preview. 4. Subscriber reads the article. ALTERNATE FLOW A1 (Single Article Access) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Inactive Subscription) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. MODULE 9: Manage Payment Actors: Registered Reader; Subscriber (generalized from Registered Reader); Payment Gateway (external) UC-9.1: Subscribe to Premium Plan Use Case Name: Manage Payment_Subscribe to Premium Plan
--	---

Figure 10. Manage Payment Use Case Diagram



Use Case Name: Subscribe to Premium Plan

Use Case ID:	UC-9.1
Author	Joshmar Clavero
Purpose	Process a Registered Reader's purchase of a recurring premium subscription plan via the Payment Gateway.
Related Use Case	• Included by: Subscribe (UC-6.3) • Includes: Process Payment Transaction (UC-9.1.1)
Requirement Traceability	PR-PAY-01
Priority	High
Preconditions	• System is online • Registered Reader is authenticated and logged in • Payment Gateway service is reachable
Postconditions	• Subscriber tier is activated on the Reader's account and the billing cycle is recorded
Actors	• Registered Reader • Payment Gateway (secondary, via include: UC-9.1.1)
Flow of Actions	BASIC FLOW 1. Registered Reader selects a subscription plan and confirms intent to subscribe. 2. System includes Process Payment Transaction (UC-9.1.1) to capture and process the payment.

	3. On receiving a success token from UC-9.1.1, System activates the Subscriber tier on the Reader's account and records the billing cycle. ALTERNATE FLOW A1 (Saved Payment Method) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Transaction Declined) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-9.1.1: Process Payment Transaction Use Case Name: Manage Payment_Process Payment Transaction
--	--

Use Case Name: Process Payment Transaction

Use Case ID:	UC-9.1.1
Author	Joshmar Clavero
Purpose	Serve as the central payment-processing engine that connects to the Payment Gateway, transmits transaction details, and returns a success or fail token to the invoking use case.
Related Use Case	• Included by: Subscribe to Premium Plan (UC-9.1), Purchase Single Article (UC-9.2), Purchase Single Digital Issue (UC-9.3)
Requirement Traceability	PR-PAY-06
Priority	High

Preconditions	• System is online • Payment Gateway service is reachable • The invoking use case has supplied the transaction amount and context
Postconditions	• Transaction is processed by the Payment Gateway and a success or fail token, along with a transaction reference, is returned to the invoking use case
Actors	• Registered Reader (indirect, via invoking use case) • Payment Gateway
Flow of Actions	BASIC FLOW 1. Invoking use case (UC-9.1, UC-9.2, or UC-9.3) triggers Process Payment Transaction and passes the transaction amount and context. 2. System connects to the Payment Gateway. 3. Reader enters or confirms payment details. 4. Payment Gateway processes the transaction. 5. Payment Gateway returns a success or fail token to the System. 6. System returns the token and transaction reference to the invoking use case. ALTERNATE FLOW A1 (Saved Payment Method) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Transaction Declined) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E2 (Gateway Unreachable) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation.

	- The system validates the current session and required role or ownership permission immediately before the state-changing action. - The system detects an expired session or missing permission. - The system blocks the state change and does not partially commit it. - The system records the denied attempt when required by security policy. - The system redirects to authentication or displays access denied. - After authorization is restored, the user may retry. UC-9.2: Purchase Single Article Use Case Name: Manage Payment_Purchase Single Article
--	---

Use Case Name: Purchase Single Article

Use Case ID:	UC-9.2
Author	Joshmar Claver
Purpose	Process a Registered Reader's one-time purchase of a single premium article without a full subscription.
Related Use Case	• Includes: Process Payment Transaction (UC-9.1.1)
Requirement Traceability	PR-PAY-02
Priority	Medium
Preconditions	- System is online - Registered Reader is authenticated and logged in
Postconditions	• Reader is granted permanent access to the purchased article
Actors	- Registered Reader - Payment Gateway (secondary, via include: UC-9.1.1)
Flow of Actions	BASIC FLOW - Registered Reader selects a premium article and chooses "Purchase Single Article." - System displays the price and includes Process Payment Transaction (UC-9.1.1) to capture and process the payment. - On receiving a success token from UC-9.1.1, System grants the Reader permanent access to that specific article. ALTERNATE FLOW A1 (Bundle Purchase) - The user reaches the condition described by this flow. - The system detects the condition during the relevant basic-flow step. - The system validates the current record, permission, and business rules. - The system performs the permitted alternate handling or blocks the invalid operation. - The system explains the resulting state and required next action. - The system records the event when audit/security rules require it. - The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Transaction Declined) - The user reaches the condition described by this flow. - The system detects the condition during the relevant basic-flow step.

	3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-9.3: Purchase Single Digital Issue Use Case Name: Manage Payment_Purchase Single Digital Issue
--	---

Use Case Name: Purchase Single Digital Issue

Use Case ID:	UC-9.3
Author	Joshmar Clavero
Purpose	Process a Registered Reader's one-time purchase of a single digital issue without a full subscription.
Related Use Case	• Includes: Process Payment Transaction (UC-9.1.1)
Requirement Traceability	PR-PAY-03
Priority	Medium
Preconditions	• System is online • Registered Reader is authenticated and logged in
Postconditions	• Reader is granted access and download rights to the purchased issue
Actors	• Registered Reader • Payment Gateway (secondary, via include: UC-9.1.1)
Flow of Actions	BASIC FLOW 1. Registered Reader selects an issue and chooses "Purchase Single Digital Issue." 2. System displays the price and includes Process Payment Transaction (UC-9.1.1) to capture and process the payment. 3. On receiving a success token from UC-9.1.1, System grants the Reader access and download rights to that issue. ALTERNATE FLOW A1 (Gift Issue) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules.

	- The system performs the permitted alternate handling or blocks the invalid operation. - The system explains the resulting state and required next action. - The system records the event when audit/security rules require it. - The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Transaction Declined) - The user reaches the condition described by this flow. - The system detects the condition during the relevant basic-flow step. - The system validates the current record, permission, and business rules. - The system performs the permitted alternate handling or blocks the invalid operation. - The system explains the resulting state and required next action. - The system records the event when audit/security rules require it. - The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) - The user initiates the protected operation. - The system validates the current session and required role or ownership permission immediately before the state-changing action. - The system detects an expired session or missing permission. - The system blocks the state change and does not partially commit it. - The system records the denied attempt when required by security policy. - The system redirects to authentication or displays access denied. - After authorization is restored, the user may retry. UC-9.4: Manage Billing Information Use Case Name: Manage Payment_Manage Billing Information
--	---

Use Case Name: Manage Billing Information

Use Case ID:	UC-9.4
Author	Joshmar Clavero
Purpose	Allow a Subscriber to view and update their stored billing and payment details.
Related Use Case	• None
Requirement Traceability	PR-PAY-04
Priority	Medium
Preconditions	- System is online - Subscriber is authenticated and logged in
Postconditions	Billing information is updated and validated with the Payment Gateway
Actors	- Subscriber - Payment Gateway
Flow of Actions	BASIC FLOW - Subscriber navigates to Billing Settings. - Subscriber selects "Manage Billing Information."

	3. System retrieves current billing details via the Payment Gateway (masked card info). 4. Subscriber updates the payment method or billing address. 5. System validates the update with the Payment Gateway. 6. System confirms the update. ALTERNATE FLOW A1 (Backup Payment Method) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Method Rejected) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. UC-9.5: Cancel Subscription Use Case Name: Manage Payment_Cancel Subscription
--	--

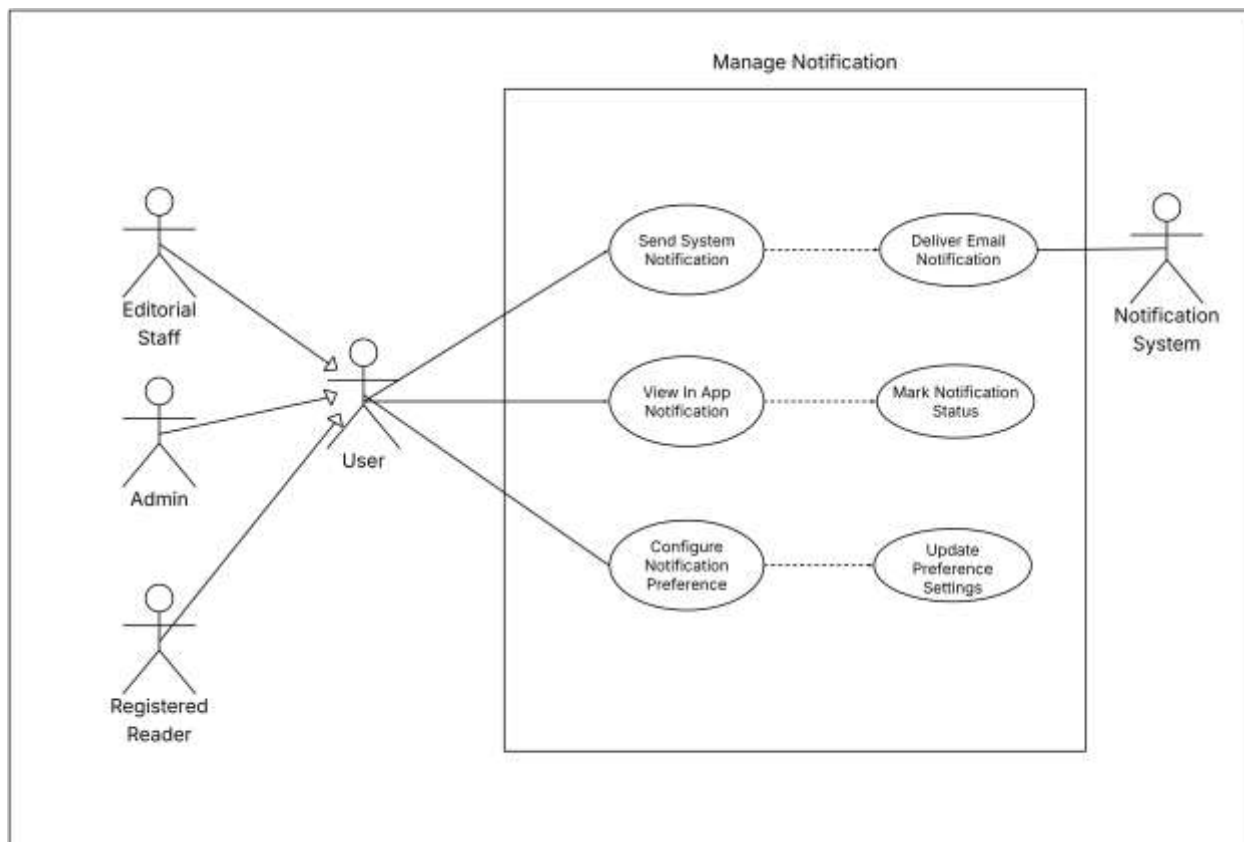
Use Case Name: Cancel Subscription

Use Case ID:	UC-9.5
Author	Joshmar Clavero
Purpose	Allow a Subscriber to cancel their recurring premium subscription.
Related Use Case	• None
Requirement Traceability	PR-PAY-05
Priority	High

Preconditions	• System is online • Subscriber is authenticated and logged in • Subscriber has an active subscription
Postconditions	• Recurring charges are stopped and the account is scheduled to downgrade at the end of the billing cycle
Actors	• Subscriber • Payment Gateway
Flow of Actions	BASIC FLOW 1. Subscriber navigates to Billing Settings. 2. Subscriber selects "Cancel Subscription." 3. System prompts for confirmation and an optional cancellation reason. 4. Subscriber confirms. 5. System notifies the Payment Gateway to stop future recurring charges. 6. System schedules the downgrade to Registered Reader tier at the end of the current billing cycle. ALTERNATE FLOW A1 (Rescind Cancellation) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E1 (Gateway Acknowledgment Failure) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry. Revision Note: Expanded every documented Alternate and Exception Flow across all nine modules into explicit numbered procedures. UC-1.4 and UC-1.8 now support Editor-requested revisions for both Writer articles and Graphics

Designer magazine designs. Module 9 contains no promo-code or discount-code processing.

Figure 11. Manage Notification Use Case Diagram



Use Case Name: Send System Notification

Use Case ID:	UC-10.1
Author	Rei Khalil Galido
Purpose	Serve as the shared, system-triggered mechanism that creates and dispatches a notification whenever a triggering event occurs in another NOVUS module (e.g., a revision request, a magazine design upload, a new staff account, a forced password reset, or account registration), and routes it to the applicable channel(s).
Related Use Case	- Includes: Deliver Email Notification (UC-10.1.1) - Invoked internally by (originating events identified in the source paper): Request Revision (UC-1.4); Upload Magazine Design (UC-1.8); Trigger Subscriber Notifications (UC-2.3.1); Add New User (UC-5.1); Force Password Reset (UC-5.2.3); Register Account (UC-6.1)

Requirement Traceability	PR-NOT-01
Priority	High
Preconditions	• System is online • An originating use case has reached a point in its flow that requires notifying a User, and has supplied the recipient, event type, and message content
Postconditions	• A notification record is created and associated with the recipient • The notification is dispatched to the applicable channel(s) — in-app and/or email, via UC-10.1.1 • Delivery status is recorded and returned to the originating use case
Actors	• System (primary — internal trigger, invoked via include from the originating use case) • User — Editorial Staff, Admin, or Registered Reader (recipient) • Notification System / Email Service (secondary, via include: UC-10.1.1 — email channel only)
Flow of Actions	BASIC FLOW 1. An originating use case (e.g., Request Revision UC-1.4, Upload Magazine Design UC-1.8, Trigger Subscriber Notifications UC-2.3.1, Add New User UC-5.1, Force Password Reset UC-5.2.3, or Register Account UC-6.1) reaches the point in its flow where the source specifies that the system notifies a role or sends the user a message. 2. The originating use case invokes Send System Notification, supplying the recipient, event type, and message content. 3. System creates a notification record and associates it with the recipient User. 4. System determines the delivery channel(s) required for the event (in-app, email, or both), consistent with the channel options documented for Trigger Subscriber Notifications (email, push, in-app). 5. System stores the notification for in-app display and, where an email channel applies, includes Deliver Email Notification (UC-10.1.1) to dispatch the email through the Notification System / Email Service. 6. System returns the delivery status to the originating use case. ALTERNATE FLOW — A1 (Segment / Batch Notification) 1. The originating use case (e.g., Trigger Subscriber Notifications, UC-2.3.1) supplies a recipient segment instead of a single recipient. 2. System resolves the segment to its member Users. 3. System creates an individual notification record for each member of the segment. 4. System proceeds with channel determination and dispatch for each member per Basic Flow steps 4–6. EXCEPTION FLOW — E1 (Notification Failure) 1. System successfully creates the notification record. 2. System attempts to dispatch the notification. 3. The notification service fails to deliver the message.

	4. System keeps the originating use case's record state unchanged; no partial state change is committed on account of the failed notification. 5. System records the failed delivery and marks the notification for retry or follow-up. 6. The affected user can see a follow-up indicator on their dashboard. 7. System retries delivery when the notification service becomes available.
Source Note	<i>Grounded directly in UC-1.4 Request Revision (Basic Flow step 11 and Exception Flow E1, which names "the notification service" and its retry behavior), UC-1.8 Upload Magazine Design (Editor is notified), UC-2.3.1 Trigger Subscriber Notifications (full use case), UC-5.1 Add New User ("sends setup credentials to the user's email"), UC-5.2.3 Force Password Reset ("emails the user a reset link"), and UC-6.1 Register Account ("sends a verification email"). The supplied UML draws an association from User to Send System Notification; the source narrative for every originating event shows the System triggering the notification internally rather than a User opening a screen to compose one. User is documented here as the recipient/beneficiary of the underlying event, consistent with the UML's inheritance structure, while the Flow of Actions reflects the system-internal trigger shown in the source.</i>

Use Case Name: Deliver Email Notification

Use Case ID:	UC-10.1.1
Author	Rei Khalil Galido
Purpose	Serve as the shared extension that connects to the Notification System / Email Service to construct and dispatch a transactional email for a given notification event.
Related Use Case	• Included by: Send System Notification (UC-10.1)
Requirement Traceability	PR-NOT-04
Priority	High
Preconditions	• System is online • Notification System / Email Service is reachable • The invoking use case (UC-10.1) has supplied the recipient email address, event type, and message content
Postconditions	• The email is transmitted to the Notification System / Email Service • A delivery status (success or failure) is returned to Send System Notification (UC-10.1)
Actors	• System (primary, invoked via include) • Notification System / Email Service (secondary)
Flow of Actions	BASIC FLOW 1. Send System Notification (UC-10.1) invokes Deliver Email Notification, supplying the recipient email address, event type, and message content.

	2. System selects the email template/content appropriate to the supplied event type (e.g., account verification, new-user setup credentials, password reset link, revision notification, design review notification, subscriber announcement). 3. System populates the template with the event-specific data provided by the invoking use case. 4. System connects to the Notification System / Email Service and transmits the email. 5. Notification System / Email Service dispatches the email and returns a delivery status. 6. System returns the delivery status to Send System Notification (UC-10.1). ALTERNATE FLOW — A1 (Digital Receipt Template) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW — E1 (Undeliverable Email) 1. System attempts to transmit the email via the Notification System / Email Service. 2. The Notification System / Email Service reports the address as undeliverable, or the connection fails. 3. System does not mark the originating event's own postconditions (e.g., password reset, verification) as complete on account of the failed email alone. 4. System records the failed delivery and marks it for retry or follow-up. 5. System retries delivery when the Notification System / Email Service becomes available, or surfaces the failure so the underlying configuration can be corrected (see UC-4.2 Configure System Integrations).
Source Note	<i>Grounded in the technical architecture: "Resend: A developer-focused email service utilized to reliably construct and dispatch transactional communications, including system notifications and digital receipts," and the Component Diagram note that "Resend supporting transactional email delivery." The Exception Flow reuses the exact scenario name ("Undeliverable Email") already documented for UC-5.2.3 Force Password Reset, and the retry behavior documented in UC-1.4 Exception Flow E1.</i>

Use Case Name: View In-App Notification

Use Case ID:	UC-10.2
Author	Rei Khalil Galido

Purpose	Allow a logged-in User to open a central Notification Panel to view recent notification events, with unread items visually marked, and navigate directly to the related screen.
Related Use Case	• Includes: Mark Notification Status (UC-10.2.1)
Requirement Traceability	PR-NOT-02
Priority	Medium
Preconditions	• System is online • Notification System / Email Service is reachable • The invoking use case (UC-10.1) has supplied the recipient email address, event type, and message content
Postconditions	• Notification Panel is displayed showing the User's recent notification events, newest first, with unread items visually marked
Actors	• User (Editorial Staff, Admin, Registered Reader)
Flow of Actions	BASIC FLOW 1. User taps the notification bell icon from any authenticated screen. 2. System opens the Notification Panel, listing recent events, newest first, with unread items visually marked. 3. User taps a notification. 4. System includes Mark Notification Status (UC-10.2.1) to mark the selected notification read. 5. System navigates the User directly to the screen relevant to that notification. ALTERNATE FLOW — A1 (Mark All As Read) 1. User taps "Mark all as read." 2. System includes Mark Notification Status (UC-10.2.1) for all of the User's unread notifications. 3. System clears the unread markers and displays a confirmation label. 4. User remains on the Notification Panel. EXCEPTION FLOW — E1 (Notification Load Failure) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved.
Source Note	<i>Basic Flow and Alternate Flow A1 are grounded directly in the source's "Viewing In-App Notifications" user story (all roles, system-wide): tapping the bell icon, the Notification Panel, unread markers, tap-to-navigate, and "Mark all as read." No</i>

	<i>dedicated Exception Flow is narrated in the source for this screen; E1 is modeled on the equivalent pattern already used for other "View" use cases (UC-1.9 View Pipeline Dashboard E1 "Data Load Failure"; UC-5.2 View User Directory E1 "Load Failure"). No E-AUTH is included, consistent with the source's pattern for read-only View use cases (UC-1.9, UC-1.10, UC-5.2, UC-7.1), none of which carry E-AUTH.</i>
--	---

Use Case Name: Mark Notification Status

Use Case ID:	UC-10.2.1
Author	Rei Khalil Galido
Purpose	Serve as the shared extension that updates a notification's read/unread status for the User, individually or in bulk.
Related Use Case	• Included by: View In-App Notification (UC-10.2)
Requirement Traceability	PR-NOT-05
Priority	Low
Preconditions	• System is online • User is authenticated and logged in • The invoking use case (UC-10.2) has supplied the target notification(s) and the new status
Postconditions	• The specified notification(s) unread marker is updated and reflected the next time the Notification Panel is displayed
Actors	• User (Editorial Staff, Admin, Registered Reader)
Flow of Actions	BASIC FLOW 1. View In-App Notification (UC-10.2) invokes Mark Notification Status, supplying the target notification and "read" as the new status. 2. System validates that the target notification belongs to the requesting User. 3. System updates the notification's status to read. 4. System returns the updated status to View In-App Notification (UC-10.2). ALTERNATE FLOW — A1 (Bulk Mark As Read) 1. View In-App Notification (UC-10.2) supplies the full set of the User's unread notifications. 2. System validates that all target notifications belong to the requesting User. 3. System updates all target notifications to read in a single operation. 4. System returns the updated count/status to View In-App Notification (UC-10.2). EXCEPTION FLOW — E1 (Update Failure) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules.

	4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW — E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry.
Source Note	<i>Basic Flow and Alternate Flow A1 are grounded in the source's "tap a notification → system marks it read" / "Mark all as read" behavior. E-AUTH is included by analogy to Bookmark Content (UC-7.1.1) — the source's closest equivalent "save/flag a personal status while viewing content" action, which carries E-AUTH.</i>

Use Case Name: Configure Notification Preference

Use Case ID:	UC-10.3
Author	Rei Khalil Galido
Purpose	Allow a User to specify which notification channel(s) — email and in-app, per the channel options already documented for Trigger Subscriber Notifications (email, push, in-app) — they wish to receive on their account.
Related Use Case	• Includes: Update Preference Settings (UC-10.3.1)
Requirement Traceability	PR-NOT-03
Priority	Low
Preconditions	• System is online • User is authenticated and logged in
Postconditions	• The User's notification preference settings are updated and applied to future notification dispatch
Actors	• User (Editorial Staff, Admin, Registered Reader)
Flow of Actions	BASIC FLOW 1. User navigates to Account Settings. 2. User selects "Notification Preferences." 3. System displays the current channel selection (email, in-app) for each notification category. 4. User edits the desired channel toggles.

	5. User saves. 6. System includes Update Preference Settings (UC-10.3.1) to validate and persist the change. 7. System confirms the update to the User. ALTERNATE FLOW — A1 (Reset to Default) 1. User selects "Reset to Default." 2. System includes Update Preference Settings (UC-10.3.1), supplying the platform default channel selection for each notification category. 3. System persists the default values as the User's current preference. 4. System confirms the reset to the User. EXCEPTION FLOW — E1 (Invalid Configuration) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved. EXCEPTION FLOW — E-AUTH (Authorization or Session Failure) 1. The user initiates the protected operation. 2. The system validates the current session and required role or ownership permission immediately before the state-changing action. 3. The system detects an expired session or missing permission. 4. The system blocks the state change and does not partially commit it. 5. The system records the denied attempt when required by security policy. 6. The system redirects to authentication or displays access denied. 7. After authorization is restored, the user may retry.
Source Note	<i>NOT SPECIFIED IN THE SOURCE as a stand-alone screen — no explicit "notification preferences" behavior was found in the paper. This use case is structured only by analogy to the account-settings pattern documented for Update Profile Information (UC-6.4) and Manage Account Security (UC-6.5), and the "Revert to Default" / "Invalid Configuration" pattern documented for Update Site Theme and Layout (UC-4.1), since Module 10 requires it per the supplied UML. Channel names (email, in-app) are the only channel vocabulary explicitly present in the source (Trigger Subscriber Notifications, UC-2.3.1). E-AUTH is included by analogy to UC-6.4 / UC-6.5, both of which carry it as personal, state-changing settings use cases.</i>

Use Case Name: Update Notification Preference

Use Case ID:	UC-10.3.1
Author	Rei Khalil Galido

Purpose	Serve as the shared extension that validates and persists a User's notification-channel preference values.
Related Use Case	• Included by: Configure Notification Preference (UC-10.3)
Requirement Traceability	PR-NOT-06
Priority	Low
Preconditions	• System is online • User is authenticated and logged in • The invoking use case (UC-10.3) has supplied the new preference values, or a reset instruction
Postconditions	• The updated preference values are persisted and take effect for future notification dispatch
Actors	• Registered User
Flow of Actions	BASIC FLOW 1. Configure Notification Preference (UC-10.3) supplies the new channel selections for each notification category. 2. System validates the submitted values against the supported channels (email, in-app, per the channel options documented for Trigger Subscriber Notifications). 3. System persists the updated preference record for the User. 4. System returns the confirmation to Configure Notification Preference (UC-10.3). ALTERNATE FLOW — A1 (Reset to Default Values) 1. Configure Notification Preference (UC-10.3) supplies a reset instruction instead of explicit values. 2. System looks up the platform default channel selection for each notification category. 3. System persists the default values as the User's current preference. 4. System returns the confirmation to Configure Notification Preference (UC-10.3). EXCEPTION FLOW — E1 (Persistence Failure) 1. The user reaches the condition described by this flow. 2. The system detects the condition during the relevant basic-flow step. 3. The system validates the current record, permission, and business rules. 4. The system performs the permitted alternate handling or blocks the invalid operation. 5. The system explains the resulting state and required next action. 6. The system records the event when audit/security rules require it. 7. The user continues from the appropriate point after the condition is resolved.

Test Cases based on Fully Dressed Use Case

Module	Use Cases	Test Cases
Module 1: Manage Editorial Pipeline	11	42
Module 2: Manage Publication	5	19
Module 3: Manage Reports	4	12
Module 4: Manage Platform Settings	3	12
Module 5: Manage User Privileges	5	16
Module 6: Manage Account	5	21
Module 7: Manage Content Interaction	3	10
Module 8: Manage Premium Content	2	6
Module 9: Manage Payment	6	25
Module 10: Manage Notification		
Total	44	163

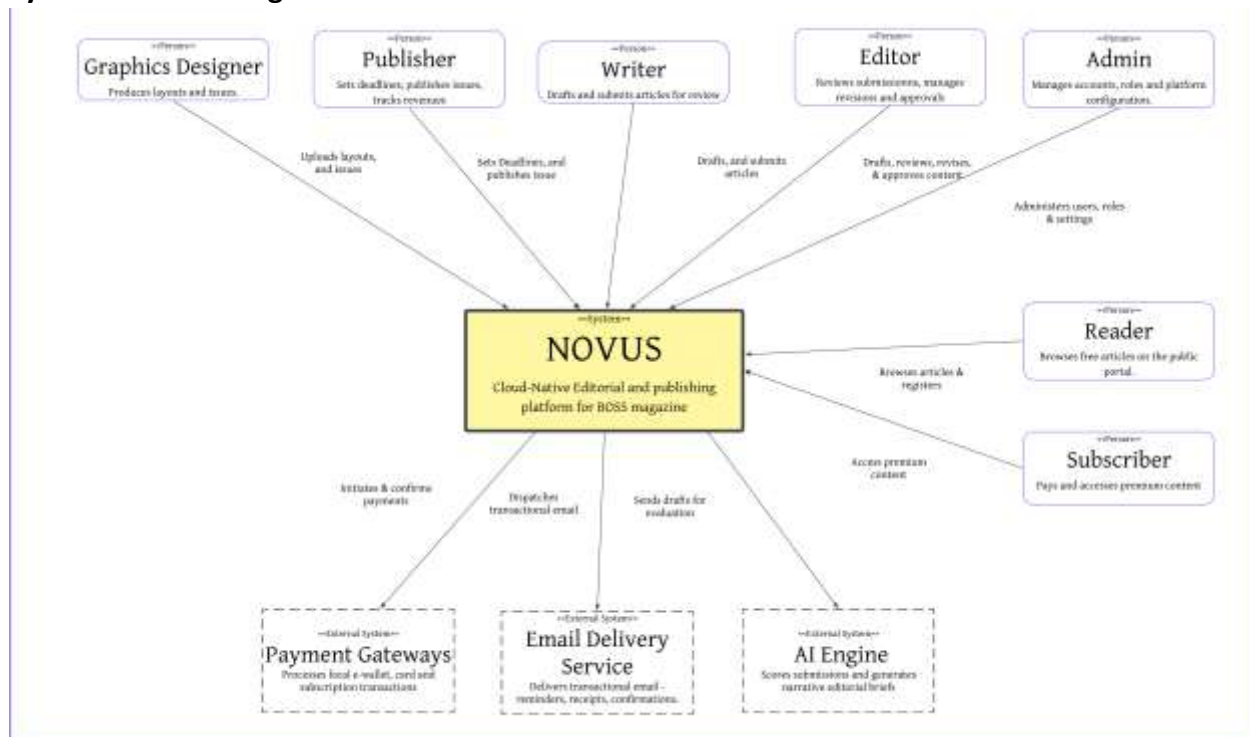
[NOVUS Test Case Table.docx](#)

Gap Analysis

Product Backlog ID	Current System	Proposed Changes	Impact

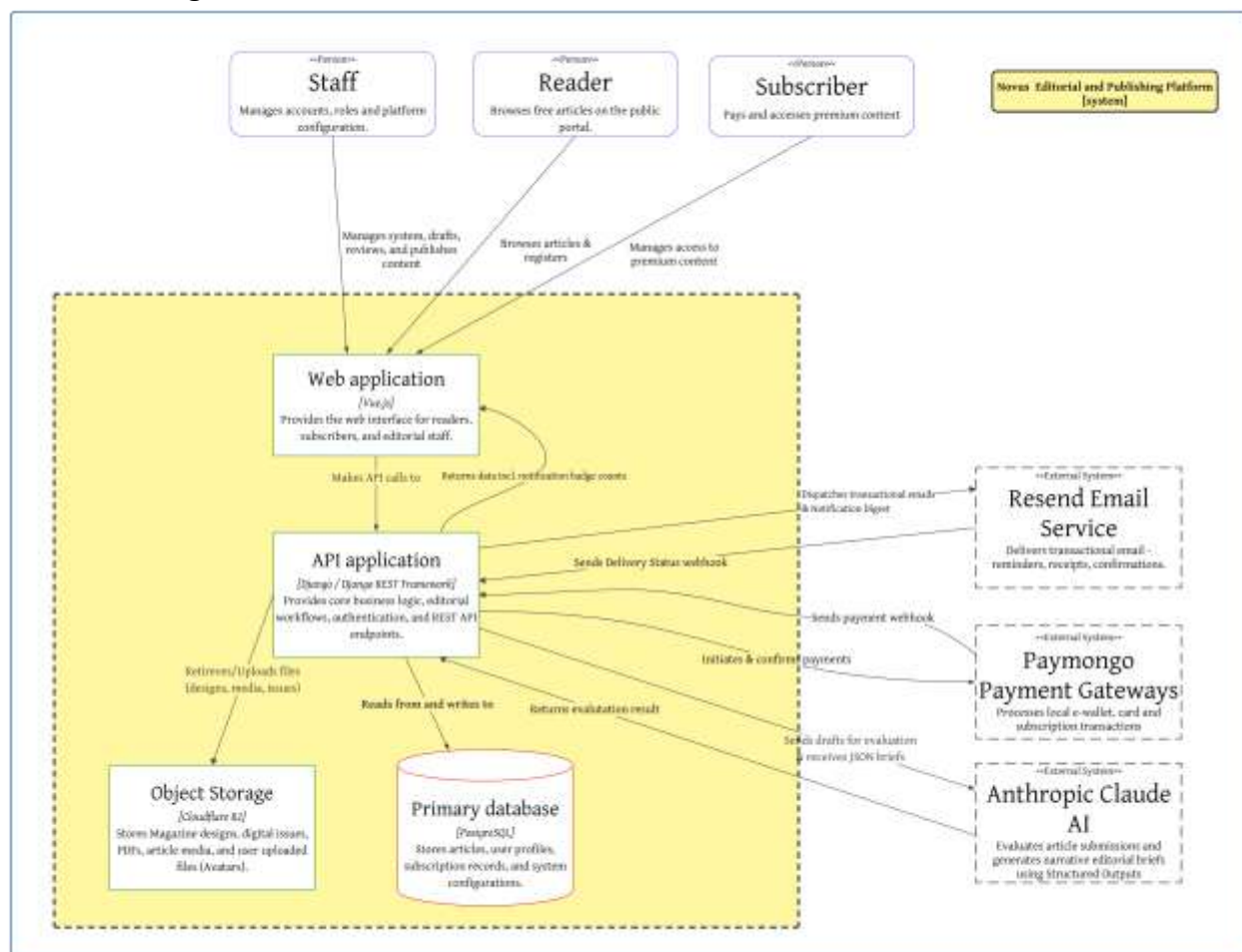
System Analysis and Design

System Context Diagram



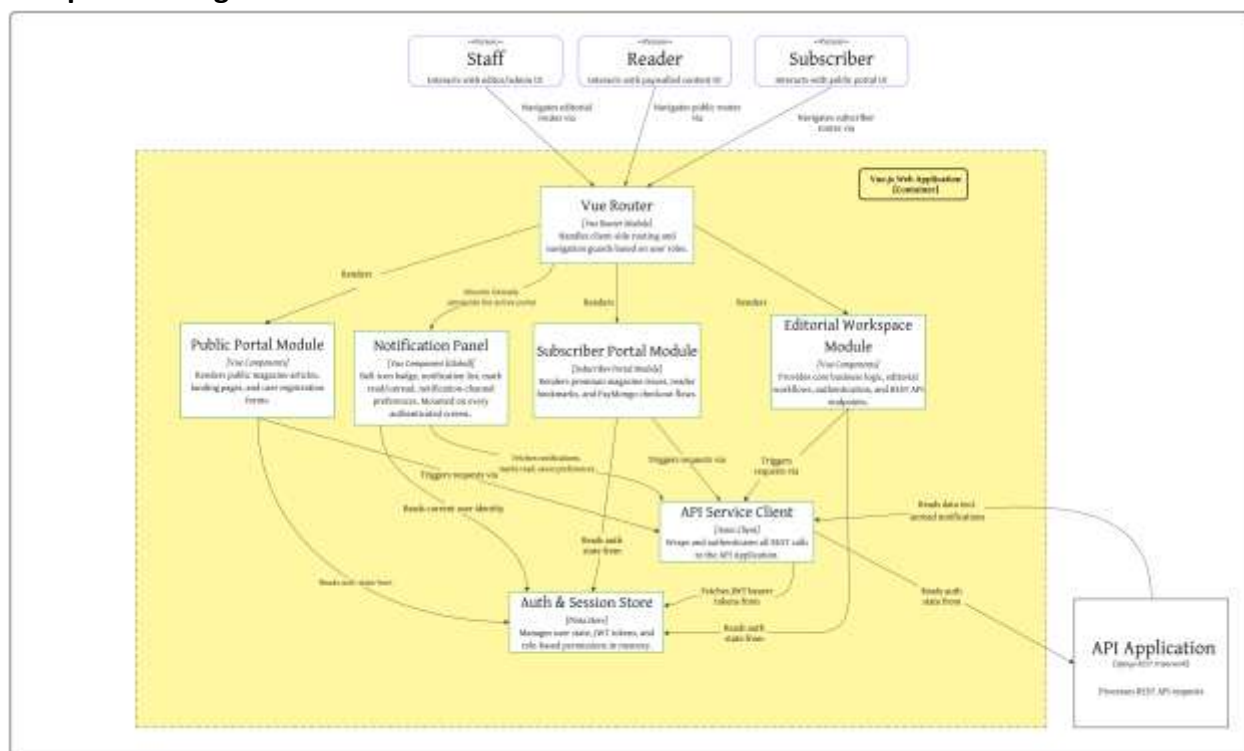
The Level 1 C4 diagram shows NOVUS as the central cloud-native publishing platform connecting its main users and external services. Writers, Editors, Publishers, Graphics Designers, and Admins manage content and platform operations, while Readers and Subscribers access published content. NOVUS also integrates payment gateways, an email delivery service, and an AI engine to support payments, notifications, and content evaluation.

Container Diagram



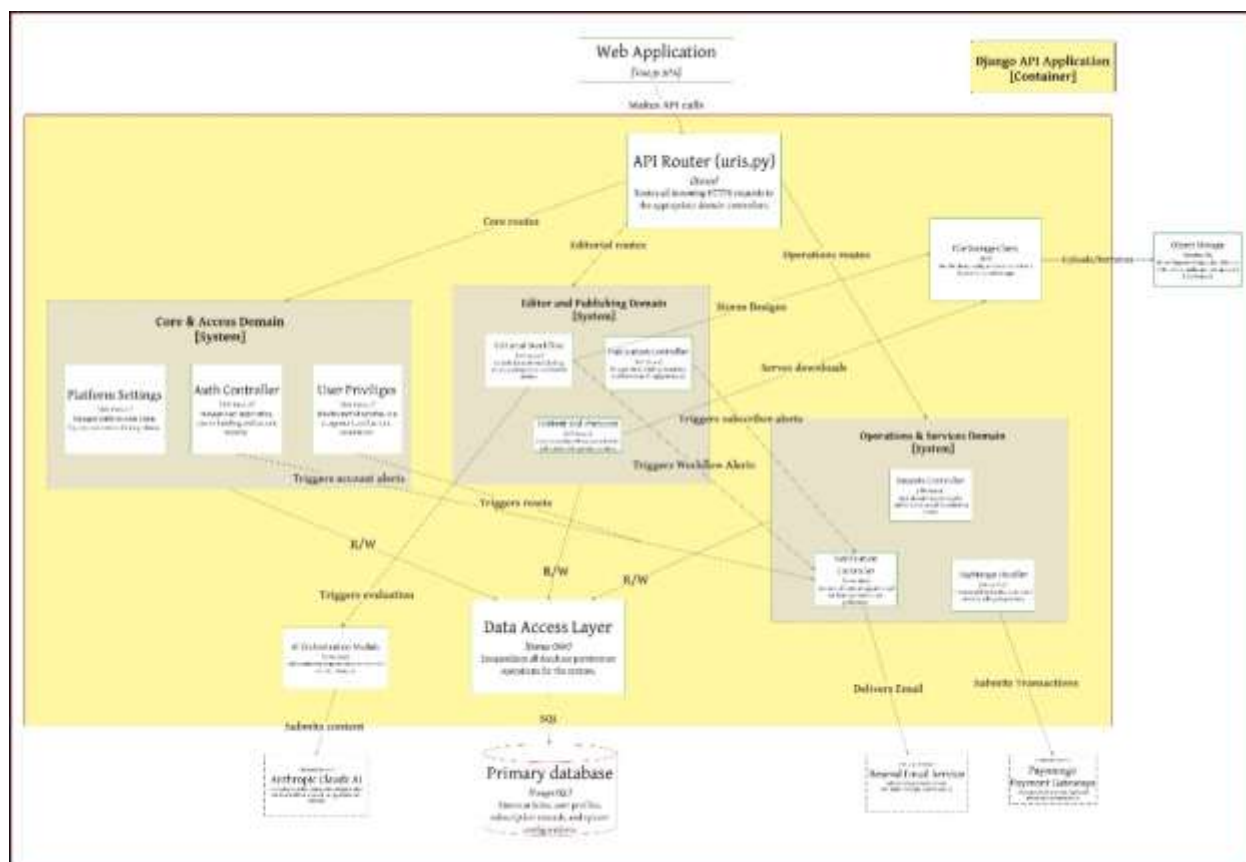
This Level 2 container diagram maps the high-level architecture of the Novus Editorial and Publishing Platform, demonstrating how Staff, Readers, and Subscribers interact with the system through a central Vue.js Web Application. All client requests are routed to a core Django API Application, which acts as the central hub for business logic, authentication, and editorial workflows. System data is securely distributed between a PostgreSQL Primary Database for structured records and Cloudflare R2 Object Storage for digital assets, layouts, and media. To deliver its full suite of features, the API application seamlessly integrates with three external services: Resend for automated email notifications, PayMongo for secure subscription billing, and Anthropic Claude AI for intelligent article evaluations.

Component Diagram



Vue.js Web Application

This Level 3 component diagram illustrates the frontend architecture of the Vue.js Web Application, detailing how different user roles (Staff, Readers, and Subscribers) interact with the system. User navigation is managed by a central Vue Router, which dynamically directs traffic to role-specific interface modules: the Public Portal, the Subscriber Portal, and the Editorial Workspace, alongside a globally mounted Notification Panel. Internal state, JWT tokens, and role-based permissions are securely managed by a Pinia-based Auth & Session Store, while an Axios-powered API Service Client handles and authenticates all outbound REST requests to the backend Django API Application.



Django API Application

This Level 3 component diagram outlines the internal architecture of the Novus Django API application, illustrating how the backend processes and routes requests from the Vue.js frontend. At its core, an API Router dispatches incoming traffic to three dedicated operational silos: the Core & Access Domain (handling authentication, settings, and user management), the Editor and Publishing Domain (managing the article lifecycle, premium content, and Claude AI evaluations), and the Operations & Services Domain (overseeing PayMongo billing, reporting, and Resend email notifications). All domain controllers share a centralized Data Access Layer for seamless PostgreSQL database persistence and utilize specialized clients for external cloud storage, creating a robust, unified backend to drive the editorial workflow for BOSS Magazine PH.

Entity Relationship Diagram

Results and Discussion

(

Write a short introduction that describes what this section will discuss.

)

Design Thinking Stages

Design Thinking is a human-centered problem-solving process with five stages: empathizing with users, defining the problem, generating ideas, creating prototypes, and test solutions. Allowing continuous improvement until the best solution is reached.

Stage 1: Empathize

The first phase of the design thinking process is to have a deep understanding of the target users and to identify the problems they have at hand. The team lay aside all their assumptions about the problem, and the target users. This allows us to objectively consider all possibilities for the customers and their needs. As design thinkers, we immerse ourselves in their environment and experiences through research, interviews, and observations. The goal here is to build empathy by viewing situations from their perspective,, understanding not only what they say, but also how they act, what they think, and how they feel.



To achieve this, we conducted interviews as a group. Our aim was to create a comfortable atmosphere that would encourage them to provide answers without hesitation. By practicing active listening and asking thoughtful, open-ended questions, we were able to gather authentic insights. This process allowed us to uncover their true needs, motivations, and pain points, helping us identify the underlying challenges that require attention.

Client Interviews

The following are the questions the group has made for our client:

Question	Answer
On average, how many articles/stories or pages are submitted per publishing cycle?	More than 10 articles/Stories
How long does it typically take (<i>in days</i>) from when an article is submitted to when you actually open and begin reviewing it?	It takes around 1-2 days from being submitted to being actually opened and reviewed

How often do multiple article submissions arrive on the same day?	Frequently more than half the time
On average, how many revision rounds does an article go through before it is approved?	It varies widely per writer
On average, how many days pass from when an article is submitted to when a final decision (approved or rejected) is made?	It normally takes around 1-2 days

5 Whys Simplified

Why does BOSS Magazine PH experience workflow delays, limited revenue, and difficulty scaling?

- Because its editorial, publishing, and business activities are handled mostly through manual processes and disconnected platforms.

Why are the processes manual and disconnected?

- Because BOSS Magazine PH **does not have a centralized digital platform** that manages content, editorial workflows, publishing, and digital business functions.

Why does the magazine not have a centralized digital platform?

- Because its current operations have relied primarily on social media, messaging applications, email, and other general-purpose tools rather than a dedicated publishing infrastructure.

Why is relying on these general-purpose tools becoming a problem?

- Because the magazine's content volume and operational responsibilities have grown, while its existing tools do not provide sufficient workflow tracking, automation, collaboration, analytics, or monetization capabilities.

Why can't the current setup support the magazine's growth and sustainability?

- Because BOSS Magazine PH lacks a dedicated digital infrastructure designed to integrate its editorial, publishing, audience reach, and revenue processes into one scalable system.

Root Cause

BOSS Magazine PH lacks a dedicated and centralized digital infrastructure that integrates editorial workflow, publishing operations, audience reach, and revenue generation, resulting in manual work, operational inefficiencies, limited scalability, and missed digital business opportunities.

Actual Personas

Theoretical Personas

PERSONA 1: MARIA

Magazine editor

Manages multiple writers and article submissions

Struggles with tracking articles, deadlines, revisions, and communication

PERSONA 2: DANIEL

Magazine writer

Writes and submits articles for publication

Struggles with unclear instructions, fragmented communication, and delayed feedback

PERSONA 3: ROBERT

Magazine publisher

Oversees the quality, production, and publishing of magazine issues

Struggles with missed deadlines, late problems, and lack of visibility into the publishing process

PERSONA 4: ALEX

Graphic designer

Creates magazine layouts and visual content

Struggles with late content changes, unclear design directions, and repeated layout revisions

Empathy Map

Say	Thinks
• "I don't have a formal publishing schedule — I just try to keep track in my head" • "When a deadline is missed, it takes more than 2 weeks to recover" • "One late article can delay the whole issue" • "I noticed some submissions were heavily AI-generated — I could tell just from reading it" • "The more writers we add, the more work falls on me" • "We need to start making money from our content digitally — we can't just keep publishing for free."	• "I'm the only one reviewing everything — if I fall behind, the whole issue falls behind" • "I can't tell which articles are genuinely in progress and which ones writers have forgotten about" • "Some of these submissions feel like they were written by AI — I had to catch it myself with no tool to help me" • "The magazine produces quality content but has no way to earn from it. We are doing the work of a real publication without any of the revenue that should come with it." • "I spend more time managing messages and chasing people than I do actually editing" • "If one writer misses their deadline, it's not just their article that's late — it's everyone else's too" • "I'm reviewing the same types of basic errors over and over — there has to be a better way" • "I don't even know how many hours I'm putting in until I look back at the week"
Does	Feels
• Receives 10+ articles per publishing cycle across 4+ communication platforms (Messenger, email, Viber, etc.) • Spends 1-2 days reviewing each submitted article • Conducts multiple revision rounds per article; writers take 3-5 days to resubmit • Makes final approval decisions in 1-2 days per article • Reviews at least 5 articles per publishing cycle — only 3-4 out of 10 need no revision • Spends 1-2 hours daily chasing writers for updates or reminders • Spends 2+ hours daily writing and explaining revision notes to writers • Handles 21-49 messages per working day related to article management alone • Commits 31-40 hours per week on editorial tasks • Takes 2+ weeks to complete a full publishing cycle	• Overwhelmed — 31-40 hours of editorial work per week with no system, no automation, no support • Frustrated — spending 2+ hours a day just chasing writers and explaining revisions that could be standardized • Anxious — one missed deadline doesn't just affect one article, it cascades into the entire issue • Unsupported — she is the only editor, and the workload scales with every new writer added • Vigilant but exhausted — having to manually detect AI-generated content on top of everything else • Scattered — managing 21-49 messages a day across 4+ platforms with no single source of truth • Responsible but isolated — she cares deeply about the magazine's quality but carries the entire editorial burden alone • She recognizes that the magazine needs to generate digital revenue to be sustainable, but has no platform, no subscription system, and no mechanism to act on that ambition under the current setup. Commercially aware but stuck.

1. EDITOR

SAY

- I don't have a formal publishing schedule I just try to keep track in my head.
- When a deadline is missed, it takes more than 2 weeks to recover.
- One late article can delay the whole issue.
- I noticed some submissions were heavily AI-generated, I could tell just from reading it.
- The more writers we add, the more work falls on me.
- We need to start making money from our content digitally we can't just keep publishing for free

THINKS

- I'm the only one reviewing everything if I fall behind, the whole issue falls behind.
- I can't tell which articles are genuinely in progress and which ones writers have forgotten about.
- Some of these submissions feel like they were written by AI I had to catch it myself with no tool to help me.
- The magazine produces quality content but has no way to earn from it. We are doing the work of a real publication without any of the revenue that should come with it.
- I spend more time managing messages and chasing people than I do actually editing.
- If one writer misses their deadline, it's not just their article that's late it's everyone else's too.
- I'm reviewing the same types of basic errors over and over there has to be a better way.
- I don't even know how many hours I'm putting in until I look back at the week.

DOES

- Receives 10+ articles per publishing cycle across 4+ communication platforms (Messenger, email, Viber, etc.).
- Spends 1–2 days reviewing each submitted article.
- Conducts multiple revision rounds per article; writers take 3–5 days to resubmit.
- Makes final approval decisions in 1–2 days per article.
- Reviews at least 5 articles per publishing cycle only 3–4 out of 10 need no revision.
- Spends 1–2 hours daily chasing writers for updates or reminders.
- Spends 2+ hours daily writing and explaining revision notes to writers.
- Handles 21–49 messages per working day related to article management alone.
- Commits 31–40 hours per week on editorial tasks.
- Takes 2+ weeks to complete a full publishing cycle.

FEELS

- Overwhelmed 31–40 hours of editorial work per week with no system, no automation, no support.
- Frustrated spending 2+ hours a day just chasing writers and explaining revisions that could be standardized.
- Anxious one missed deadline doesn't just affect one article, it cascades into the entire issue.
- Unsupported she is the only editor, and the workload scales with every new writer added.

- Vigilant but exhausted having to manually detect AI-generated content on top of everything else.
- Scattered managing 21–49 messages a day across 4+ platforms with no single source of truth.
- Responsible but isolated, she cares deeply about the magazine's quality but carries the entire editorial burden alone.

Say	Thinks
• "I need a dedicated Workspace for my tasks." • "Communication is fragmented across multiple apps." • "I want a more structured workflow and timeline." • "I'm unsure if the editor has even seen my submission." • "Feedback is sometimes somewhat vague or unclear." • "I've had to ask for clarification before proceeding with revisions." • "Personal schedule conflicts make meeting deadlines hard." • "I wait and assume it's fine if I hear nothing back."	• Did my editor miss my message in the 80% Messenger noise? • I wish I knew exactly what "Under Review" actually looks like. • Is the feedback coming today, or in 4–7 days like last time? • I'm probably guessing what they want in this revision round. • Why do I have to check Gmail, Viber, AND Messenger for one brief? • I'd be more productive if I didn't have to 'check in' myself. • I hope I'm not writing something that will be rejected later. • There has to be a better way to brainstorm than this.
Does	Feels
• Switches between 3+ communication platforms daily. • Starts writing even when the brief is only "mostly clear." • Frequently scrolls back through chat history to find buried notes. • Manually checks in with the editor to find out the article status. • Balances multiple assignments without a central progress tracker. • Resubmits work while still feeling a bit uncertain of requirements. • Uses Google Docs/Sheets as a makeshift organization tool. • Delays work due to waiting on a response from the editorial side.	• Anxious about missed messages getting "buried" in group chats. • Disconnected due to the lack of a "clear acknowledgment" (20%). • Frustrated by the "It varies a lot" feedback timeline (60%). • Uncertain when starting an article without a 100% clear brief. • Pressured by deadlines when the instructions arrive late. • Blind regarding where their work sits in the editorial pipeline. • Overwhelmed by the fragmentation of the current "manageable" system. • Under-supported when brainstorming and expanding on new ideas.

2. WRITERS

SAY

- I need a dedicated workspace for my tasks.
- Communication is fragmented across multiple apps.
- I want a more structured workflow and timeline.
- I'm unsure if the editor has even seen my submission.
- Feedback is sometimes somewhat vague or unclear.
- I've had to ask for clarification before proceeding with revisions.
- Personal schedule conflicts make meeting deadlines hard.
- I wait and assume it's fine if I hear nothing back.

THINKS

- Did my editor miss my message in the 80% Messenger noise?
- I wish I knew exactly what 'Under Review' actually looks like.
- Is the feedback coming today, or in 4–7 days like last time?
- I'm probably guessing what they want in this revision round.
- Why do I have to check Gmail, Viber, AND Messenger for one brief?
- I'd be more productive if I didn't have to 'check in' myself.

- I hope I'm not writing something that will be rejected later.
- There has to be a better way to brainstorm than this.

DOES

- Switches between 3+ communication platforms daily.
- Starts writing even when the brief is only mostly clear.
- Frequently scrolls back through chat history to find buried notes.
- Manually checks in with the editor to find out the article status.
- Balances multiple assignments without a central progress tracker.
- Resubmits work while still feeling a bit uncertain of requirements.
- Uses Google Docs/Sheets as a makeshift organization tool.
- Delays work due to waiting on a response from the editorial side.

FEELS

- Anxious about missed messages getting “buried” in group chats.
- Disconnected due to the lack of a “clear acknowledgment” (20%).
- Frustrated by the “It varies a lot” feedback timeline (60%).
- Uncertain when starting an article without a 100% clear brief.
- Pressured by deadlines when the instructions arrive late.
- Blind regarding where their work sits in the editorial pipeline.
- Overwhelmed by the fragmentation of the current “manageable” system.
- Under-supported when brainstorming and expanding on new ideas.

Say	Thinks
• “I need everything submitted on time — no exceptions, deadlines are deadlines” • “I don’t want to receive any issues by the time it reaches me for final review” • “If there are still errors when it gets to me, that means somewhere in the process broke down” • “The material needs to look good and read well before it goes to print — I won’t publish something that isn’t ready” • “I keep having to remind people about deadlines like it’s my full-time job” • “By the time I find out there’s a problem, it’s usually too late to fix it without delaying the whole issue” • “We need to start earning from what we publish — we can’t keep printing and posting for free” • “I want BOSS Magazine to look and feel like a professional international publication”	• “My job is to make sure what goes out represents BOSS Magazine at its best — if the quality isn’t there, it doesn’t go out” • “I shouldn’t have to be the one chasing deadlines every cycle — there should be a system that enforces this automatically” • “By the time an article reaches me, I still find things that should have been caught much earlier in the process” • “Every delay in the editorial side becomes my problem at the publishing side — I absorb everything that slips” • “We are doing the work of a real print and digital publication but with none of the tools or revenue that should come with it” • “I have no visibility into where things stand until the editor tells me — and sometimes that update comes too late” • “Printing costs money, time, and effort — I need the content to be right the first time, not after three rounds of fixing” • “If we had a proper platform, we could reach more people and actually earn from the content we work so hard to produce” • “I want this magazine to grow internationally but right now I can barely keep the local publishing schedule consistent”
Does	Feels
• Receives approved articles from the editor and conducts a final quality check before publishing — reviews layout, copy, visuals, and overall presentation • Personally ensures the material meets the magazine’s standards before it goes out — acts as the last checkpoint before print or post • Handles the physical printing and production of the magazine issue — coordinates print materials, layout formatting, and distribution • Publishes digital versions of content manually to social media platforms — Facebook and Instagram • Enforces publishing deadlines by personally following up with the editor and writers through chat • Formats and prepares content for both print and digital output manually with no dedicated publishing tool • Designs or sources cover visuals and layout elements using Canva or phone-based apps • Tracks the publishing schedule personally — through memory, notes, or informal reminders — with no formal calendar system • Manages the end-to-end production of each issue from content collection to final distribution • Has no system to monitor where each article is in the pipeline until the editor manually reports back	• Quality-protective — deeply invested in making sure nothing goes out unless it meets the magazine’s standard; takes personal ownership of what gets published under the BOSS name • Deadline-driven but unsupported — wants a clean, on-time publishing cycle every issue but has no system to enforce it and ends up doing it manually through follow-ups • Pressure-absorbing — every delay, every missed deadline, every last-minute issue from the editorial process lands on the publisher at the final stage with little time to fix it • Professionally proud — views the printed and published magazine as a direct reflection of her standards and reputation, which makes any quality slip feel personal • Financially frustrated — puts significant effort and cost into printing and producing each issue with no revenue mechanism in place to sustain or grow the operation • Isolated at the finish line — operates as the sole person responsible for final quality and production, with no system, no dashboard, and no real-time view of what is coming her way • Growth-oriented but grounded — wants to scale BOSS Magazine into an international brand but knows the current manual, informal publishing process cannot support that ambition without a major structural change

3. PUBLISHER

SAY

- I need everything submitted on time no exceptions, deadlines are deadlines.
- I don't want to receive any issues by the time it reaches me for final review.
- If there are still errors when it gets to me, that means somewhere in the process broke down.
- The material needs to look good and read well before it goes to print I won't publish something that isn't ready.
- I keep having to remind people about deadlines like it's my full-time job.
- By the time I find out there's a problem, it's usually too late to fix it without delaying the whole issue.
- We need to start earning from what we publish we can't keep printing and posting for free.
- I want BOSS Magazine to look and feel like a professional international publication.

THINKS

- My job is to make sure what goes out represents BOSS Magazine at its best if the quality isn't there, it doesn't go out.
- I shouldn't have to be the one chasing deadline every cycle there should be a system that enforces this automatically.
- By the time an article reaches me, I still find things that should have been caught much earlier in the process.
- Every delay in the editorial side becomes my problem at the publishing side I absorb everything that slips.
- We are doing the work of a real print and digital publication but with none of the tools or revenue that should come with it.
- I have no visibility into where things stand until the editor tells me and sometimes that update comes too late.
- Printing costs money, time, and effort I need the content to be right the first time, not after three rounds of fixing.
- If we had a proper platform, we could reach more people and actually earn from the content we work so hard to produce.
- I want this magazine to grow internationally but right now I can barely keep the local publishing schedule.

DOES

- Receives approved articles from the editor and conducts a final quality check before publishing reviews layout, copy, visuals, and overall presentation.
- Personally, ensures the material meets the magazine standards before it goes out acts as the last checkpoint before printing or post.
- Handles the physical printing and production of the magazine issue coordinates print materials, layout formatting, and distribution.
- Publishes digital versions of content manually to social media platforms Facebook and Instagram.
- Enforces publishing deadlines by personally following up with the editor and writers through chat.

- Formats and prepares content for both print and digital output manually with no dedicated publishing tool.
- Designs or sources cover visuals and layout elements using Canva or phone-based apps.
- Tracks the publishing schedule personally through memory, notes, or informal reminders, with no formal calendar system.
- Manages the end-to-end production of each issue from content collection to final distribution.
- Has no system to monitor where each article is in the pipeline until the editor manually reports back.

FEELS

- Quality-protective deeply invested in making sure nothing goes out unless it meets the magazine's standard; takes personal ownership of what gets published under the BOSS name.
- Deadline-driven but unsupported wants a clean, on-time publishing cycle on every issue but has no system to enforce it and ends up doing it manually through follow-ups.
- Pressure-absorbing every delay, every missed deadline, and every last-minute issue from the editorial process lands on the publisher at the final stage with little time to fix it.
- Professionally proud views the printed and published magazine as a direct reflection of her standards and reputation, which makes any quality slip feel personal.
- Financially frustrated puts significant effort and cost into printing and producing each issue with no revenue mechanism in place to sustain or grow the operation.
- Isolated at the finish line operates as the sole person responsible for final quality and production, with no system, no dashboard, and no real-time view of what is coming her way.
- Growth-oriented but grounded wants to scale BOSS Magazine into an international brand but knows the current manual, informal publishing process cannot support that ambition without a major structural change.

Say	Thinks	
• "I need all the final articles in one place before I can start laying out the magazine." • "When the content keeps changing after I've already placed it, I have to redo everything from scratch." • "I don't always know if what I'm designing matches what the editor actually wants." • "Nobody tells me the deadline until it's almost here." • "I spend more time waiting for content than actually designing." • "Sometimes I get feedback that's too vague — "make it look better" doesn't tell me what to fix." • "I want to know upfront what the theme and feel of the issue is supposed to be." • "If the articles are still being revised, I can't finalize the layout — everything depends on the content being locked."	• Will the editor send me the final versions today, or will I be waiting again? • How am I supposed to design a layout when the article count keeps changing? • I hope what I submit doesn't come back with a completely different direction this time. • Is there a style guide I should be following, or am I just guessing the aesthetic again? • Why is the deadline moved up when the content wasn't even finalized yet? • No one tells me when articles get added or removed — I only find out when I check myself. • I'd work faster if I had the text and photos ready at the same time instead of getting them piece by piece. • Am I supposed to wait for the editor to approve the layout before moving to the next section, or just keep going?	
Does	Graphic designer	Feels
• Waits for all articles to be finalized by the editor before beginning the magazine layout. • Receives articles and assets through chat or email — no centralized file handoff system. • Manually arranges text, images, and visuals across multiple pages using design tools. • Revises layout multiple times based on feedback from the editor with no version tracking. • Checks in with the editor or publisher to find out if the content count or direction has changed. • Works without a formal design brief — interprets the magazine's visual identity based on past issues. • Adjusts layout from scratch whenever last-minute article changes or additions come in. • Submits completed layout back to the editor through chat for review and confirmation. • Manages multiple layout versions manually with no shared workspace or feedback tool.	• Dependent — cannot start or finish work without the content being locked and delivered first. • Uninformed — rarely receives a proper design brief or issue theme upfront, so a lot of decisions are made by guessing. • Frustrated — redoing layouts from scratch when articles change late in the process wastes significant time and effort. • Disconnected — has no visibility into the editorial pipeline, so changes always come as a surprise. • Pressured — absorbs deadline pressure from the editorial side even when delays were caused upstream. • Undervalued — design feedback is often vague or focuses on content rather than the visual work itself. • Isolated — works independently without a shared tool where the editor can leave layout-specific comments directly. • Motivated but constrained — wants to produce a polished, professional-looking magazine but the disorganized handoff process makes consistent quality hard to achieve.	

4. GRAPHIC DESIGNER

SAY

- I need all the final articles in one place before I can start laying out the magazine.
- When the content keeps changing after I've already placed it, I have to redo everything from scratch.
- I don't always know if what I'm designing matches what the editor actually wants.
- Nobody tells me the deadline until it's almost here.
- I spend more time waiting for content than actually designing.
- Sometimes I get feedback that's too vague 'make it look better' doesn't tell me what to fix.
- I want to know upfront what the theme and feel of the issue is supposed to be.
- If the articles are still being revised, I can't finalize the layout, everything depends on the content being locked.

THINKS

- Will the editor send me the final versions today, or will I be waiting again?
- How am I supposed to design a layout when the article count keeps changing?
- I hope what I submit doesn't come back with a completely different direction this time.
- Is there a style guide I should be following, or am I just guessing the aesthetic again?
- Why is the deadline moved up when the content wasn't even finalized yet?
- No one tells me when articles get added or removed, I only find out when I check myself.
- I'd work faster if I had the text and photos ready at the same time instead of getting them piece by piece.
- Am I supposed to wait for the editor to approve the layout before moving to the next section, or just keep going?

DOES

- Waits for all articles to be finalized by the editor before beginning the magazine layout.
- Receives articles and assets through chat or email with no centralized file handoff system.
- Manually arranges text, images, and visuals across multiple pages using design tools.
- Revises layout multiple times based on feedback from the editor with no version tracking.
- Checks in with the editor or publisher to find out if the content count or direction has changed.
- Works without a formal design brief interprets the magazine's visual identity based on past issues.
- Adjusts layout from scratch whenever last-minute article changes or additions come in.
- Submits completed layout back to the editor through chat for review and confirmation.
- Manages multiple layout versions manually with no shared workspace or feedback tool.

FEELS

- Dependent cannot start or finish work without the content being locked and delivered first.
- Uninformed rarely receives a proper design brief or issue theme upfront, so a lot of decisions are made by guessing.
- Frustrated redoing layouts from scratch when articles change late in the process wastes significant time and effort.
- Disconnected has no visibility into the editorial pipeline, so changes always come as a surprise.
- Pressure absorbs deadline pressure from the editorial side even when delays were caused upstream.
- Undervalued design feedback is often vague or focuses on content rather than the visual work itself.
- Isolated works independently without a shared tool where the editor can leave layout-specific comments directly.
- Motivated but constrained wants to produce a polished, professional-looking magazine, but the disorganized handoff process makes consistent quality hard to achieve.

Pain-Gain Analysis Summary**Pains**

- Fragmented communication, unclear workflows, and lack of a centralized system cause delays, confusion, and repeated follow-ups.
- Manual tracking, revisions, deadline chasing, and late content changes create heavy workloads and unnecessary rework.
- Limited visibility, unclear feedback, and lack of standardized processes make it difficult for writers, editors, publishers, and designers to coordinate effectively.
- The current manual process makes it difficult to maintain consistent quality, meet publishing schedules, scale operations, and generate digital revenue.

Gains

- Achieves a centralized and structured publishing workflow where articles, files, feedback, deadlines, and progress are managed in one place.
- Reduces delays, repetitive work, missed deadlines, and unnecessary revisions through automation, notifications, tracking, and standardized feedback.
- Improves collaboration and transparency by giving writers, editors, publishers, and designers clear visibility of article status, responsibilities, and deadlines.
- Improves content quality, publishing efficiency, scalability, and opportunities for digital monetization while reducing workload and operational stress.

Analysis

- They share a common need for better coordination, visibility, efficiency, and centralized workflow management.
- The root cause of their pains is the reliance on fragmented communication channels, manual processes, unclear responsibilities, and lack of a centralized publishing system.
- A centralized, user-friendly platform with workflow tracking, automated reminders, structured feedback, file management, version control, and clear article statuses can reduce delays and workload.
- Improving the publishing process benefits the entire team by increasing productivity, reducing errors and rework, improving collaboration, maintaining quality, and supporting the magazine's long-term digital growth and revenue goals.

Stage 2: Define

The second phase of the design thinking process is by putting together all the information gathered from the previous phase. This stage is to define the problem statements clearly which should be based on human-centered needs, rather than being focused on the business goal.

Clustered Problem

Core Problem Statement BOSS Magazine PH is a comprehensive Filipino lifestyle publication, covering a broad spectrum of topics including fashion, culture, modern trends, business, and society, that currently operates without any dedicated digital infrastructure to support its editorial, publishing, or revenue functions.

Cluster 1: Workflow & Operational Inefficiencies

- **Fragmented Editorial Workflow:** There is no centralized system; submissions come from multiple platforms, making tracking and management difficult.

- **Time-Consuming Manual Reviews:** All articles are reviewed manually, taking hours daily and increasing editor workload.

Cluster 2: Business & Scalability Constraints

- **Zero Revenue Generation:** There are no subscriptions, paywalls, or monetization strategies in place despite continuous content production.
- **Inability to Scale Operations:** The growing workload causes delays, inefficiency, and difficulty in handling an increased volume of content.

Cluster 3: Infrastructure & Brand Limitations

- **No Dedicated Digital Platform:** The current reliance on Facebook limits the publication's reach, discoverability, and professional credibility.

Release Plan

Updated Release Plan and Progress. The Novus release plan follows the three-phase academic delivery schedule. Release 1, completed during MNTSDEV, delivered the system requirements, Product Backlog, use cases, system design, and prototypes. Release 2 under MSYSADD will focus on detailed architecture and development while Release 3 under MCSPROJ will deliver the coded and integrated system, followed by testing, UAT, and production deployment.

The project is currently on schedule. The major deliverables for the MNTSDEV phase have been completed

Access the full release plan through the following link:

[Novus Release plan - .docx](#)

Prototype

Wireframes

The wireframes of the proposed system were developed to visualize the interface layout and navigation flow for each user role prior to implementation. Fig. 1 shows the wireframe of the login page, which serves as the entry point for all users. Figs. 2–6 present the wireframe designs for the writer screens, while Figs. 7–12 illustrate the wireframes for the editor screens. Figs. 13–14 show the wireframe designs for the graphic designer screens, and Figs. 15–16 present those for the publisher screens. Lastly, Figs. 17–21 illustrate the wireframes for the reader screens, representing the overall layout of the system's website interface.



Welcome back!

Log in to continue to your workspace.

EMAIL

PASSWORD

[Forgot password?](#)

Log in

OR LOG IN AS

 Writer

 Editor

 Publisher

 Designer

 Reader



 Dashboard

 Submissions

 Articles

 Report

 Messages

 Settings

01 WRITER DASHBOARD

WRITER

Welcome back, Writer!

Here's what's happening with your submissions.



3

ACTIVE

Submissions in progress



1

REVISION

Awaiting your revisions



5

APPROVED

Ready for the next stage

MY LATEST ACTIVITY

[VIEW ALL](#)

	AI in Everyday Workspaces	2 hours ago	SUBMITTED	>
	Sustainable Tech in 2026	1 day ago	UNDER REVIEW	>
	The Future of Digital Publishing	2 days ago	DRAFT	>
	Creator Economy: The Next Wave	3 days ago	DRAFT	>

[+ NEW SUBMISSION](#)





Dashboard

Submissions

Articles

Report

Messages

Settings



02 ARTICLE SUBMISSION FORM

WRITER

ARTICLE TITLE

Enter article title

SECTION / CATEGORY

Select category

WORD COUNT TARGET

Enter word count

Suggested range: 400 - 1,500 words

ARTICLE BODY

Paragraph B I U

Write or paste your article here...

We've got your back!

Your article will be automatically checked for readability and grammar before review.

TIPS FOR A GREAT ARTICLE

- Use clear and concise language
- Structure your content with headings
- Include relevant images and links
- Proofread before submitting

SAVE DRAFT

SUBMIT FOR REVIEW

Articles will be automatically checked for readability and grammar.



Dashboard

Submissions

Articles

Report

Messages

Settings



Studio > Draft Review

Mode: Grammar & Read

Article Editor

Paragraph B I U

As organizations adapt to lean operations, AI is no longer just a tool—it is becoming the core engine.

Leaders must evaluate how...

(Grammar: Passive voice detected)

AI Optimization Panel



Readability Score

Grade Level: 10.2 (Good)



Flesch Reading Ease: 65



Grammar & Syntax (2)

- Issue #1: Passive voice

Suggest Active Fix

- Issue #2: Typo found

Accept Correction

Apply All AI Fixes

Words: 132 Characters: 762

All changes saved



Dashboard

Submissions

Articles

Report

Messages

Settings

Writer
writer@bosimedia.ph



Dashboard

Submissions

Articles

Report

Messages

Settings

Writer
writer@bosimedia.ph

04 SUBMISSION STATUS VIEW

WRITER

ALL (5)

ACTIVE (3)

UNDER REVIEW (1)

REVISION (1)

APPROVED (5)

ARTICLE (TITLE)

STATUS



AI in Everyday Workspaces

Submitted • May 15, 2025

UNDER REVIEW



Sustainable Tech in 2026

Submitted • May 14, 2025

REVISION



The Future of Digital Publishing

Submitted • May 12, 2025

APPROVED



Creator Economy: The Next Wave

Draft • May 10, 2025

DRAFT



Remote Work Best Practices

Draft • May 9, 2025

DRAFT

05 REVISION NOTES INBOX

WRITER

REVISION NOTES FROM EDITOR

AI in Everyday Workspaces

May 15, 2025 • 10:30 AM

Sustainable Tech in 2026

May 14, 2025 • 02:15 PM

From: Editor Team

Editor

May 15, 2025 • 10:30 AM



PARAGRAPH 2

Great point! Please add a real-world example to support this.



CONCLUSION

Consider strengthening the conclusion with a call to action.

VIEW FULL ARTICLE

REVISE ARTICLE



Dashboard

Articles

Report

Comments

Team

Settings

Editor

editor@novus.com

01

EDITOR DASHBOARD

EDITOR

Good day, Editor!

Here's an overview of your workflow.

10

Articles Total

4

Under Review

2

Approved

PENDING REVIEWS

View all

AI

AI in Everyday Workspaces

Submitted by Writer • May 15, 2025

91

Score

ST

Sustainable Tech in 2026

Submitted by Writer • May 14, 2025

74

Score

FT

The Future of Digital Publishing

Submitted by Writer • May 13, 2025

68

Score



Dashboard

Articles

Report

Comments

Team

Settings

Editor

editor@novus.com

02

ARTICLE REVIEW SCREEN

EDITOR

The Future of Digital Publishing

Under Review

Submitted by Writer • May 13, 2025 at 09:41 AM

84

Overall Score

ARTICLE CONTENT

AI REPORT SUMMARY

Grammar

88

Readability

84

REQUEST REVISIONS

APPROVE ✓



Dashboard

Articles

Report

Comments

Team

Settings

Editor

editor@novuscms.com

03

REVISION NOTE COMPOSER

EDITOR

SEND REVISION NOTE

SELECT SECTION

Select section to focus on...

NOTE TYPE

GRAMMAR

TONE

STRUCTURE

FACTUAL

REVISION INSTRUCTION

Provide specific feedback or instructions for the writer...

0/500

PRIORITY

Select priority level...

+ ADD ANOTHER NOTE

SEND TO WRITER



Dashboard

Articles

Report

Comments

Team

Settings

Editor

editor@novuscms.com

04

APPROVAL SCREEN

EDITOR

ARTICLE APPROVED

This article is approved and ready for the next steps.

ARTICLE PREVIEW

AI SCORE

91

Excellent

Readability

Grammar

91

93

REVISION HISTORY

1 revision round • May 15, 2025 at 10:30 AM

SEND TO PUBLISHER

Notify publisher to review and publish.

SEND TO GRAPHIC DESIGNER

Notify designer for visuals and layout



Dashboard

Articles

Report

Comments

Team

Settings



Editor
editor@novus.com

05

DEADLINE MANAGEMENT PANEL

EDITOR

DEADLINE PRESET

Select deadline template...

FOR ARTICLES ASSIGNED

AI in Everyday Workspaces	MAY 18, 2025	3 DAYS LEFT
Sustainable Tech in 2026	MAY 16, 2025	1 DAY LEFT
The Future of Digital Publishing	MAY 20, 2025	5 DAYS LEFT
Creator Economy: The Next Wave	MAY 22, 2025	7 DAYS LEFT

AUTO REMINDER

Send reminders 24 hours before the deadline.

SAVE DEADLINES

06

PIPELINE OVERVIEW

EDITOR

SEARCH & FILTER PIPELINE

Search articles...

All Status

DRAFT

REVIEW

APPROVED

ARTICLE	STATUS
AI in Everyday Workspaces	APPROVED
Sustainable Tech in 2026	APPROVED
The Future of Digital Publishing	APPROVED
Remote Work Best Practices	APPROVED
Creator Economy: The Next Wave	APPROVED

5 of 10 articles approved



Dashboard

Articles

Team

Layout Handoff

Settings

Designer

designer@novus.com

01

LAYOUT HANDOFF SCREEN

DESIGNER

NOVUS · DESIGNER · [NAME]

Welcome back! Here are the approved layouts ready for handoff.

APPROVED ARTICLES — READY FOR LAYOUT



AI in Everyday Workspaces

Last updated: May 15, 2025 · Approved v2

All content final and approved

ARTICLE

PHOTOS: 8

DOWNLOAD



Sustainable Tech in 2026

Last updated: May 14, 2025 · Approved v1

All content final and approved

ARTICLE

PHOTOS: 12

DOWNLOAD

All files are final approved versions.
Download the full article package including content brief, assets, and guidelines.

02

LAYOUT SUBMISSION SCREEN

DESIGNER

SUBMIT COMPLETED LAYOUT

ISSUE / ARTICLE

Issue #12

UPLOAD LAYOUT FILE

Drag and drop your file here

PDF or INDD files (Max 100MB)

LAYOUT VERSION

v1.0

NOTES TO EDITOR

Add notes about the layout...

0/500

SUBMIT LAYOUT FOR EDITOR REVIEW

You will be notified once the editor reviews your submission.



Dashboard

Articles

Report

Comments

Team

Settings

Publisher
publisher@novus.com



Dashboard

Articles

Report

Comments

Team

Settings

Publisher
publisher@novus.com

01 PUBLISHER PIPELINE VIEW

PUBLISHER

PN

NOVUS · PUBLISHER · [NAME]

Welcome back! Here's your publishing pipeline overview.



6

APPROVED

Ready to publish



4

IN PROGRESS

Awaiting finalization

ISSUE READINESS

6 of 10 articles ready

60%

LAYOUT STATUS

7 of 10 articles in layout

70%

PUBLISHING DATE

May 16, 2026

This is the target date for the selected issue.

DAYS REMAINING



2

days remaining

Until publishing deadline

02 PUBLISHING CONFIRMATION

PUBLISHER

FINAL SIGN-OFF · ISSUE #12



Issue #12

May 16, 2026

READY TO PUBLISH

FORMAT: Digital & Print

LANGUAGE: English

CHECKLIST



Editorial review completed

DONE



Layout and design approved

DONE



All assets and files verified

DONE

PUBLISH TO



Website



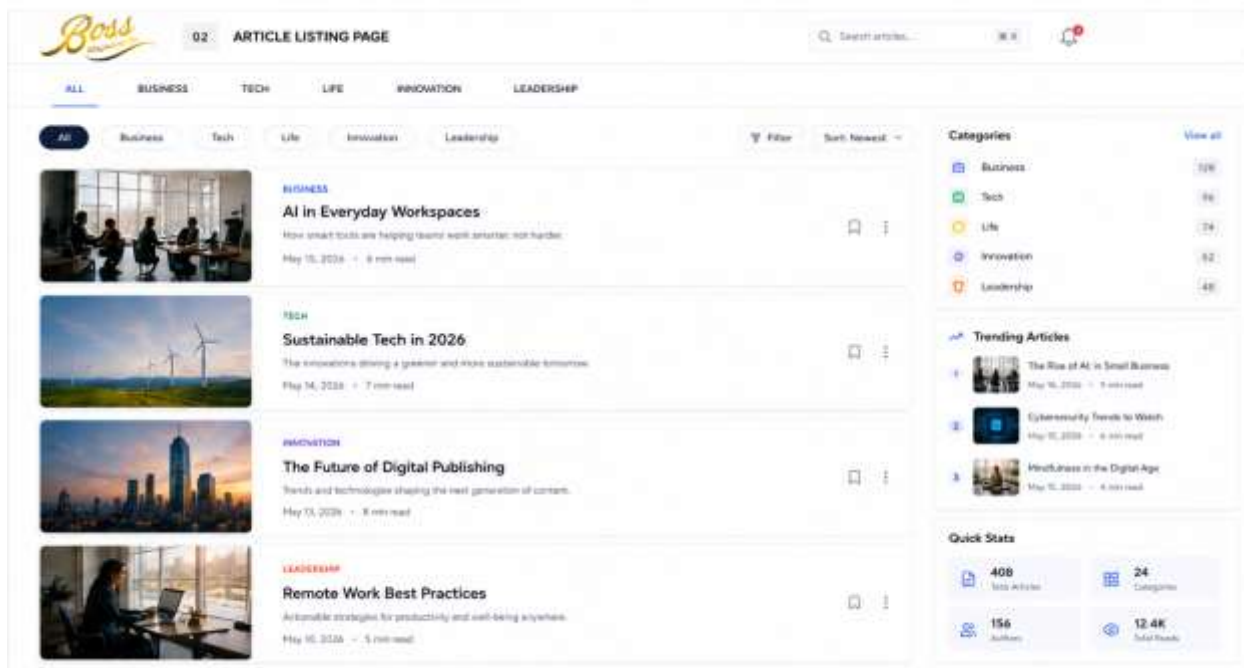
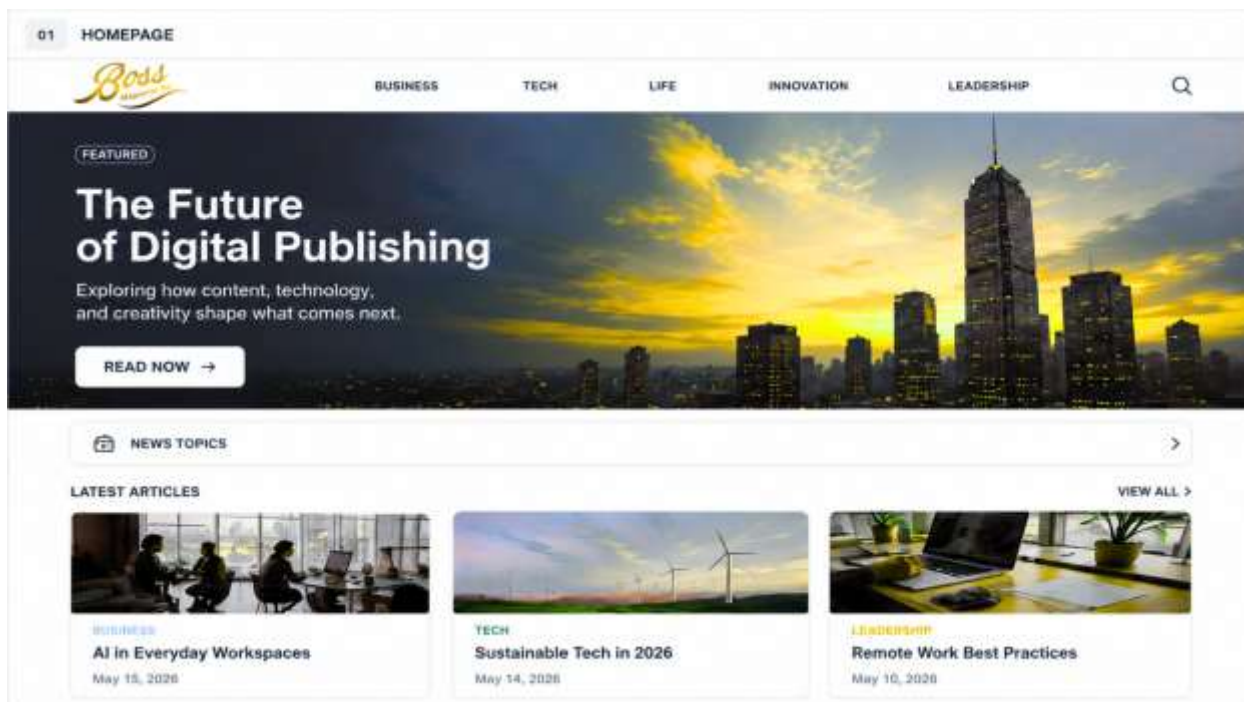
Social Media

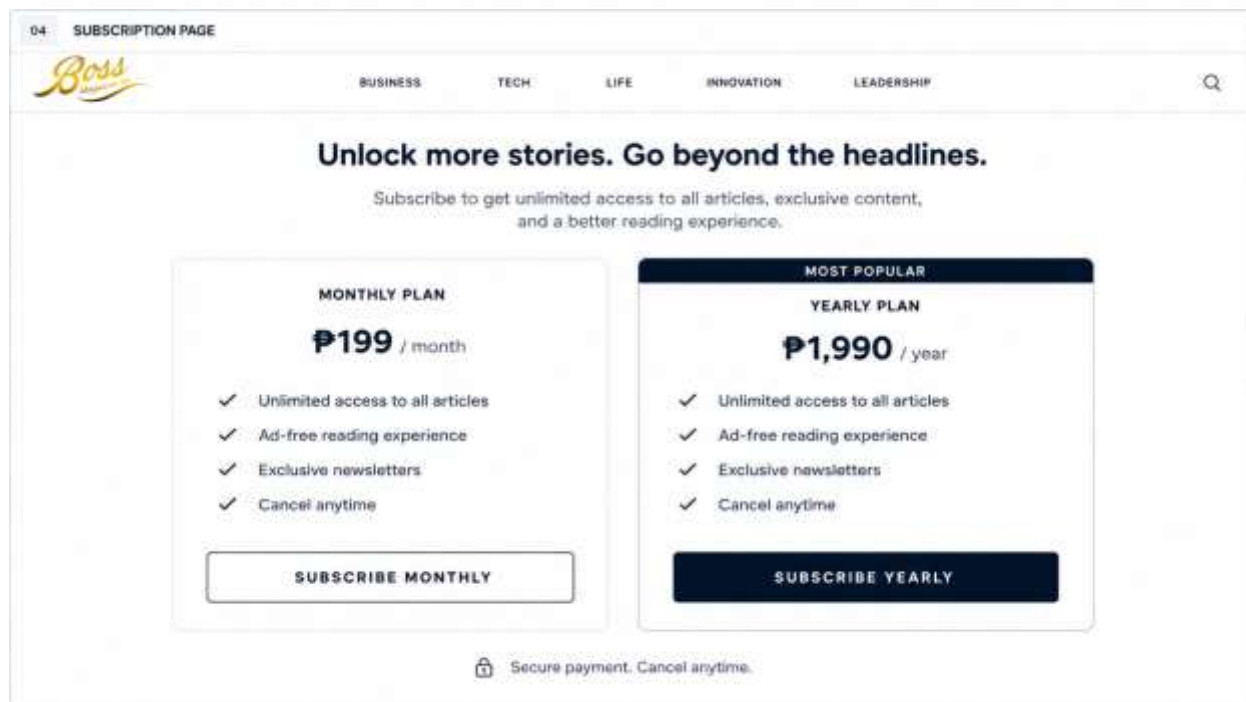
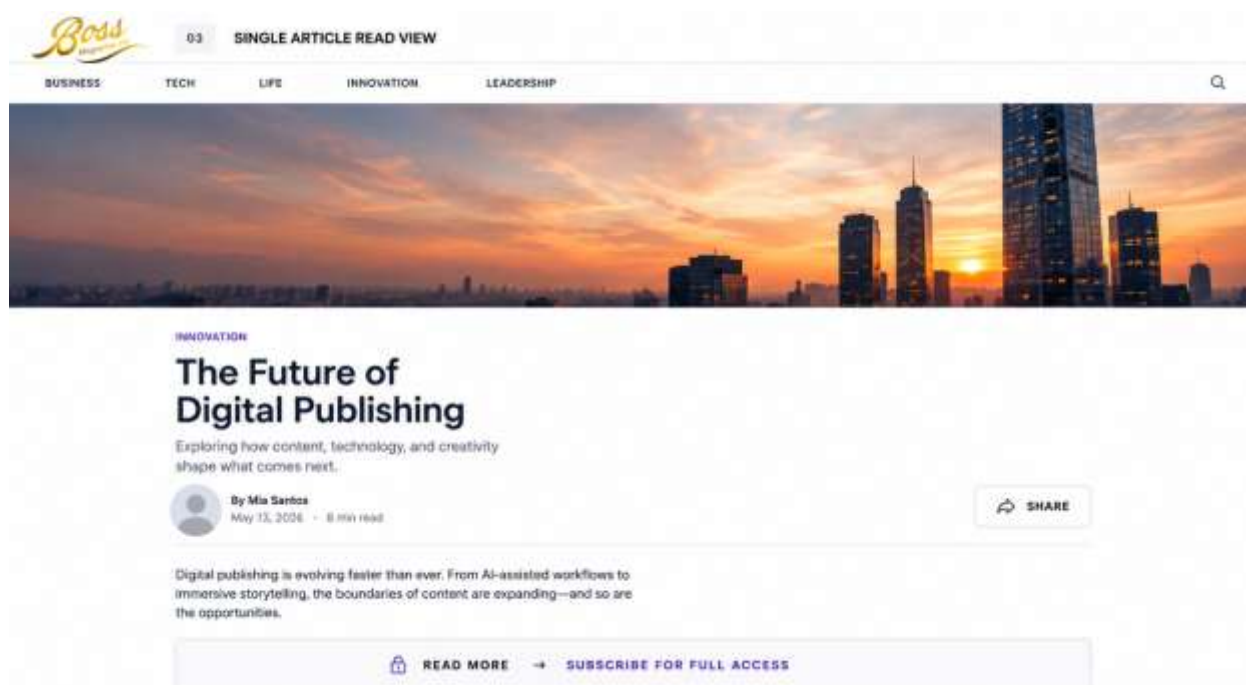


CONFIRM AND PUBLISH ISSUE #12



Once published, the issue will be live and visible to the public.





05 PAYMENT CHECKOUT



BUSINESSTECHLIFEINNOVATIONLEADERSHIP

Order Summary

Plan	Yearly Plan
Billing	₱1,990.00 / year
Total	₱1,990.00

Why subscribe?

- ✓ Unlimited access to all articles
- ✓ Ad-free reading experience
- ✓ Exclusive newsletters
- ✓ Cancel anytime

 **Secure & encrypted payment**
Your payment information is safe with us.

Payment Method

 GCash
 Maya
 Credit Card
 Debit Card

GCash Payment

GCash Number

11-digit mobile number

Account Name

Account name will appear in your GCash app

 Please make sure to send the exact amount: **₱1,990.00** to confirm your payment.

Scan to Pay



Scan this QR code using your GCash app

 **I HAVE MADE THE PAYMENT**

By completing your purchase, you agree to our [Terms of Service](#) and [Privacy Policy](#).

Storyboard

To illustrate the problems addressed by the system and how each feature resolves them, storyboards were developed depicting real-world user scenarios. Fig. 22 presents the storyboards for the editor's AI pre-screening engine and sprint board. Fig. 23 shows the storyboards for the graphic designer's file submission and the writer's centralized drafting workspace. Fig. 24 illustrates the storyboards for the writer's deadline tracking and contextual revision notes features.

NOVUS - STORYBOARDS (PROTOTYPE)

STORYBOARD 1: Editor – AI Pre-Screening Engine



STORYBOARD 2: Editor – Sprint Board



NOVUS - STORYBOARDS (PROTOTYPE)

STORYBOARD 3: Graphic Designer – File Submission



STORYBOARD 4: Writer – Centralized Drafting Workspace



NOVUS - STORYBOARDS (PROTOTYPE)



Use Classes and Characteristics

<i>Roles</i>	<i>Description</i>
<i>Admin</i>	The Administrator user class represents platform managers responsible for system governance, user account management, and role-based access control. Admins maintain oversight of the centralized editorial workflow and access administrative dashboards to monitor subscription revenue, cloud infrastructure costs, and newsroom productivity metrics.
<i>Publisher</i>	The publisher oversees the entire editorial and publishing operations of BOSS Magazine PH. This user has the highest level of access within the internal system, responsible for setting issue deadlines, publishing approved content to the public website, and monitoring subscription revenue and web analytics.
<i>Editor</i>	The editor manages the editorial workflow end-to-end. This user creates and assigns article briefs to writers, reviews submitted drafts, requests revisions or approves final articles, and forwards approved content to the client for sign-off before publication.
<i>Writer</i>	The writer is a content creator assigned to produce articles for the magazine. This user receives assignment briefs through the platform, drafts and submits articles for editorial review, and responds to revision notes until the article is approved.
<i>Graphics Designer</i>	The graphics designer is responsible for the visual production of the magazine. This user receives approved articles and their materials, uploads layout previews and asset versions, and submits the final PDF bundle for editorial and client review.
<i>Reader</i>	A reader is a member of the public who accesses the BOSS Magazine PH public portal. This user can browse articles by category, toggle between English and

	Filipino content, interact with the AI chat assistant, and opt into the email newsletter. Readers have access to free-tier content only.
<i>Subscriber</i>	A subscriber is a reader who has completed registration and payment to access premium content. This user inherits all capabilities of a standard reader and additionally gains access to paywalled articles, full digital issues, bookmarking, and subscription management through their profile.

Conclusion

(

Write what you have accomplished so far (which of your objectives have been achieved and what needs to be done to achieve the others, if there are still any left). Summarize what you have done: specify the objectives and briefly describe how it was achieved. At the end of this chapter, write what will be done for the next iteration (for the next course in the PBL flow of subjects—MCSPROJ)

)

References

(

All ideas which are not yours should be properly cited. To insert a citation,

1. *Put the cursor on the location where you want to put the citation*
2. *Go to the **References** tab and click on **Insert Citation**. Fill in the required details. Use **IEEE** as the **Style**.*

To insert the list of references in this section,

1. *Go to the **References** tab and click on **Bibliography**.*
2. *Choose **References**.*
3. *Adjust the heading of this section accordingly.*

)

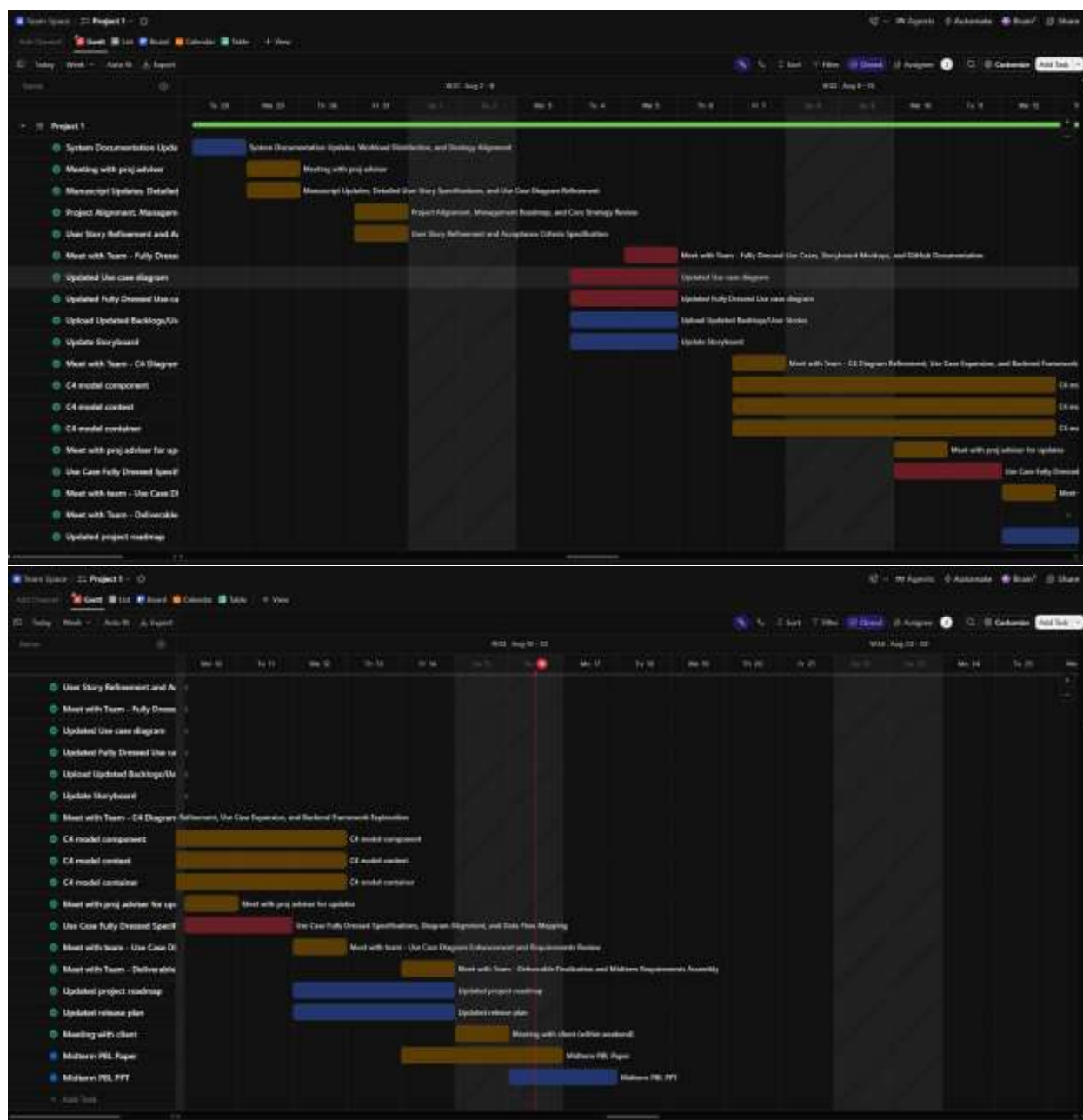
Appendices

Appendix A: Project Vision

Write your updated project vision, please refer to the reference uploaded in MS Teams

Appendix B: Schedule

Gantt chart or table containing your project schedule.



Appendix C: Product Roadmap



Appendix D: Users' Manual

(

Add this section only if you think Section 5.2 is not sufficient for users to grasp how your system works.

)

Appendix E: Team Meetings

(

Include here screenshots of meetings with adviser/consultants (please ask permission), and if possible, minutes of your meetings

)

Appendix F: Source Code

(
Screenshot and link of GitHub repository.
)

Appendix G: Finalized Design Thinking Deliverables

(

Screenshot of finalized design thinking deliverables

)